13/10/2026 - Avisos:

1. Parcial 1 (Octubre 22 / Horas de clase)

Quiz → 1 sola oportunidad + Tiempo limitado

(Temas: Modulo 1 + Modulo 2)

https://github.com/udea-so/apuntes_2026-2_SO-virtual

2. Inicio del Módulo 3

Apuntes – Clase 14

Threads

Sistemas Operativos

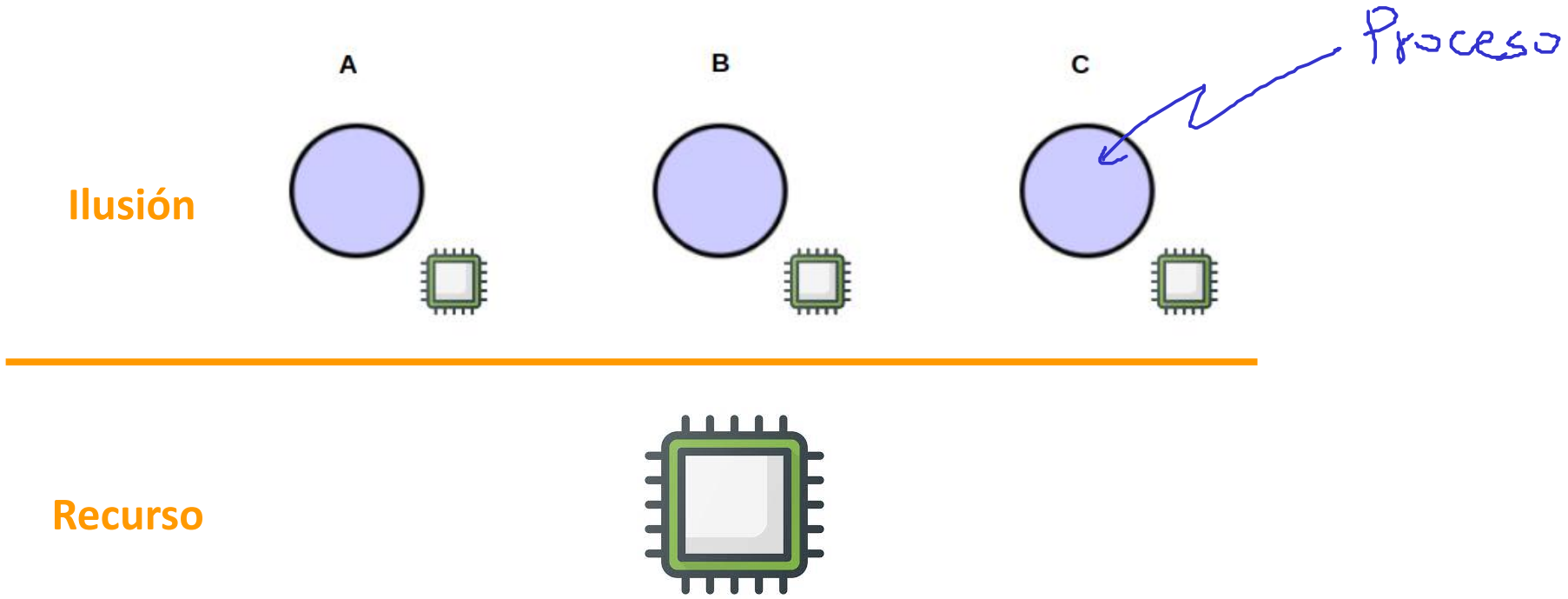
Universidad de Antioquia

Facultad de Ingeniería

Ingeniería de Sistemas

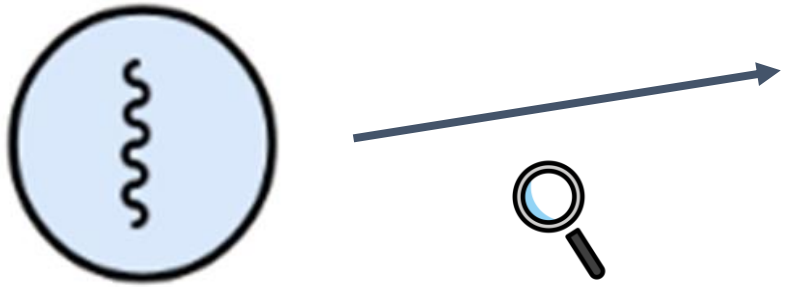
Resumen virtualización (Modulos 1 y 2)

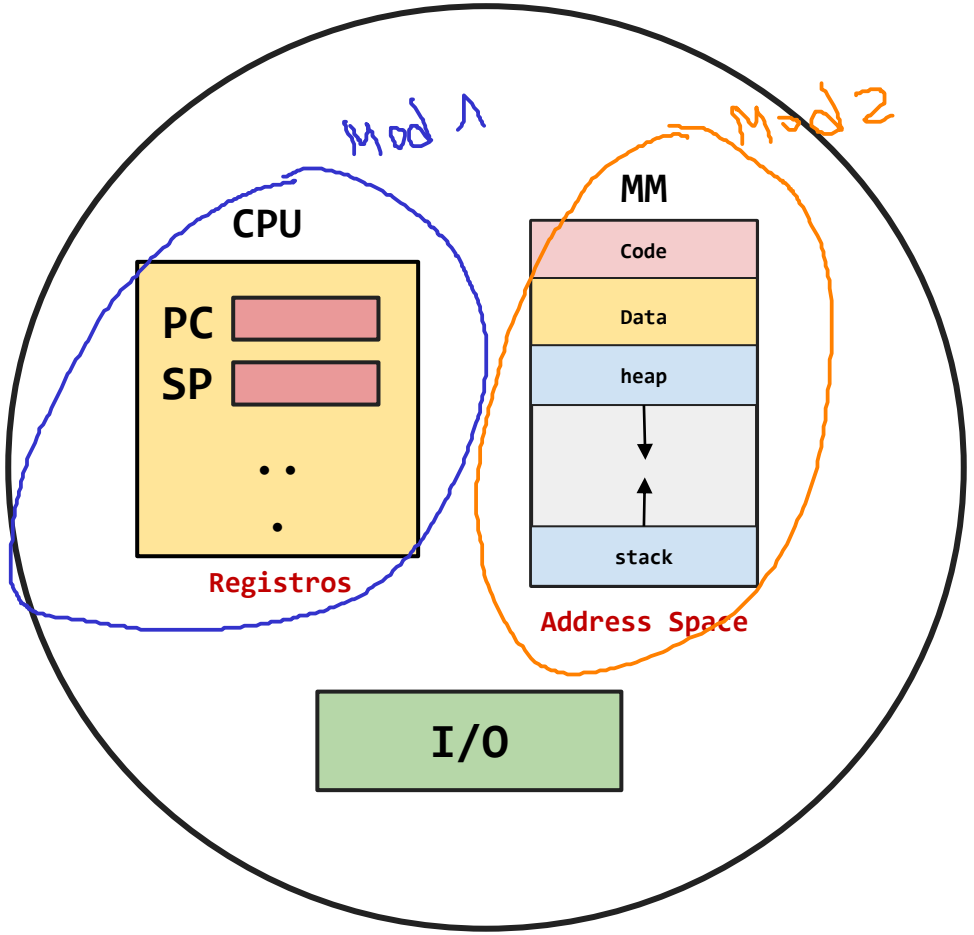
Virtualización de CPU (Modulo 1)



Resumen virtualización

Virtualización de CPU - Abstracción: Procesos



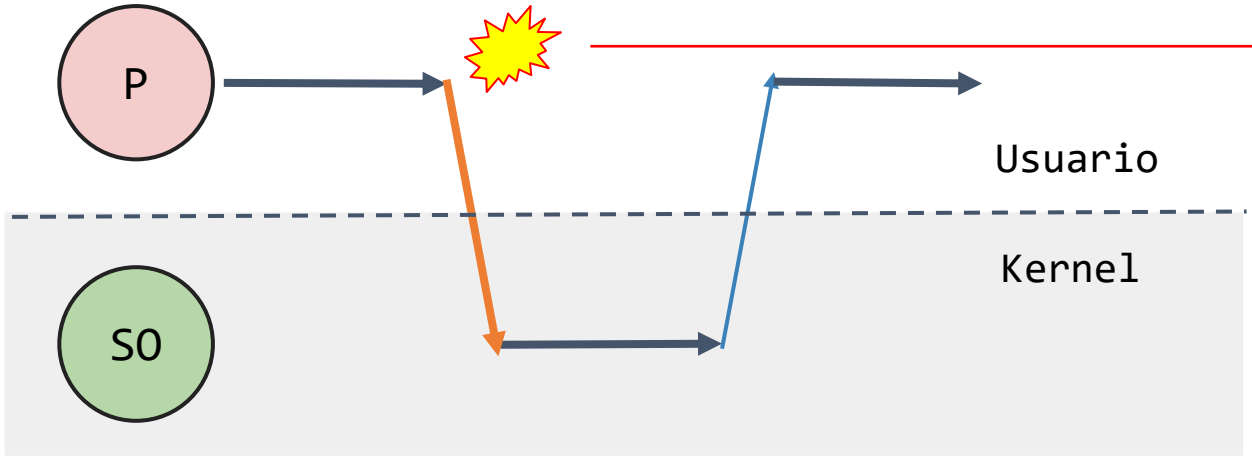



```
Proceso = CPU + Memory + I/O Info
```

Resumen virtualización

Virtualización de CPU - Mecanismos: LDE

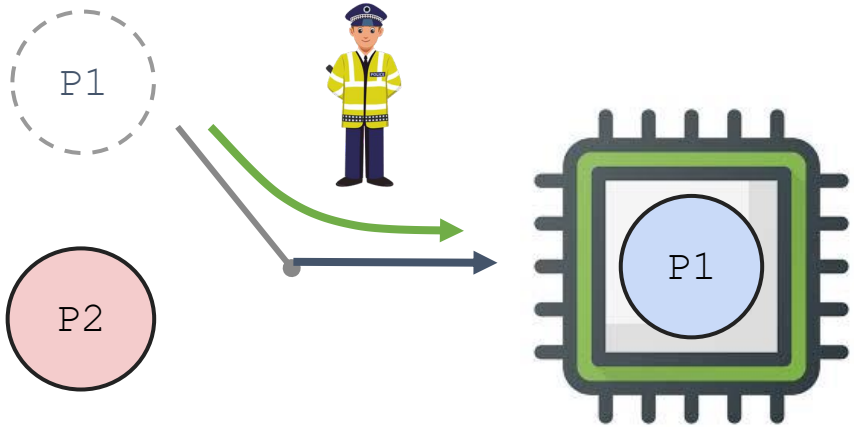
Limited



Eventos:

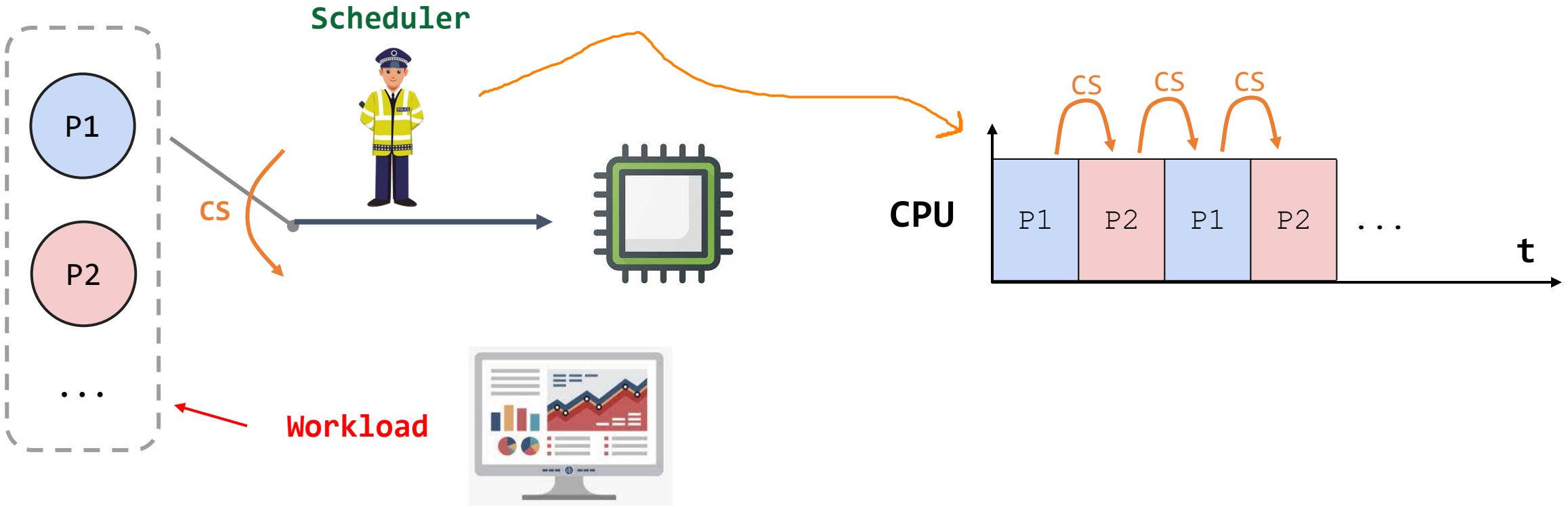
- Excepción
- Syscall
- Interrupción

Direct Execution



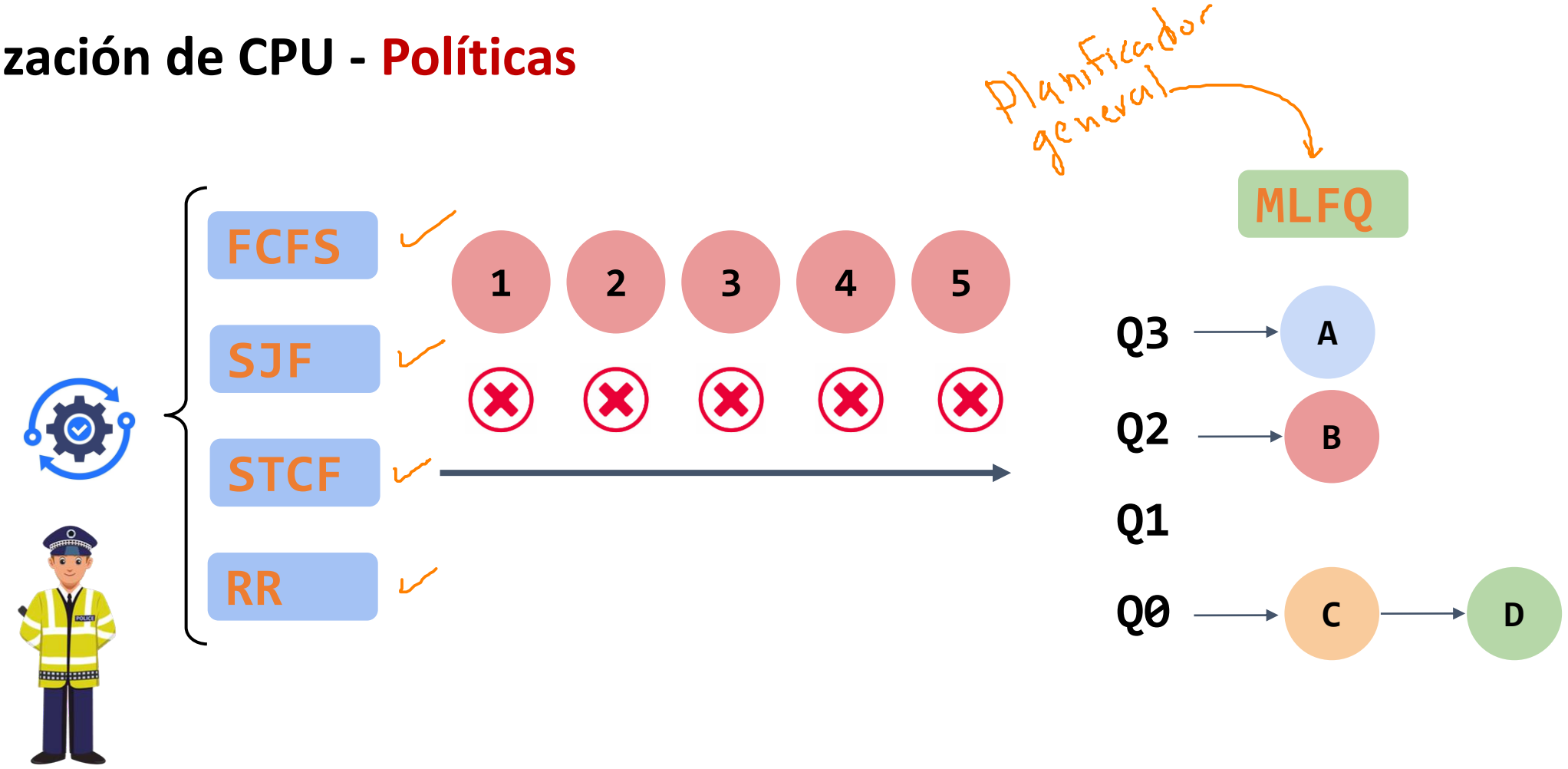
Resumen virtualización

Virtualización de CPU - Políticas



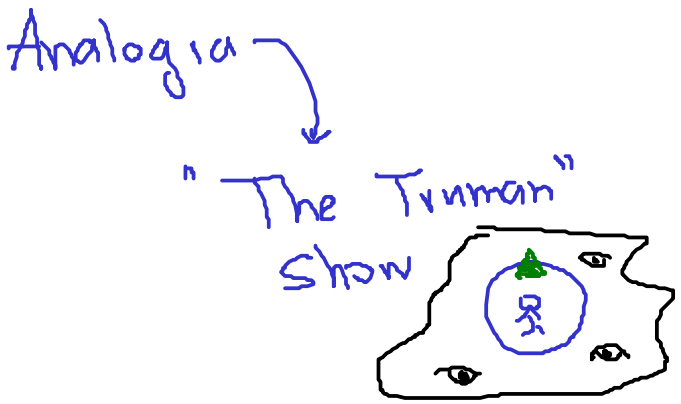
Resumen virtualización

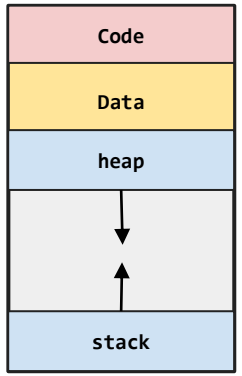
Virtualización de CPU - Políticas

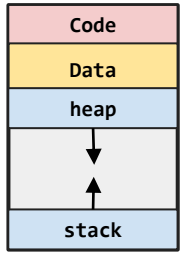


Resumen virtualización

Virtualización de memoria (Modulo 2)



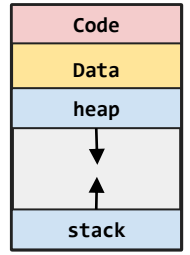






P0

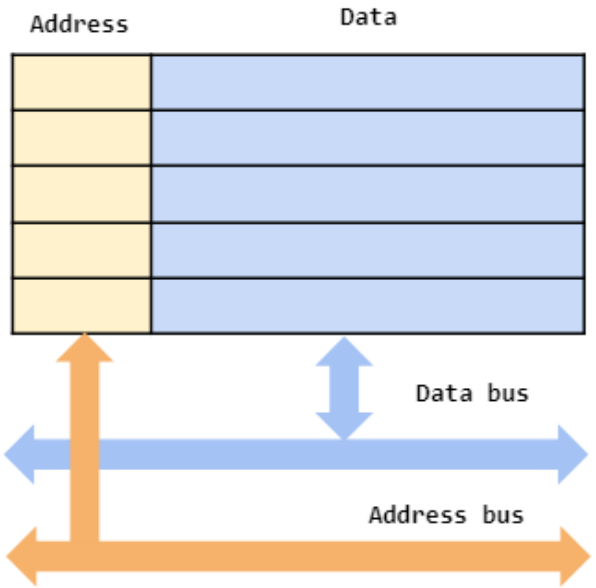




P1

Ilusión

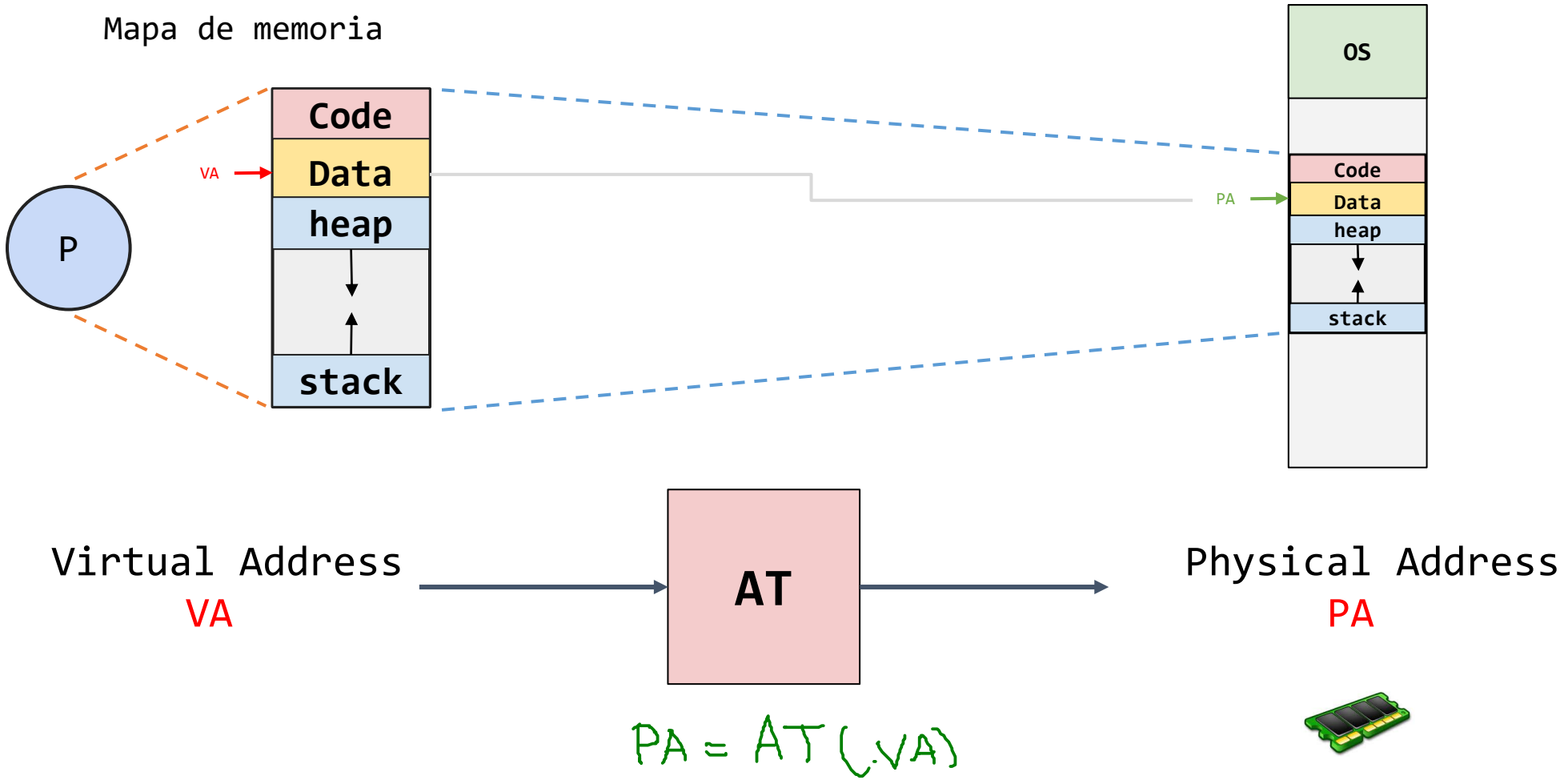




Recurso

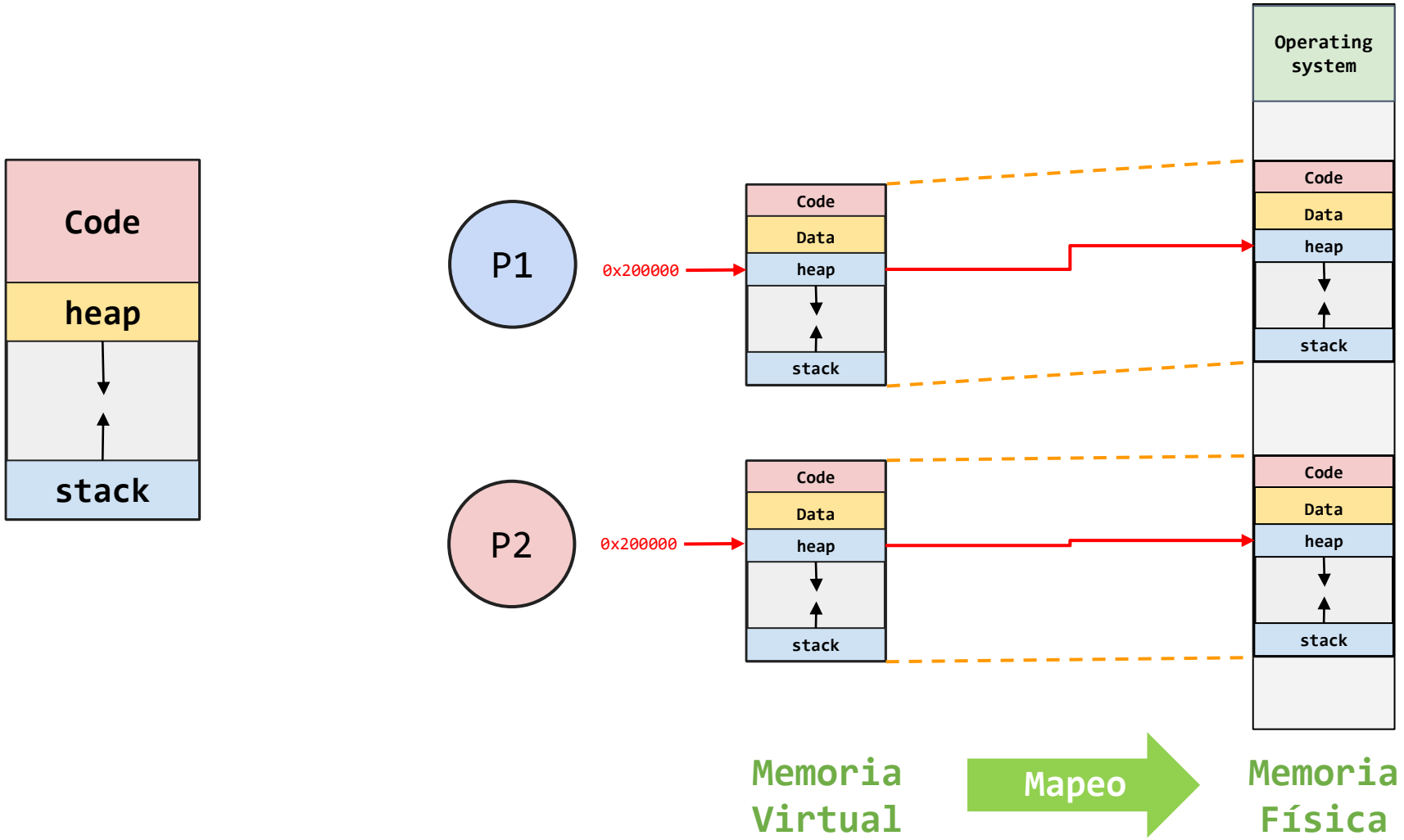
Resumen virtualización

Virtualización de memoria



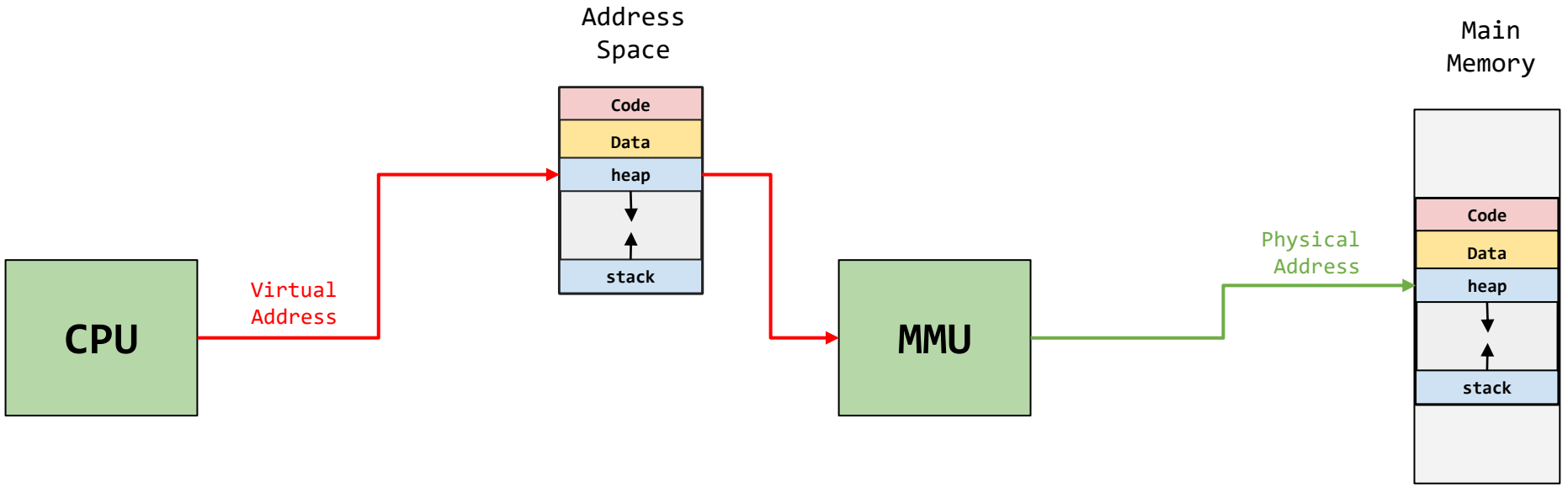
Resumen virtualización

Virtualización de memoria - Abstracción: Address Space



Resumen virtualización

Virtualización de memoria - Address Translation



Resumen virtualización

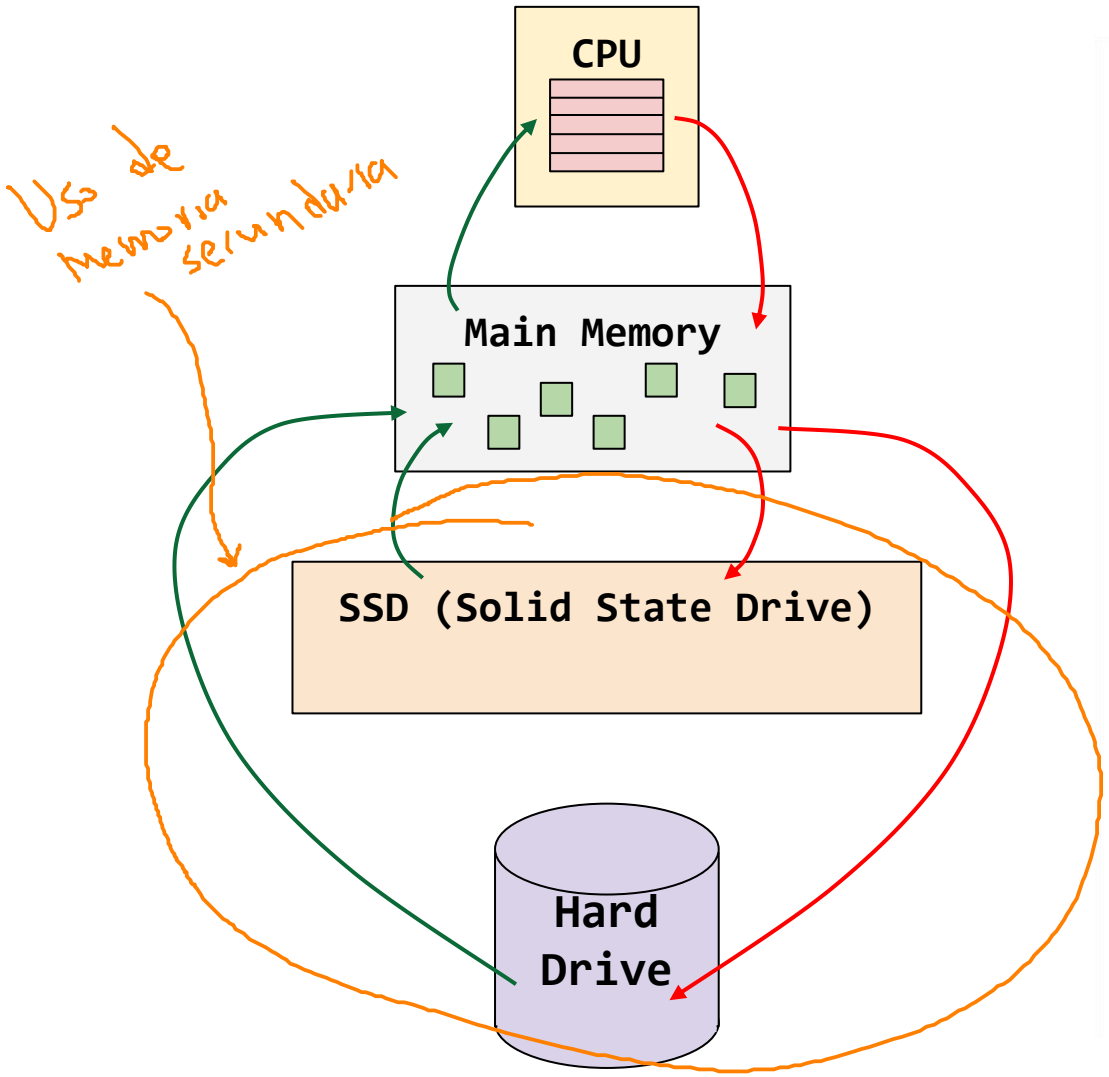
Virtualización de memoria - Address Translation

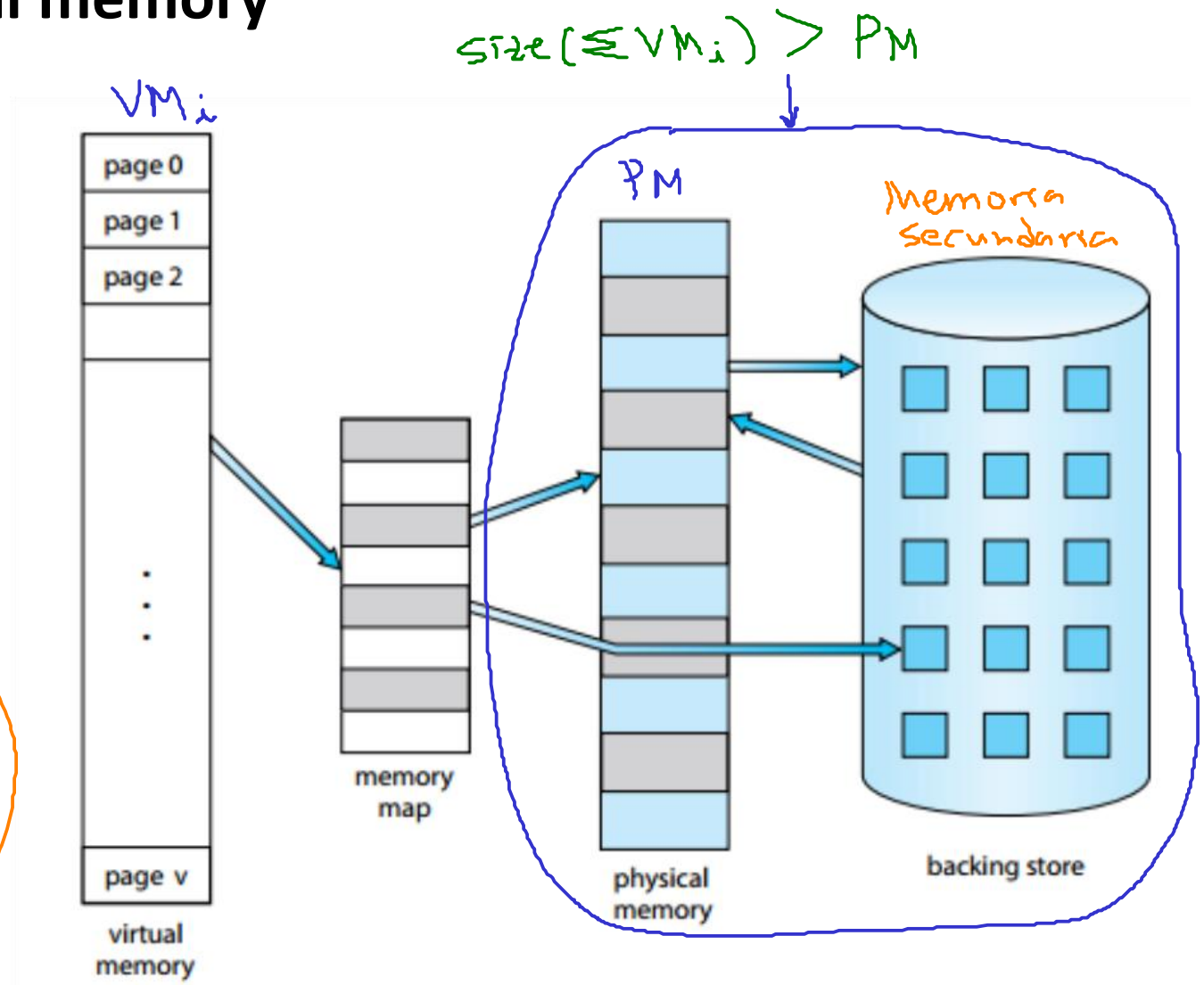
- Dynamic Reallocation (Base & Bound)
- Segmentation (General Base & Bound)
- Paging
- Translation Lookaside Buffers (TLB)
- Advanced Page Tables

No la vimos.

Resumen virtualización

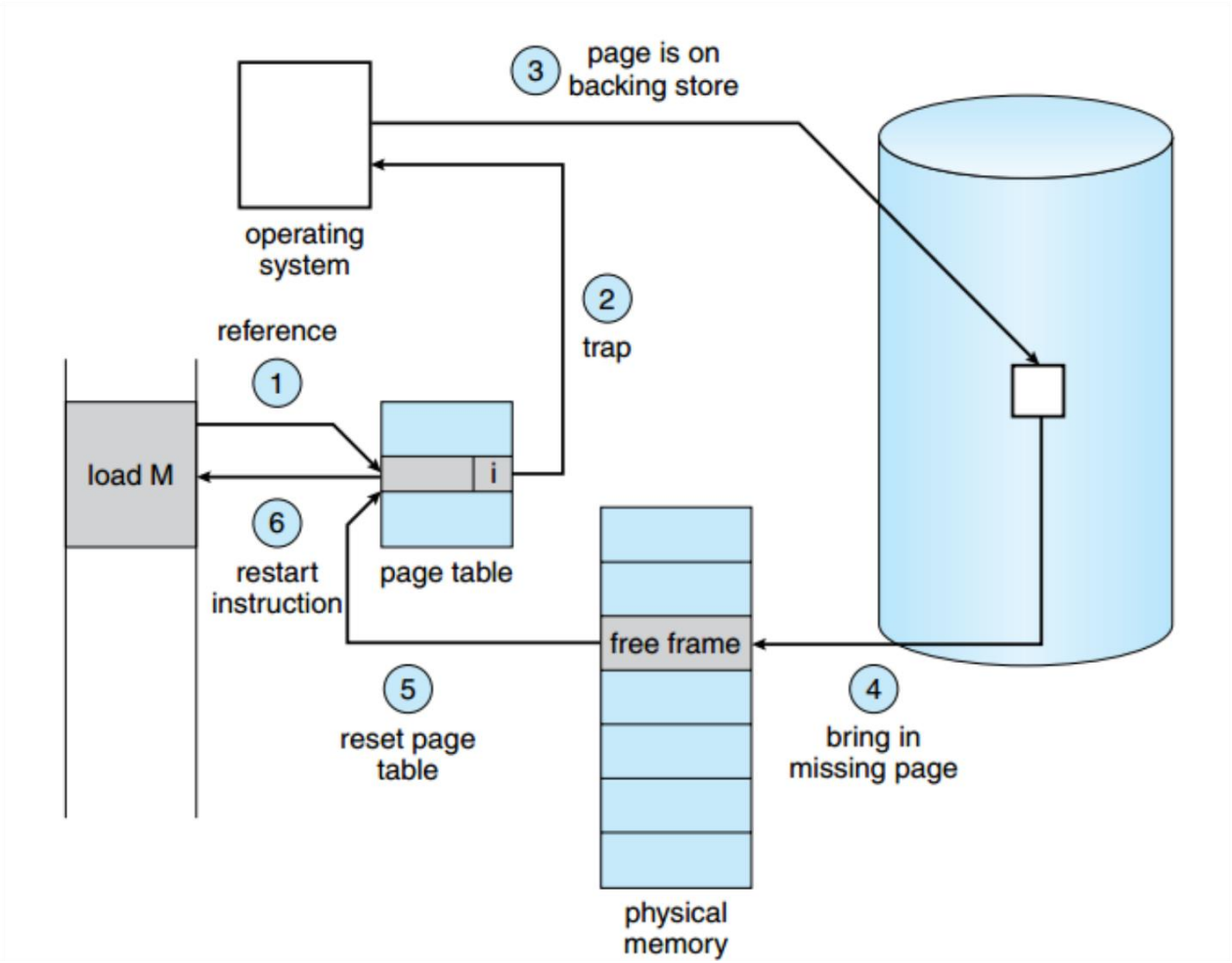
Virtualización de memoria - Virtual memory





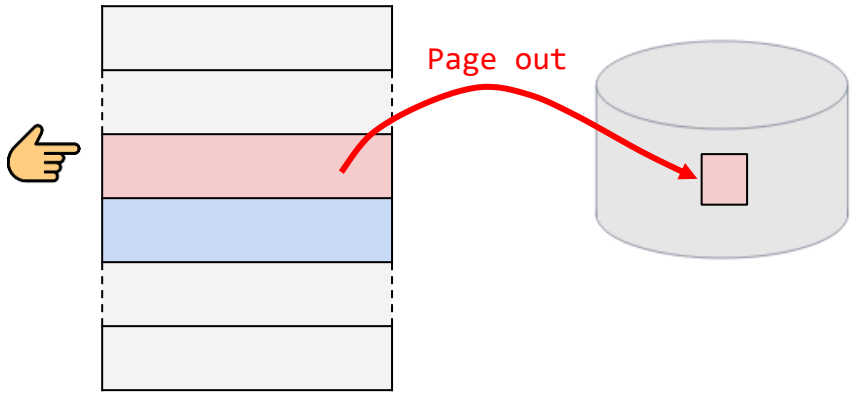
Resumen virtualización

Virtualización de memoria - Virtual memory: Address Translation



Resumen virtualización

Virtualización de memoria - Virtual memory - Políticas

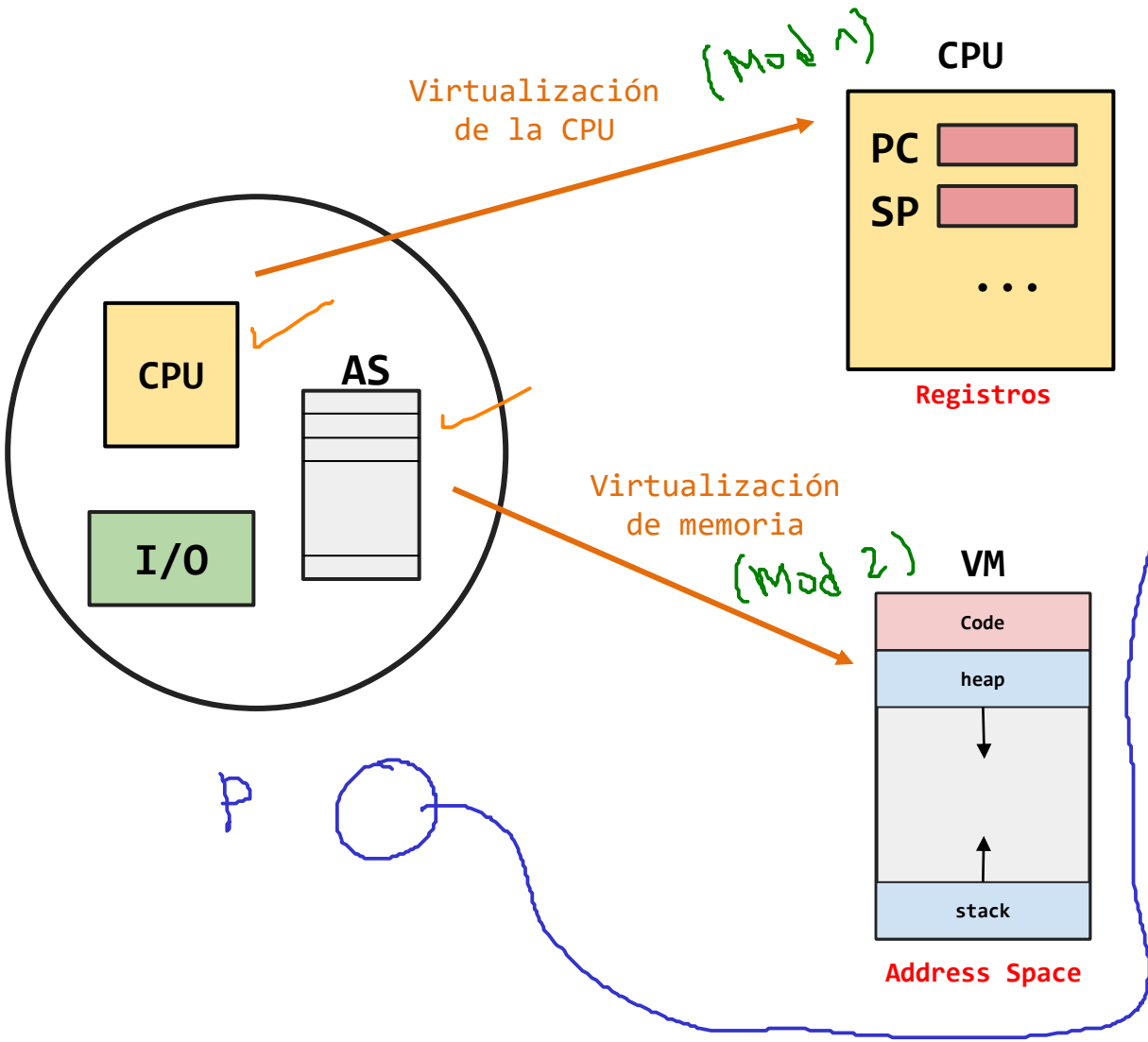


Políticas de reemplazo

- Optimal (OPT)
- FIFO
- RANDOM
- Least Recently Used (LRU)

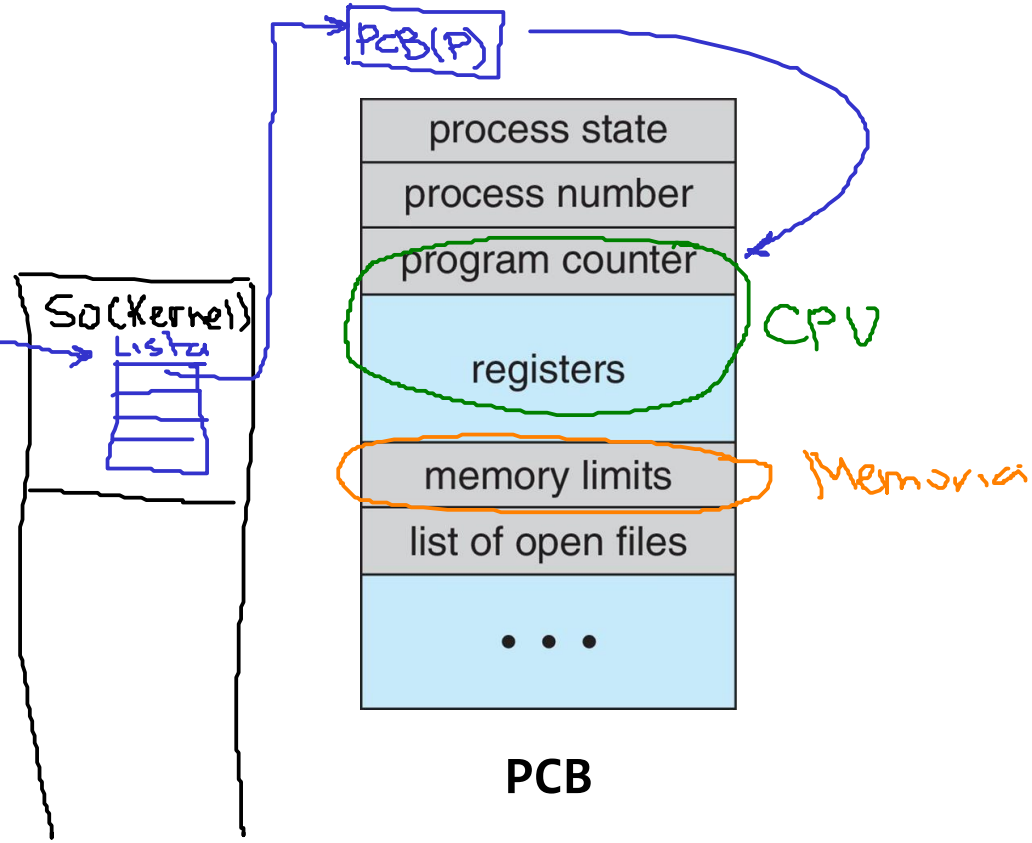
Contextualización

Procesos y virtualización



Un proceso incluye:

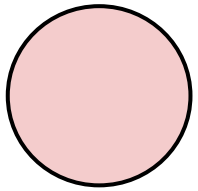
- **CPU context** (Registers)
- **OS Resources** (Address Space, Open files, etc).
- **Otra información** (PID, state, owner, etc).



Contextualización

Procesos e hilos





Proceso

Process





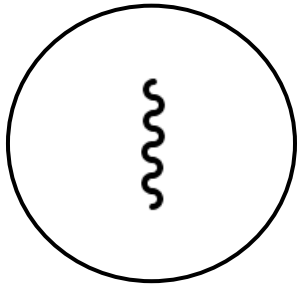
Hilo

New: Thread



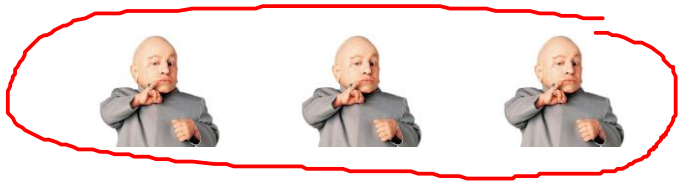
Modulos 1 y 2

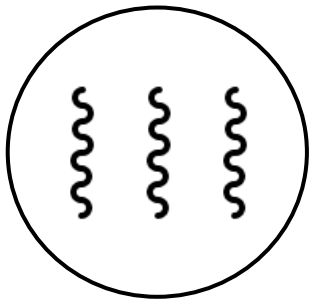




Un hilo de ejecución

Modulo 3





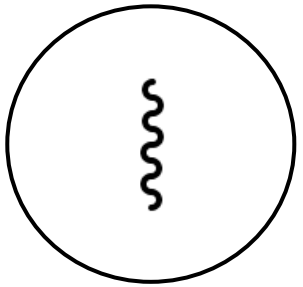
Múltiples hilos de ejecución

Actualizar el modelo de 1 y 2

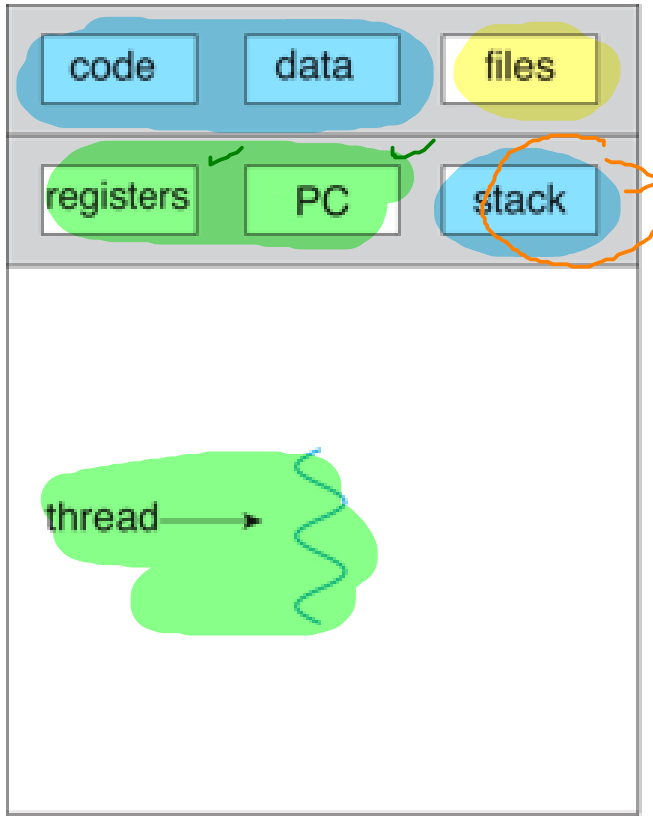
Contextualización

Proceso con un solo hilo

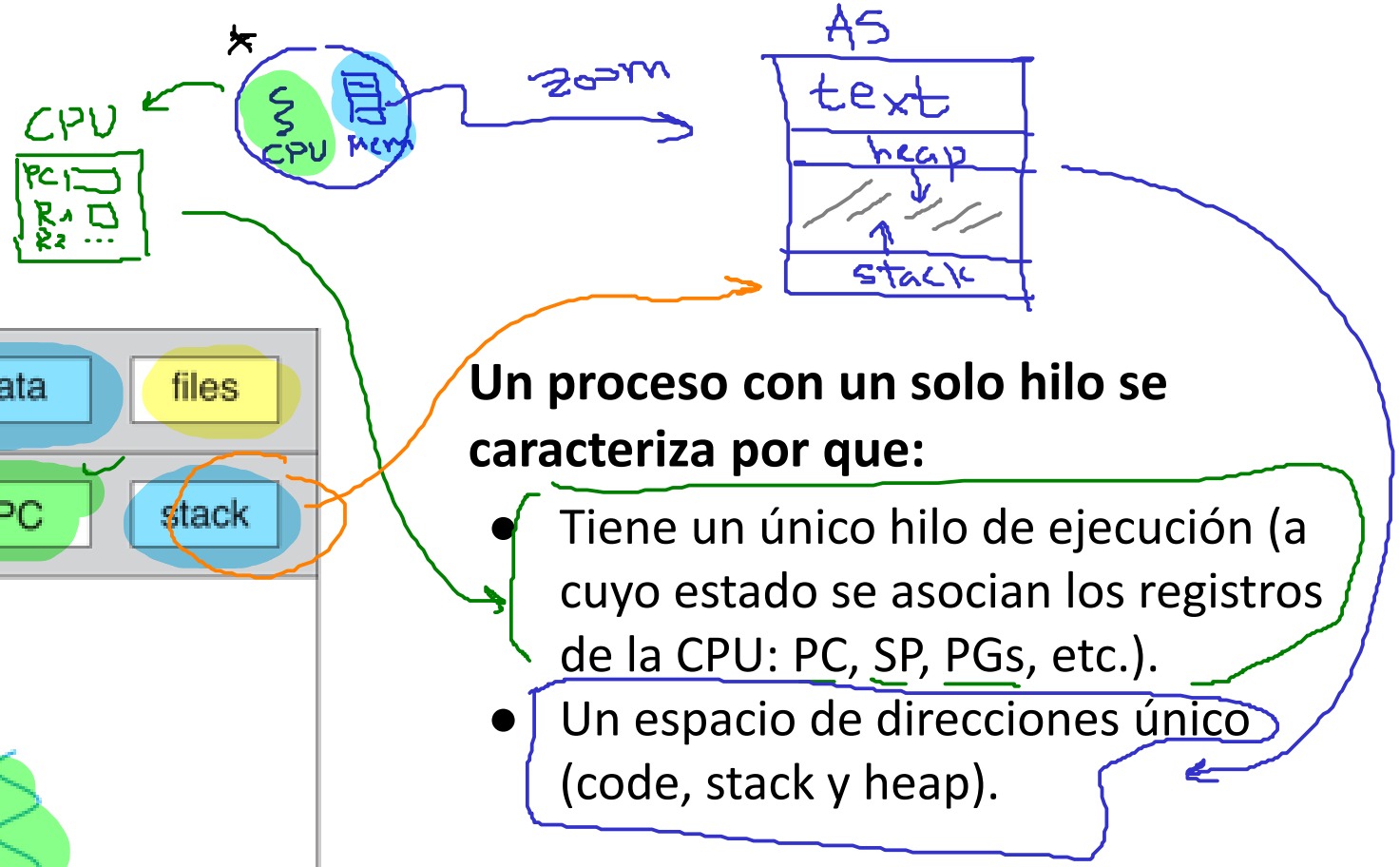




Un hilo de ejecución



single-threaded process



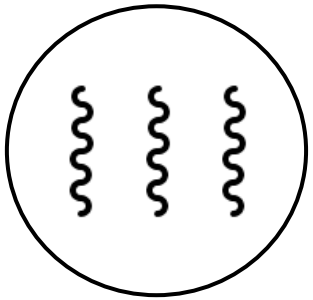
Un proceso con un solo hilo se caracteriza por que:

- Tiene un único hilo de ejecución (a cuyo estado se asocian los registros de la CPU: PC, SP, PGs, etc.).
- Un espacio de direcciones único (code, stack y heap).

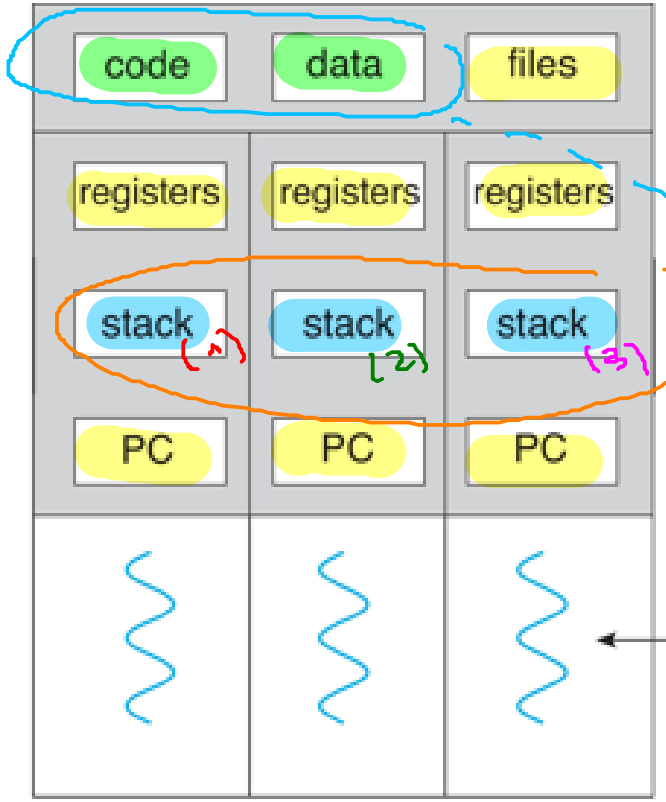
Contextualización

Proceso multihilo



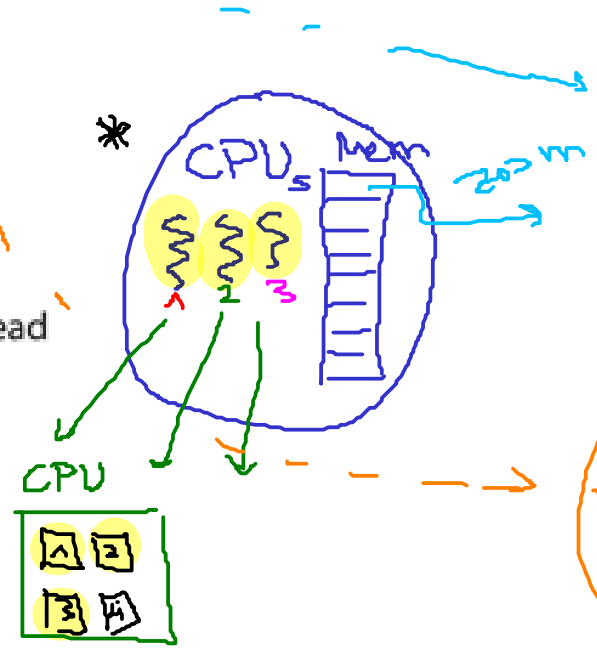


Múltiples hilos de ejecución



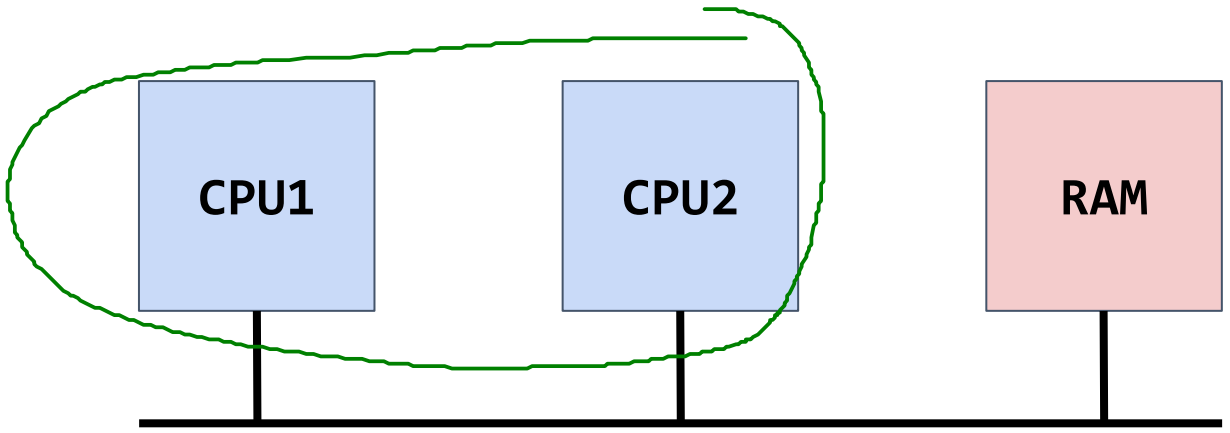
Un proceso con varios hilos se caracteriza por:

- Múltiples puntos de ejecución.
- Múltiples PC (Program Counter)
- Compartir el mismo espacio de direccionamiento entre hilos.



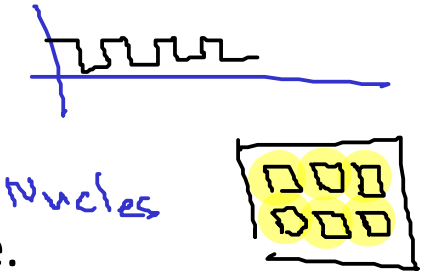
Contextualización

Tendencias de CPUs





- **Tendencia:**
  - Igual velocidad. → ↑ CPU_{clock}
  - Más núcleos. → Más Fácil ↑ Nucleos



- Programas más rápidos → Ejecución concurrente.

- **Objetivo**: Escribir aplicaciones que utilicen varias CPUs → **¿Oh y ahora quién podrá ayudarme?**

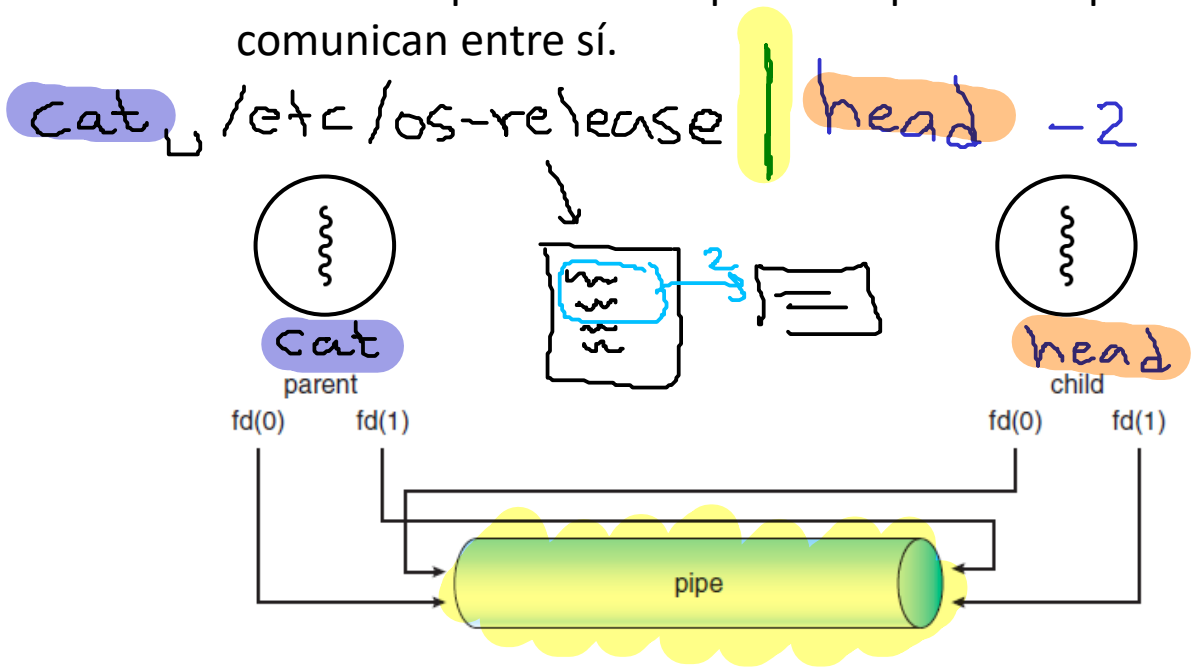


Contextualización

Desarrollo de aplicaciones concurrentes

Aproximación 1

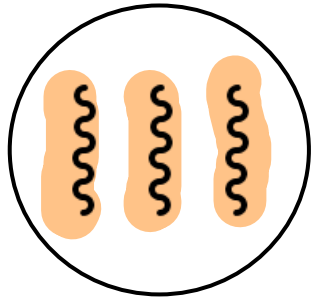
- Construir aplicaciones a partir de procesos que se comunican entre sí.



- **Ejemplo**: Pestañas de un navegador.

Aproximación 2

- Usando una nueva abstracción conocida como thread (hilo - hebra - proceso ligero - miniproceso).
- Los hilos son como los procesos excepto que, múltiples hilos de un mismo proceso comparten el mismo espacio de direcciones (AS).

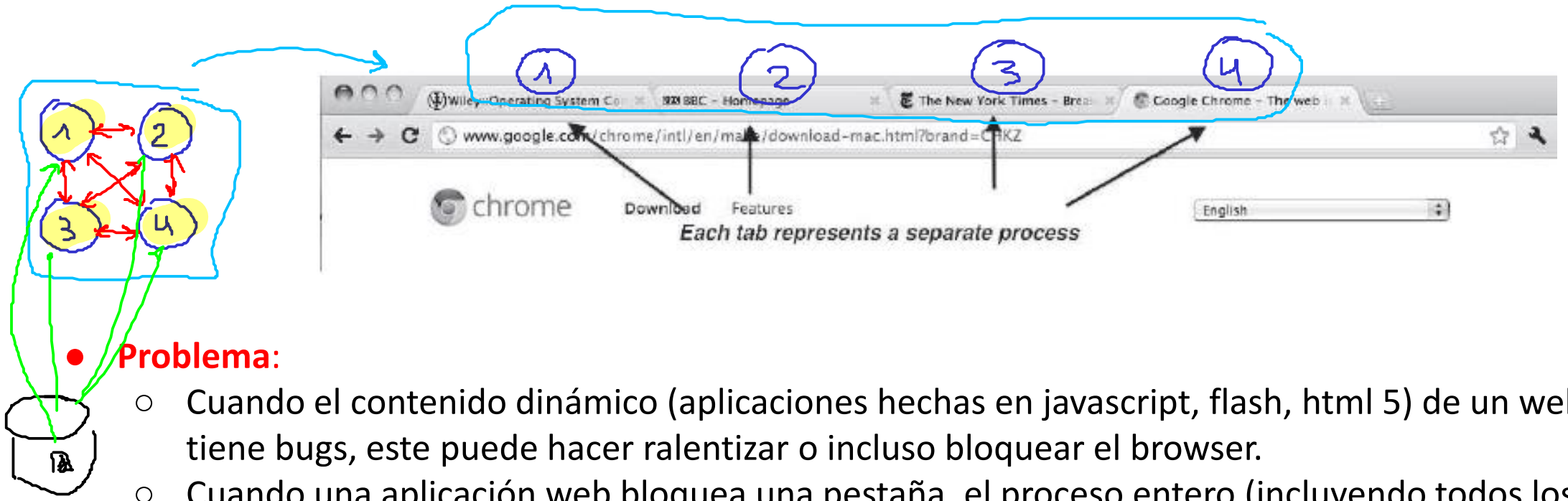


- **Ejemplo**: Procesador de Texto.

Contextualización

Ejemplos - Browser

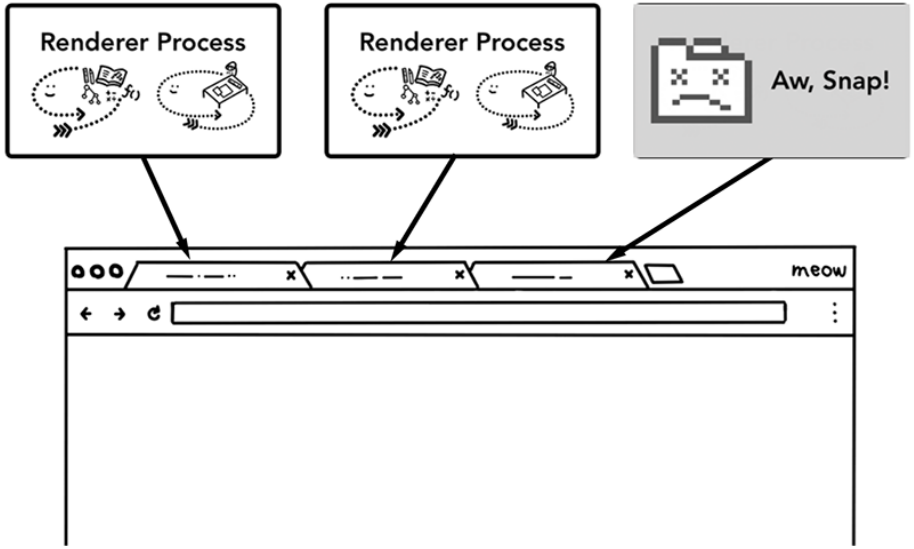
- Un browser es una aplicación que permite desplegar el contenido de un sitio web.
- Muchos navegadores modernos permiten navegar a través de diferentes páginas usando pestañas.
- Cada una de las pestañas del navegador representa un proceso separado.

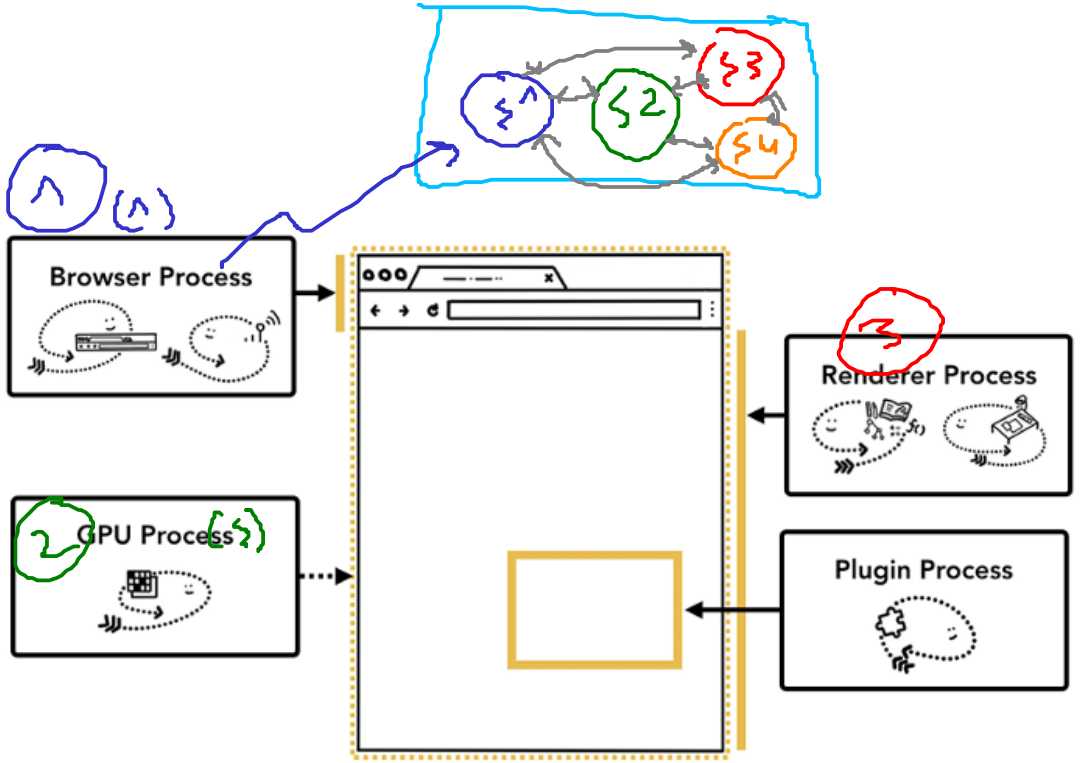


- **Problema**:
  - Cuando el contenido dinámico (aplicaciones hechas en javascript, flash, html 5) de un website tiene bugs, este puede hacer ralentizar o incluso bloquear el browser.
  - Cuando una aplicación web bloquea una pestaña, el proceso entero (incluyendo todos los sitios webs desplegados en las demas pestañas) tambien se pueden bloquear.

Contextualización

Browser





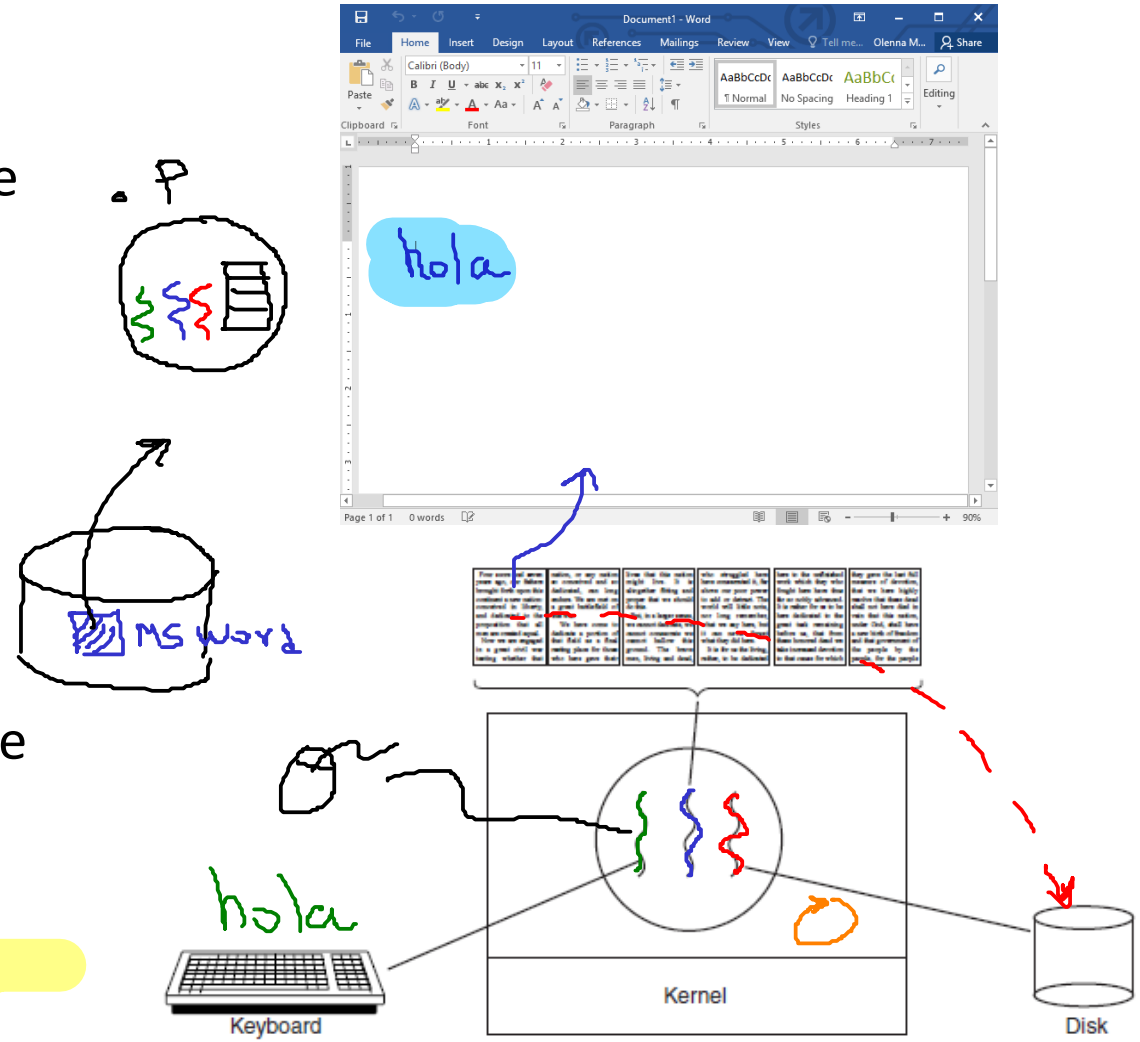
Link animación

- **Solución:** Arquitectura multiproceso de google donde se identifican tres tipos de procesos (link):
  - Browser: Encargado de la interfaz de usuario, el disco y los datos de red.
  - Render: Contiene la lógica para renderizar las páginas web (html, javascript, imágenes, etc).
  - Plug-in: Proceso creado para cada tipo de pluging (flash, quicktime)

Contextualización

Ejemplo - Procesador de texto

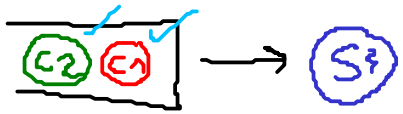
- Un procesador de texto es un programa que muestra un documento en pantalla exactamente como aparecerá cuando se imprima.
- Puede ser concebido como un proceso con varios hilos, por ejemplo suponiendo tres hilos:
  - **Hilo 1**: Encargado de la interacción con el usuario.
  - **Hilo 2**: Encargado de dar el formato al documento cuando se le indica.
  - **Hilo 3**: Encargado de escribir periódicamente el contenido de la RAM al disco duro.
- Los tres hilos operan sobre el documento (memoria común compartida por los procesos).



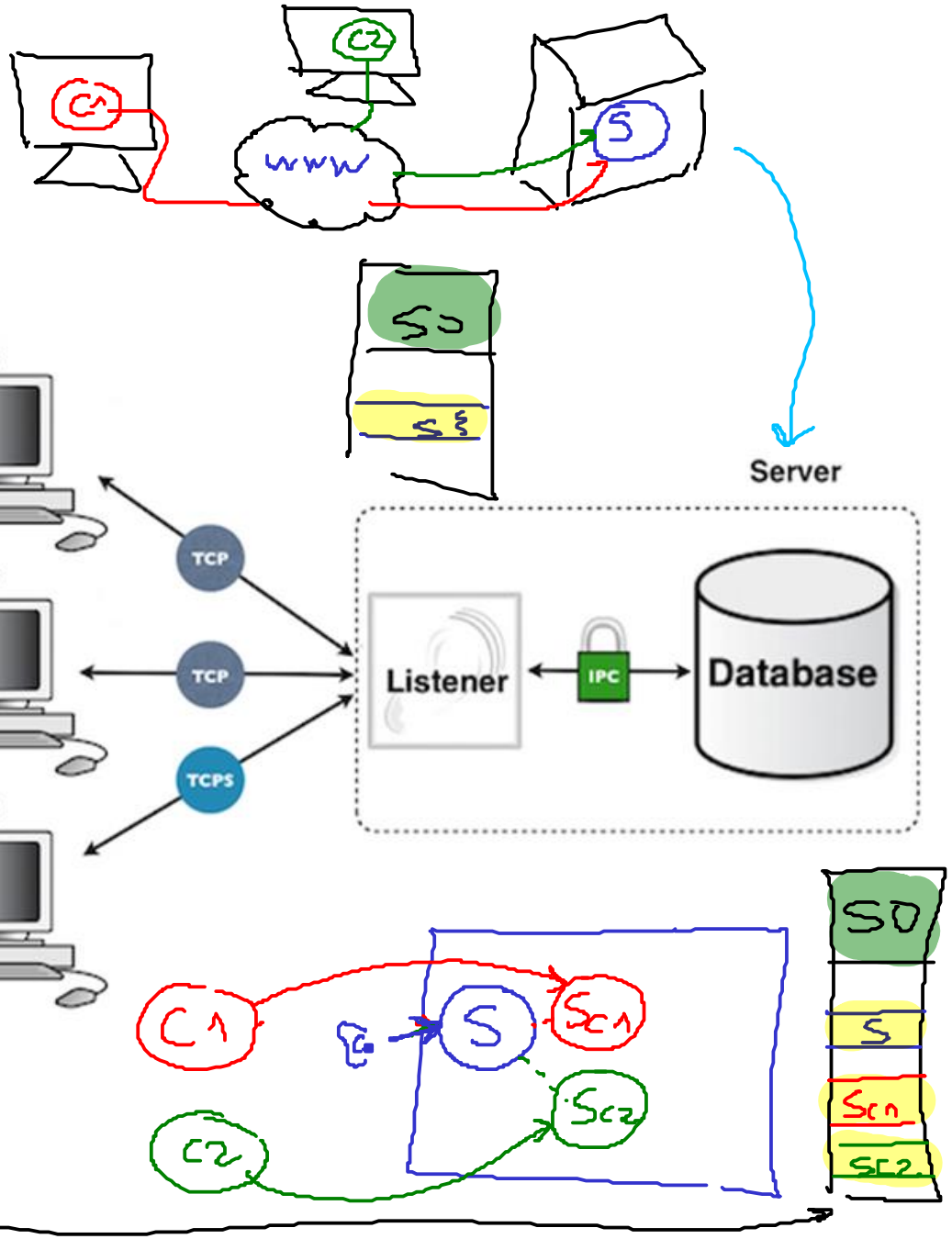
Contextualización

Ejemplo - Servidor

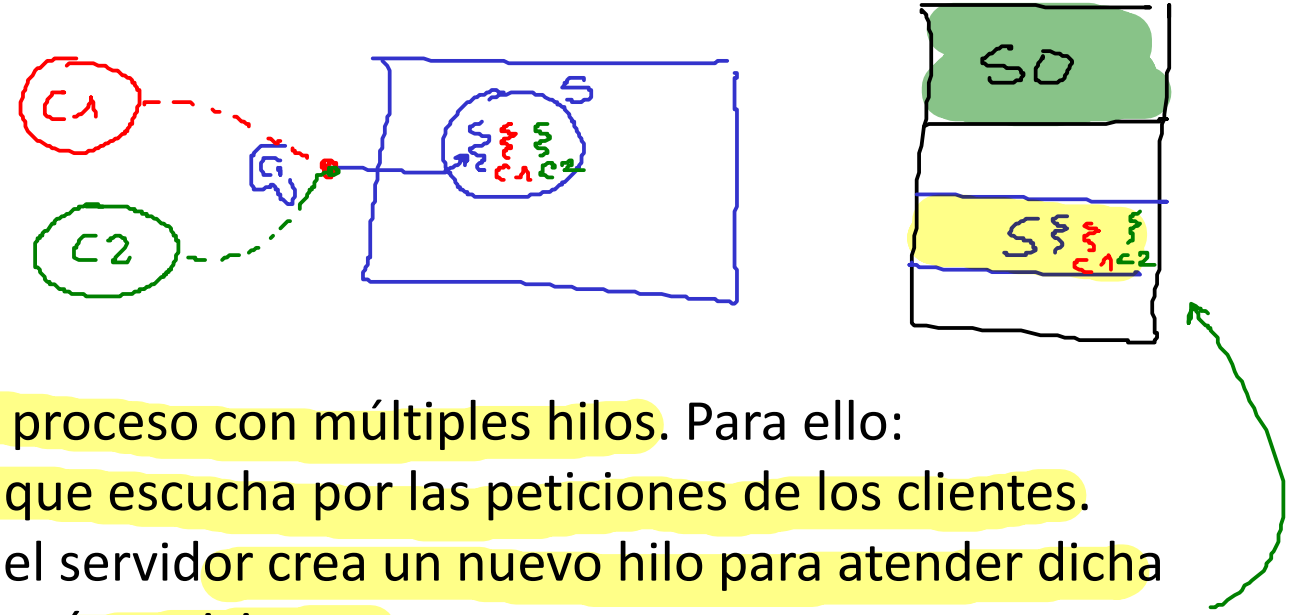
Un servidor acepta y atiende peticiones hechas por clientes:



1. **Servidor como solo proceso**:
   - Solo puede atender a un cliente a la vez.
   - Ineficiente ya que los demás clientes tiene que esperar para ser atendidos.
2. **Servidor de varios procesos**:
   - El servidor se ejecuta como un único proceso, el cual cada vez que recibe una petición, crea un proceso nuevo para atender dicha petición.
   - La creación de procesos consume tiempo y es costosa.

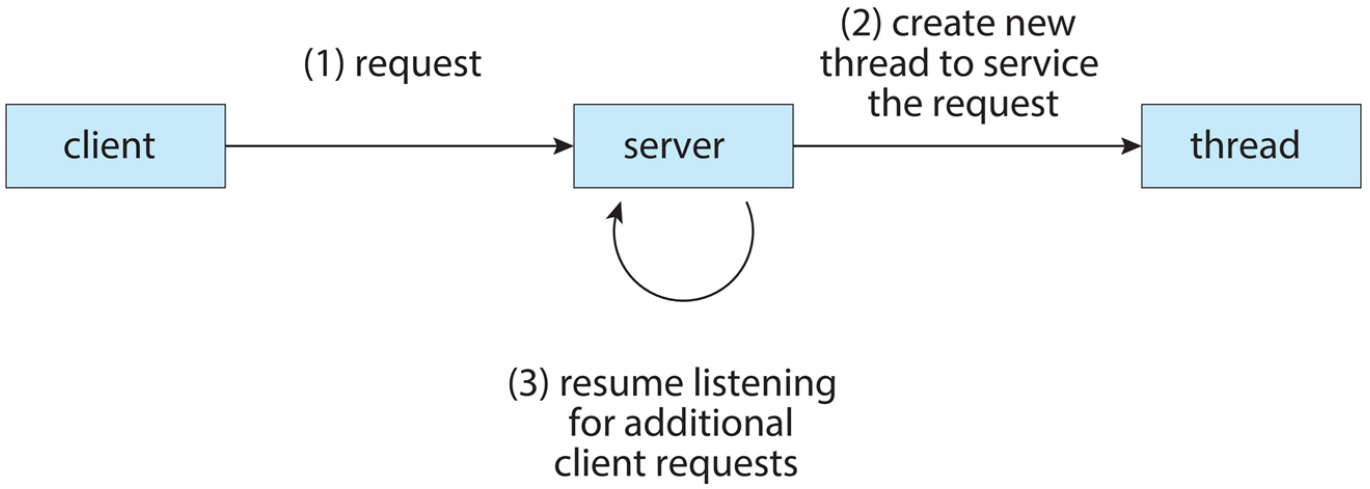


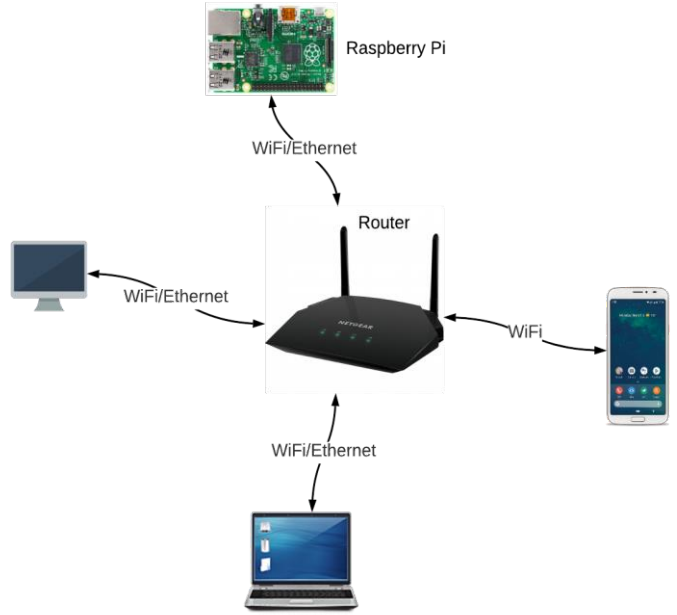
Contextualización



Ejemplo - Servidor

3. **Servidor Multihilo**:
   - El servidor se implementa usando un proceso con múltiples hilos. Para ello:
     - El servidor crea un hilo separado que escucha por las peticiones de los clientes.
     - Cuando una petición es recibida, el servidor crea un nuevo hilo para atender dicha petición y continúa pendiente de más peticiones.



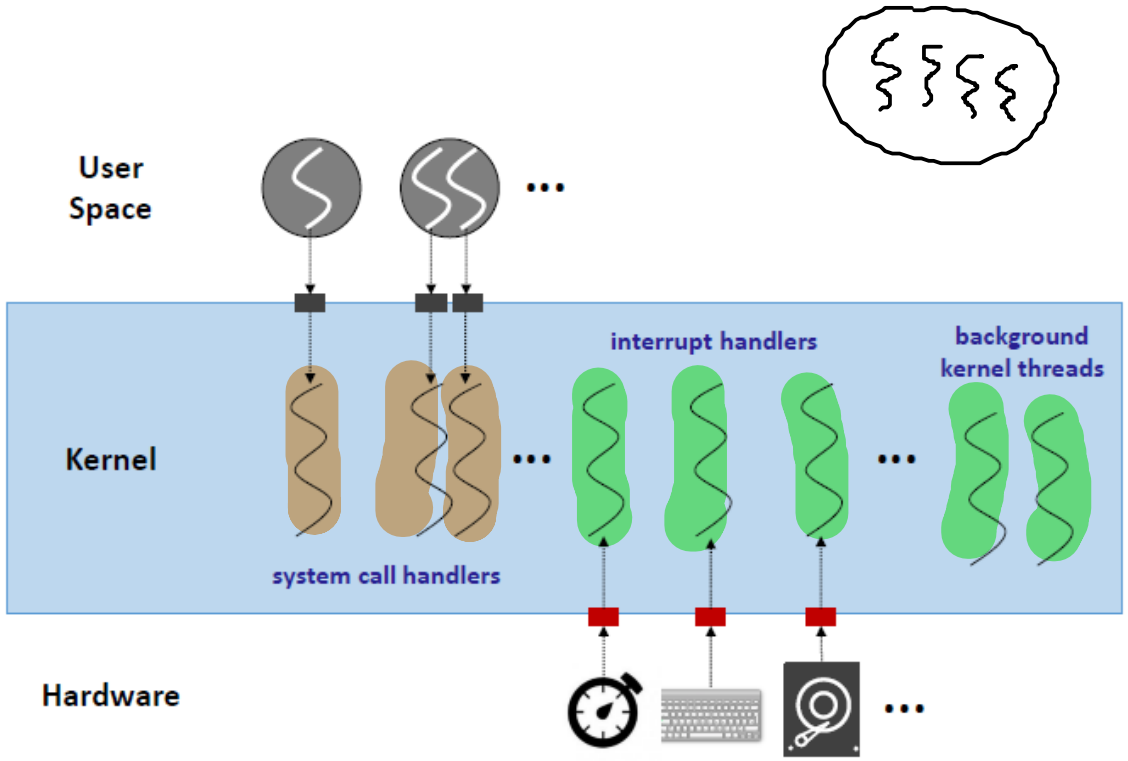


Contextualización

Ejemplo - Sistemas operativos

- Muchos sistemas operativos también son multihilo.
  - En los sistemas Linux, durante el tiempo de booteo, varios hilos del kernel son creados para realizar una tarea específica (administrar dispositivos, gestionar memoria, manejar interrupciones, etc).

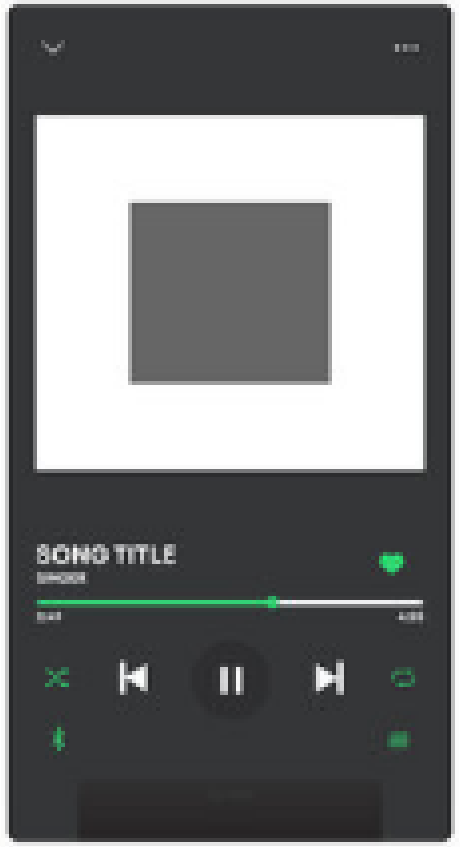




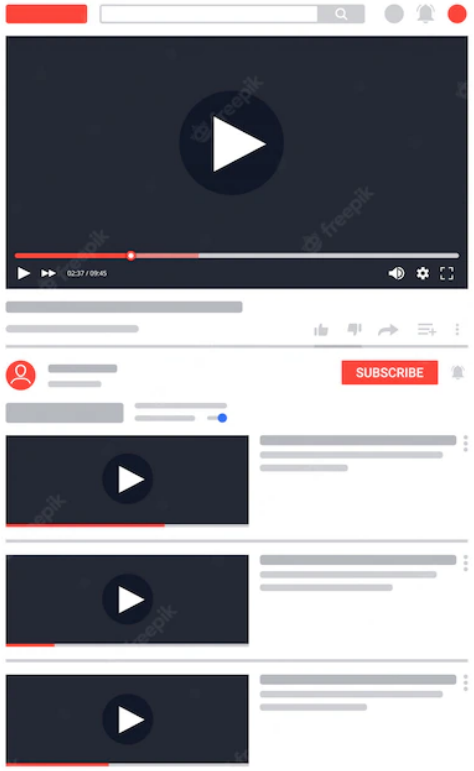
Contextualización

Ejemplo - Otras aplicaciones

Spotify®

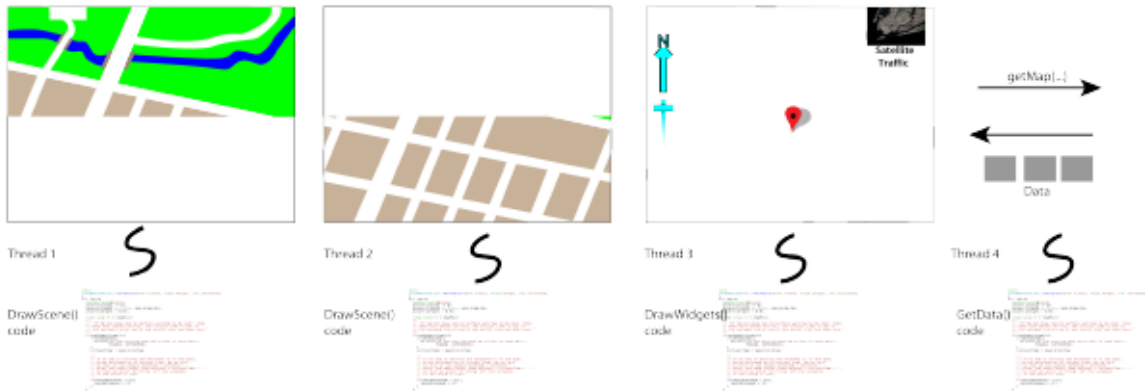


YouTube



Google Earth

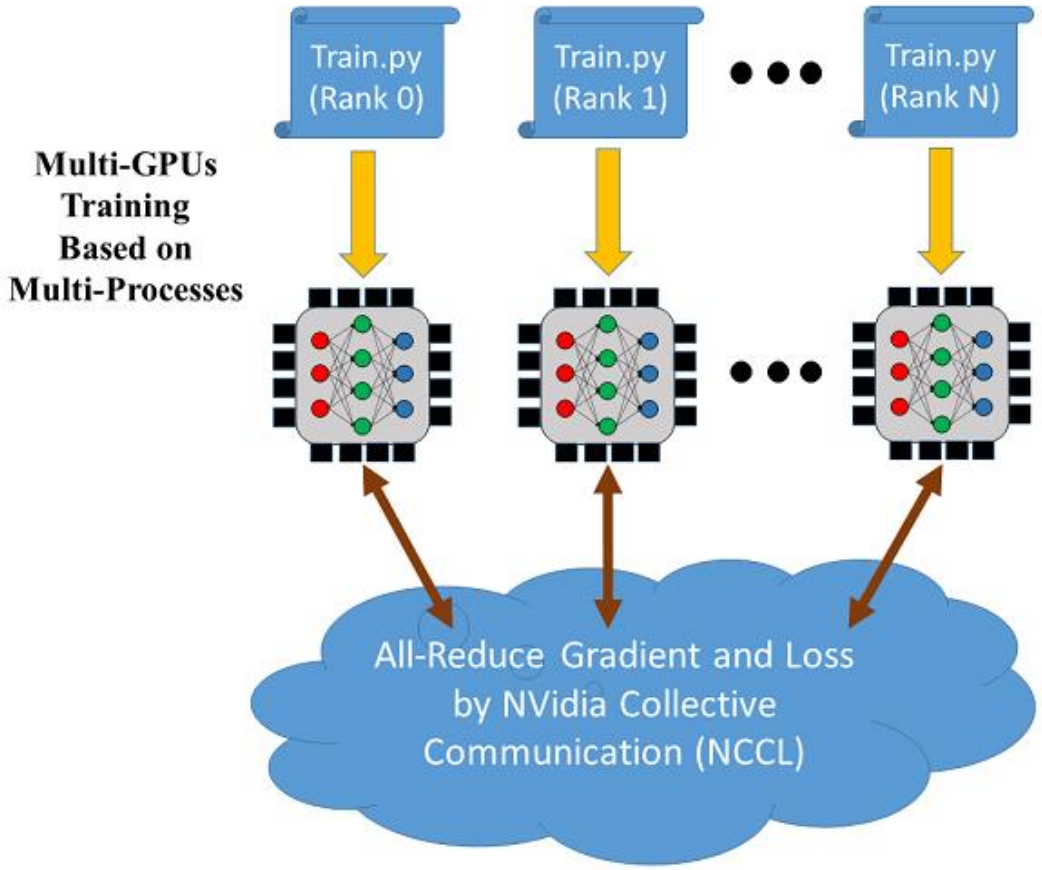




Contextualización

Algoritmos y aplicaciones que hacen uso intensivo de la CPU

- Basic sorting
- Trees
- Graph algorithms
- Simulations
- Data mining.
- Artificial intelligence.
- Machine learning.
- Graphics.



Hilos

Modulo 1 → Proceso

Fin: 13/10/2026

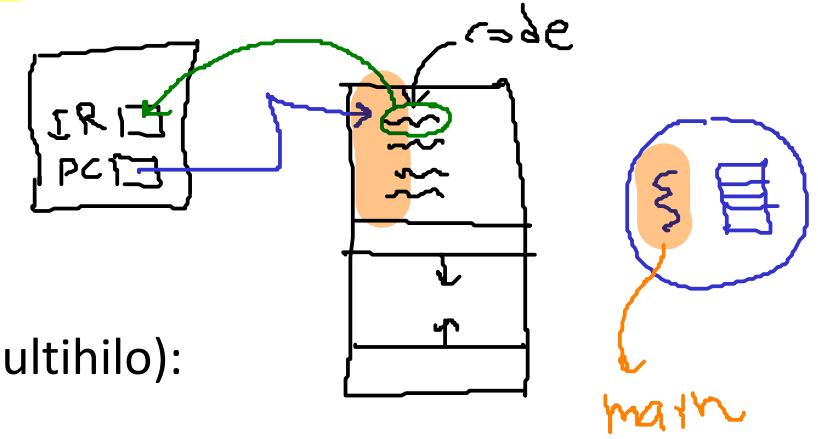
Inicio: 15/10/2026

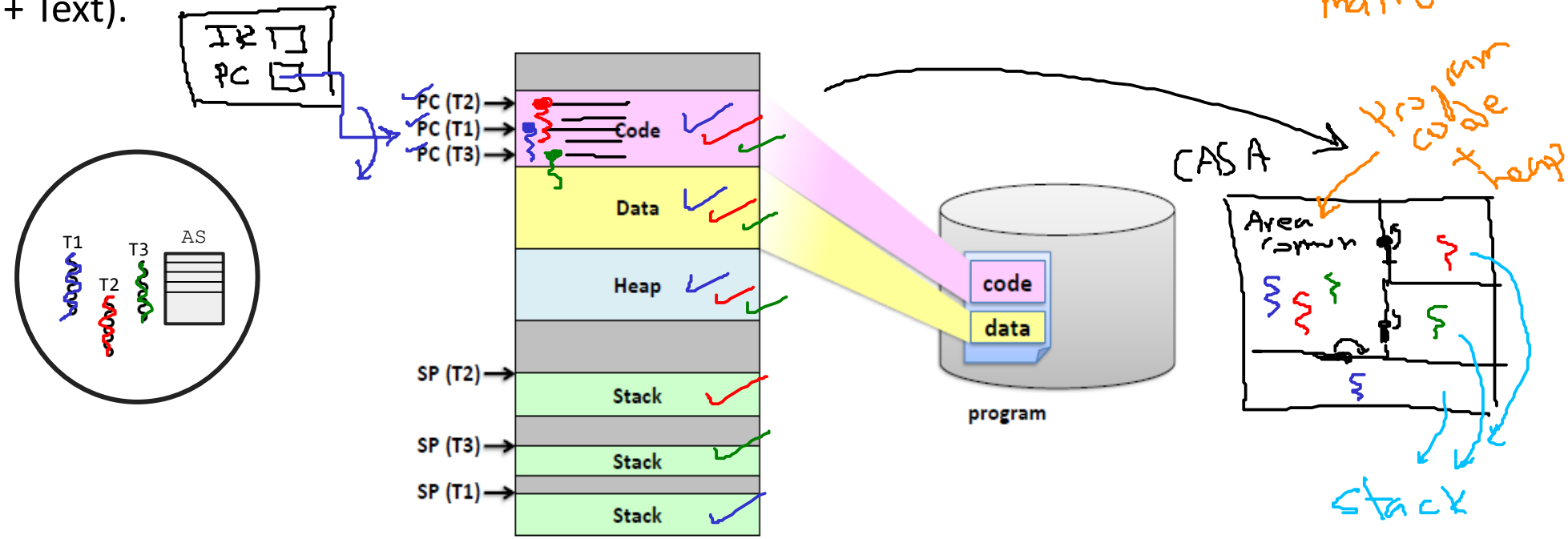
Modulo 3 → Hilo

Aspectos importantes

- Un hilo de control hace referencia a una secuencia de instrucciones que están siendo ejecutadas en un programa.
- Cada hilo (thread) tiene:
  - Un Thread ID (TID)
  - Un conjunto de registros asociados, incluidos el PC y el SP.
  - Stack
- Cada hilo comparte (con otros los demás hilos si el proceso es multihilo):
  - Code (Data + Text).
  - Heap.

- TID
- Registros
- stack

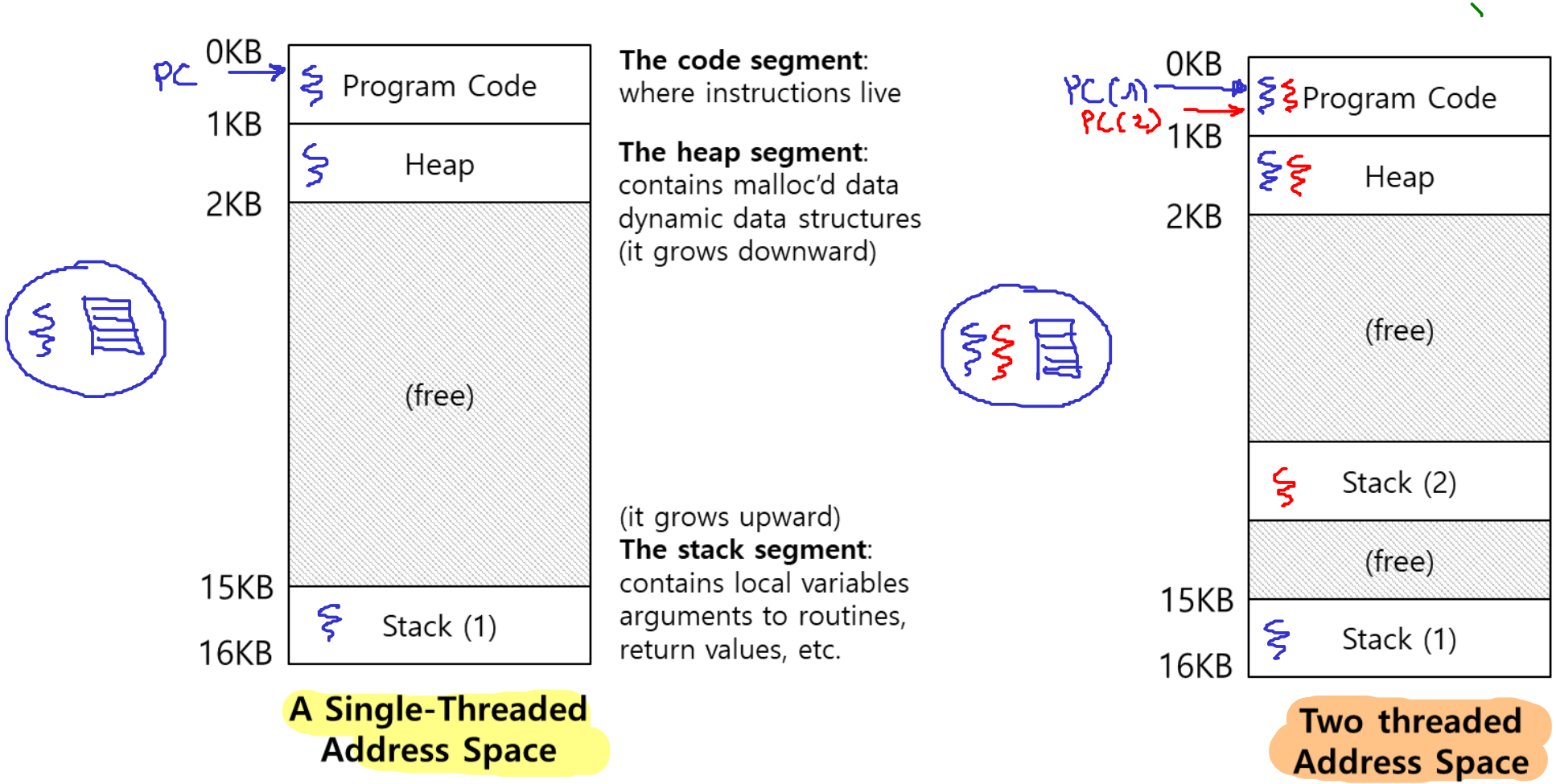




Hilos

Hilos y espacios de direcciones - Address Space multi-stack

- Todos los hilos comparten los segmentos code y heap del espacio de direcciones.
- Cada hilo tiene su propio stack.



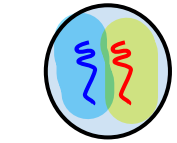
Hilos

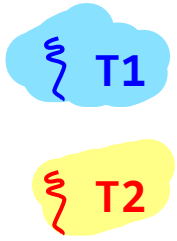
Thread control block

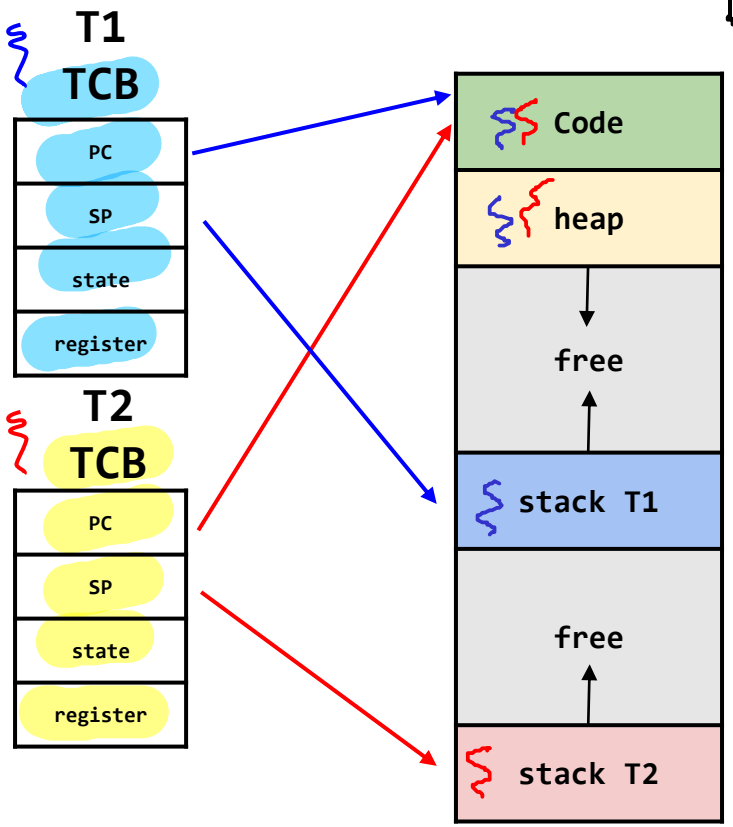
Estructura que almacena el estado de cada hilo.

Almacenar contexto

Espacio de direcciones Multi-stack



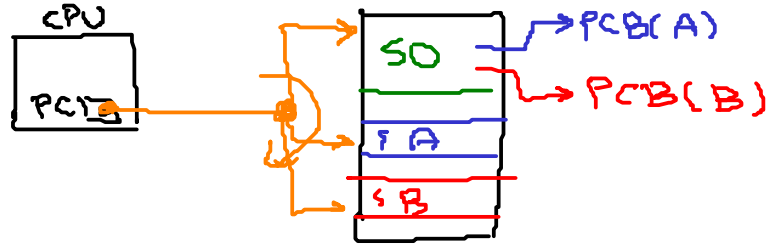




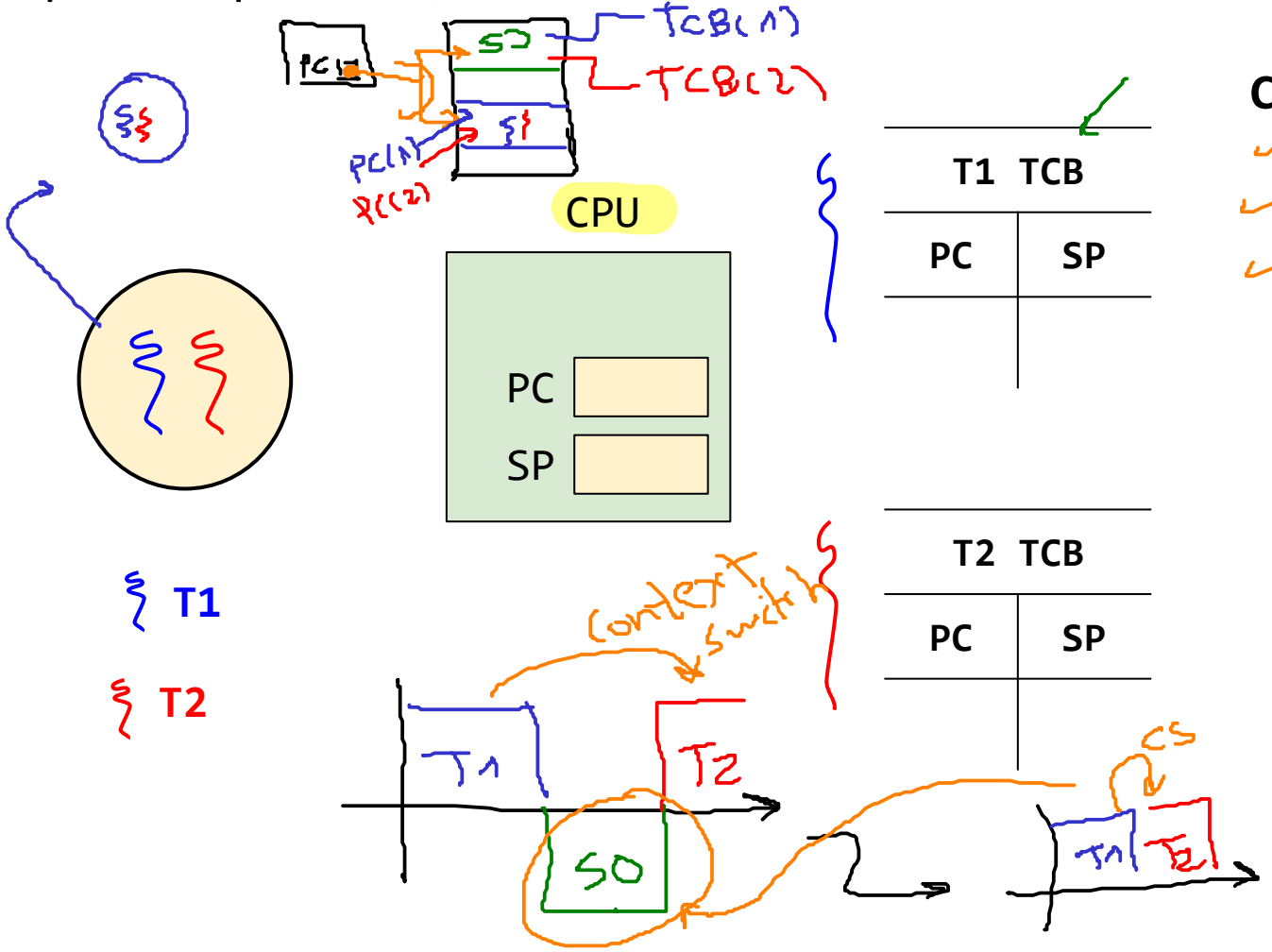
Hilos

Cambio de contexto



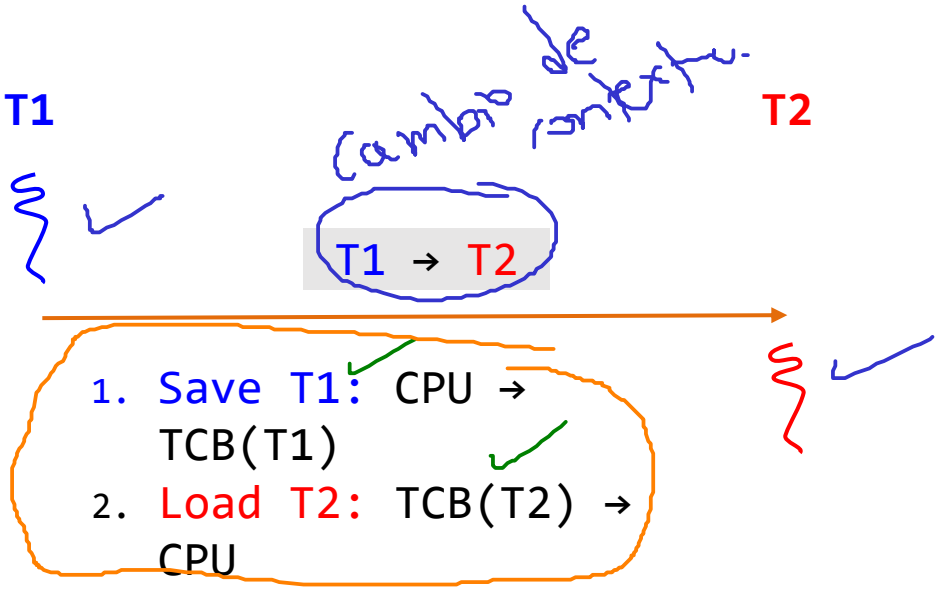


- El cambio de contexto entre hilos es similar al de los procesos. Sin embargo, a diferencia del caso para los procesos, en el cambio de contexto entre hilos, el espacio de direcciones no cambia.



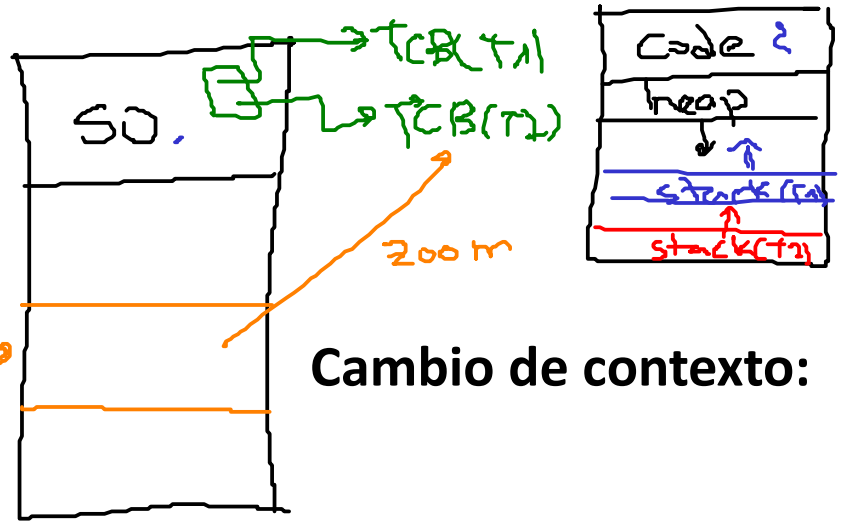
Cambio de contexto entre hilos T1 a T2:

- Se almacenan los valores de los registros de T1.
- Se cargan los valores de los registros T2.
- Se mantiene el mismo espacio de direccionamiento.



Hilos

Cambio de contexto - Ejemplo



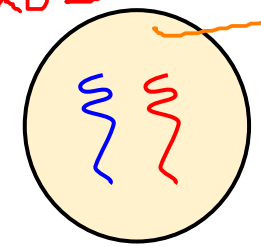
CPU	
PC	SP
?	?

T1 TCB	
PC	SP
0x02	0x40

Cambio de contexto: T1 → T2

PC(T1) = 0x02
SP(T1) = 0x40

PC(T1) = 0x02
SP(T1) = 0x40

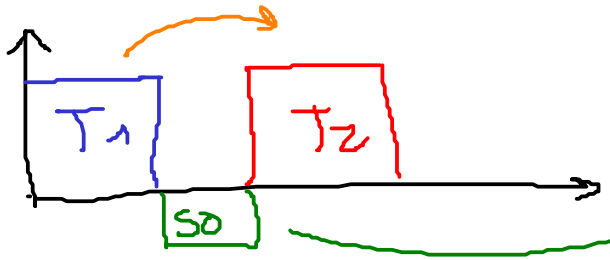


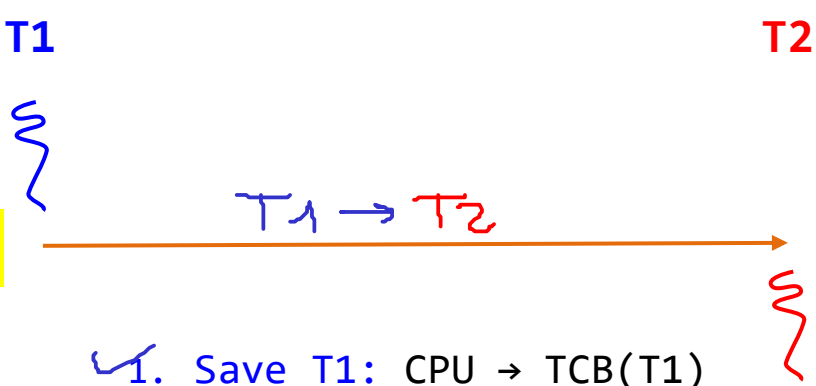
T1

T2

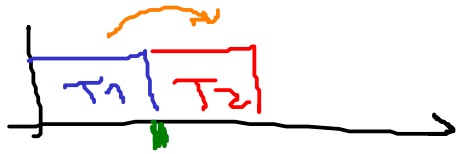
T2 TCB	
PC	SP
0x02	0x60

PC(T1) = 0x08



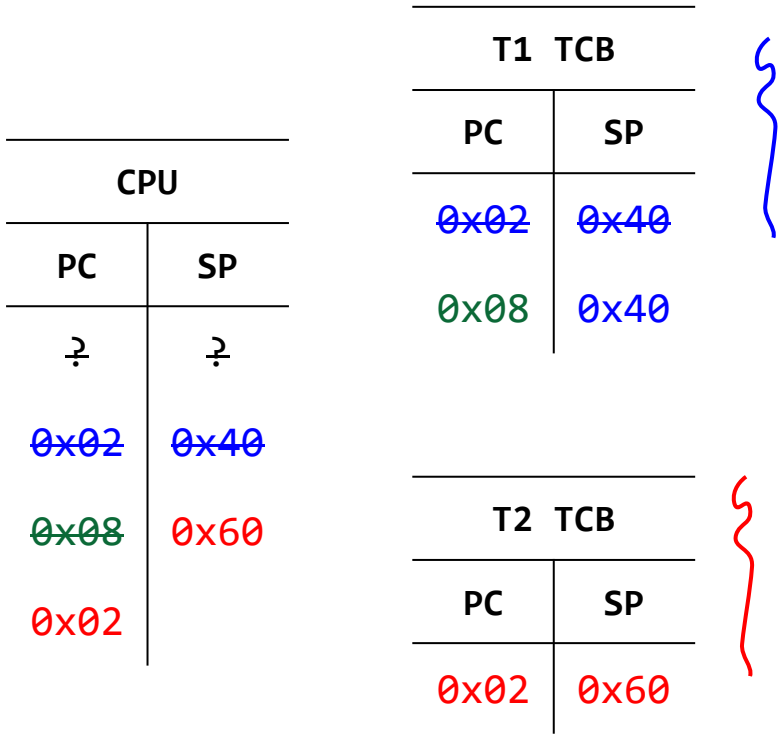


1. Save T1: CPU → TCB(T1)
2. Load T2: TCB(T2) → CPU



Hilos

Cambio de contexto - Ejemplo (Solución)



Cambio de contexto: T1 → T2

PC(T1) = 0x02
SP(T1) = 0x40

PC(T1) = 0x02
SP(T1) = 0x40

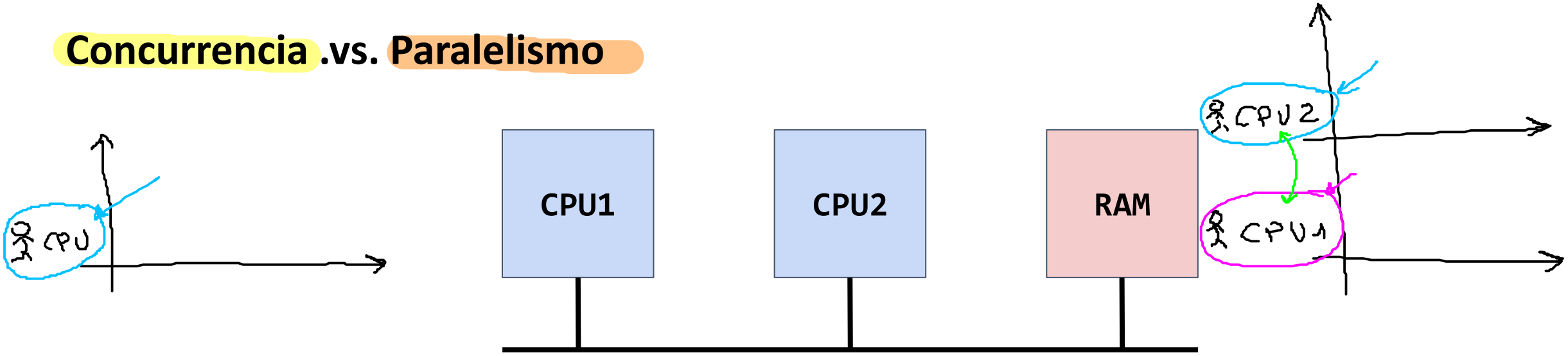
T1 **T2**

PC(T1) = 0x08

1. Save T1: CPU → TCB(T1)
2. Load T2: TCB(T2) → CPU

Hilos

Concurrencia .vs. Paralelismo

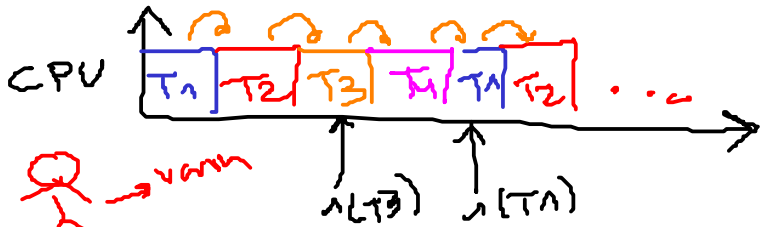


- El aprovechamiento de los sistemas multicore o multiprocesador impone nuevos retos a los programadores los cuales incluyen:
  - División de actividades.
  - Balance de carga.
  - Segmentación de datos.
  - Dependencia de datos.
  - Testeo y depuración.

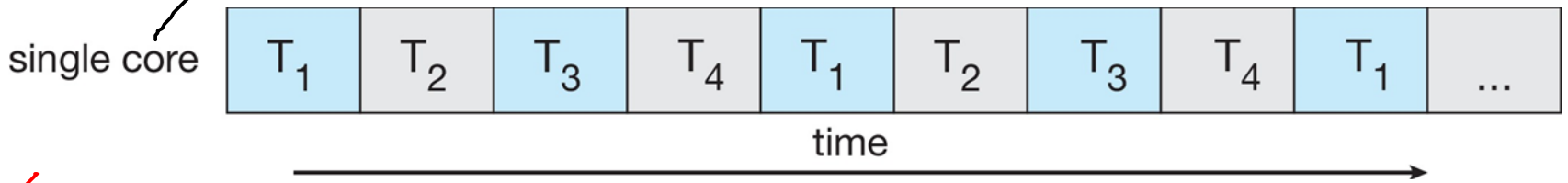
pipeline → Dependencia

Hilos

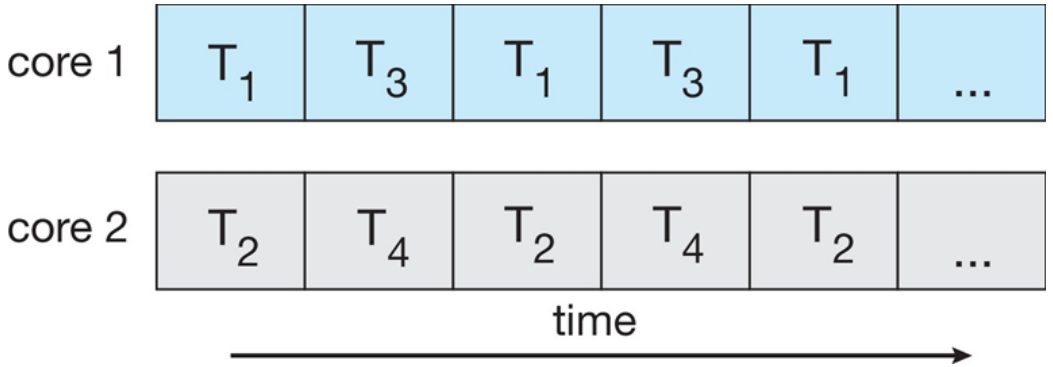
Concurrencia .vs. Paralelismo

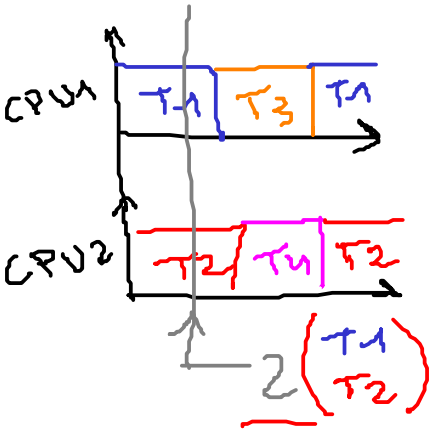


- **Concurrencia**:
  - Permite la ejecución de más de una tarea conforme transcurre el tiempo.
  - En un sistema con un solo procesador, el scheduler proporciona la concurrencia.



- **Paralelismo**:
  - Implica que un sistema puede hacer de manera simultánea (a la vez) más de una tarea.
  - Propio de sistemas multicore o multiprocesador.



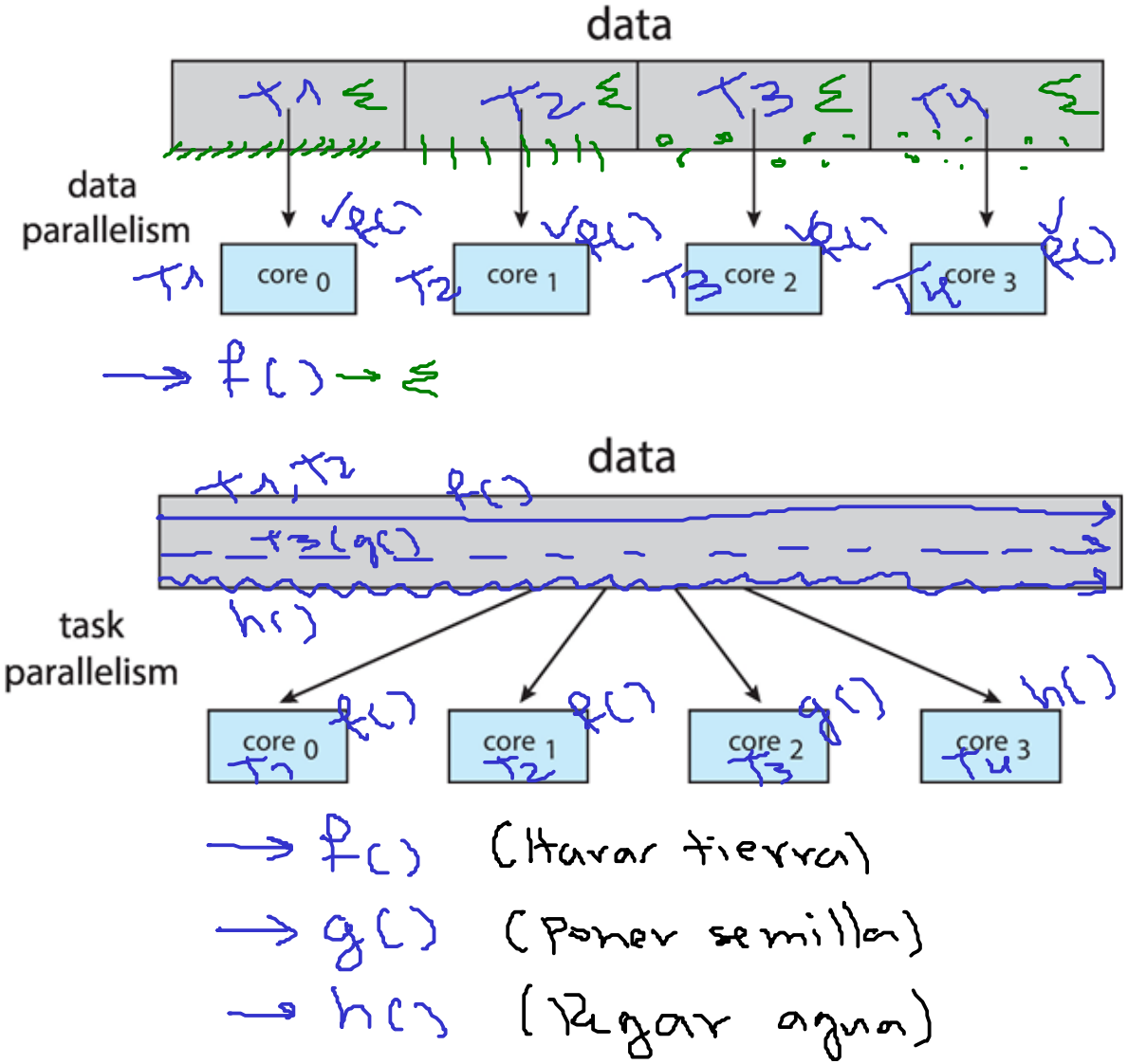


Hilos

Tipos de paralelismo

- **Paralelismo de datos**: Divide los datos en subconjuntos los cuales son procesados por diferentes cores realizando el mismo conjunto de operaciones

- **Paralelismo de tareas**: Distribuye hilos a través de los núcleos de tal manera que cada hilo realiza una única operación.

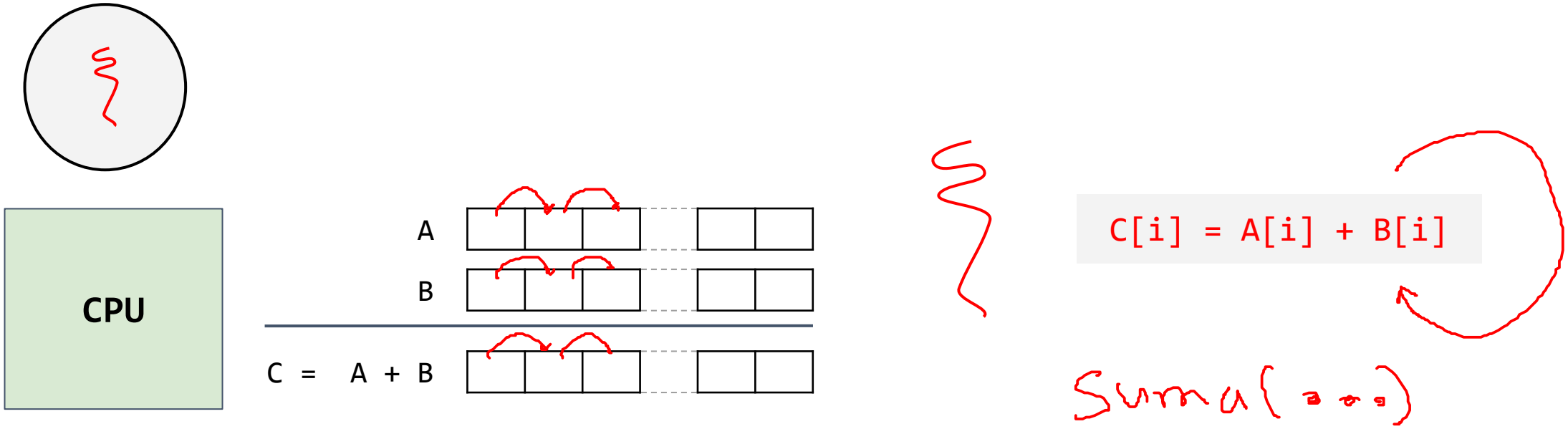


Hilos

¿Por qué se usan los hilos? - Paralelizar

Suponga que se desean sumar dos arreglos muy grandes (A y B) y llevar el resultado a un tercero. ¿Como se podría mejorar el desempeño de la aplicación?

Una sola CPU:

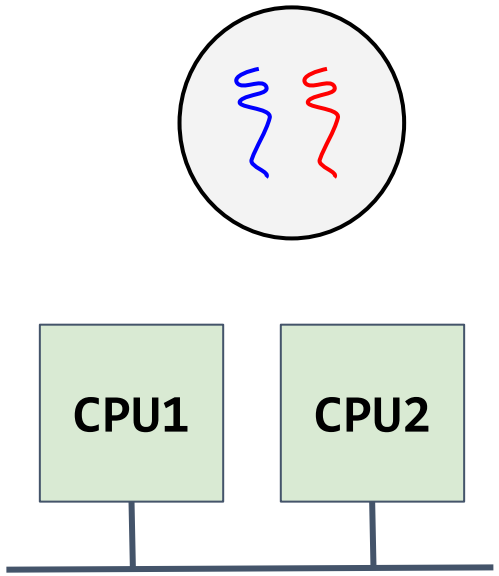


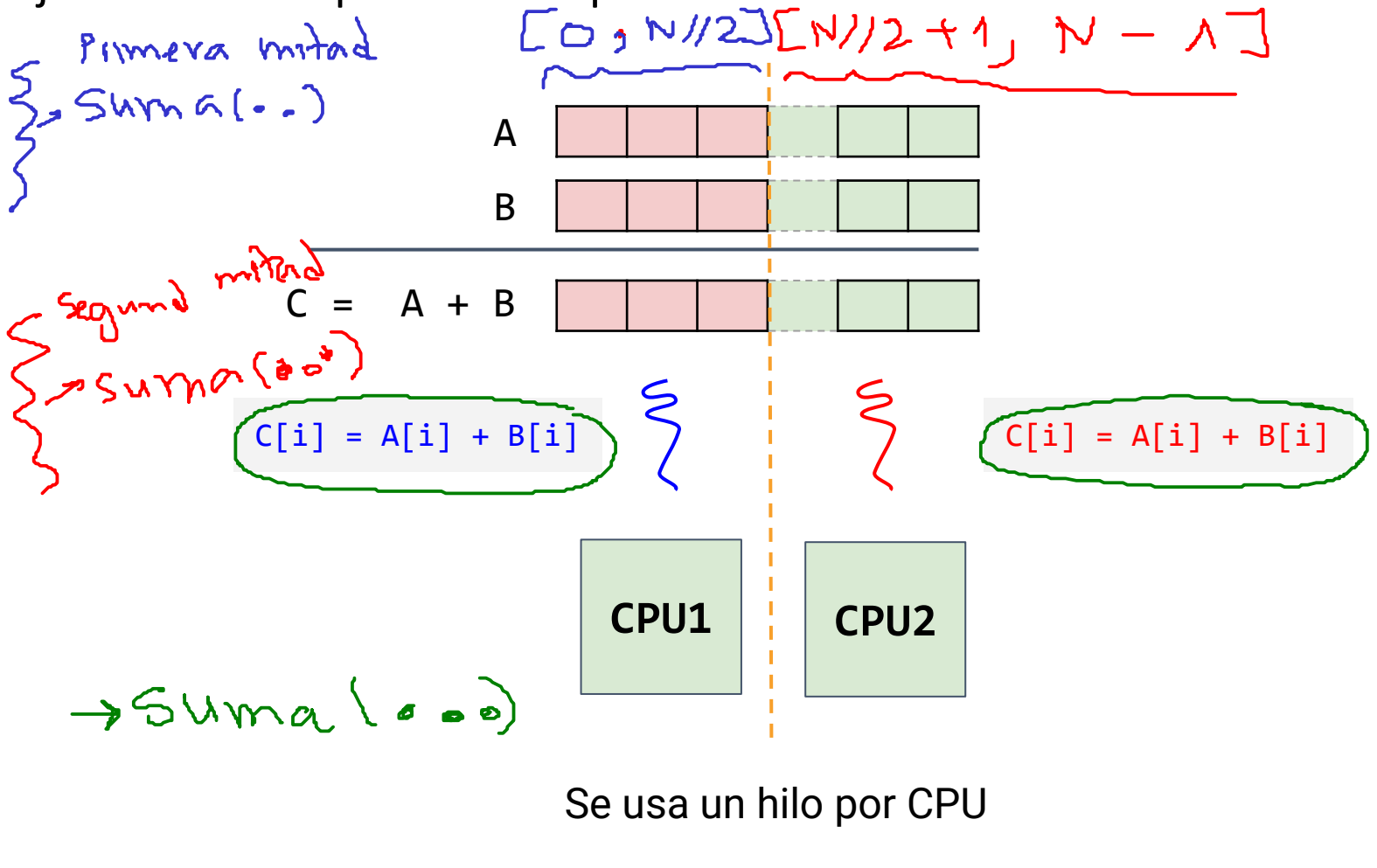
Hilos

¿Por qué se usan los hilos? - Paralelizar

Suponga que se desean sumar dos arreglos muy grandes (A y B) y llevar el resultado a un tercero. ¿Como se podría mejorar el desempeño de la aplicación?

Varias CPUs:



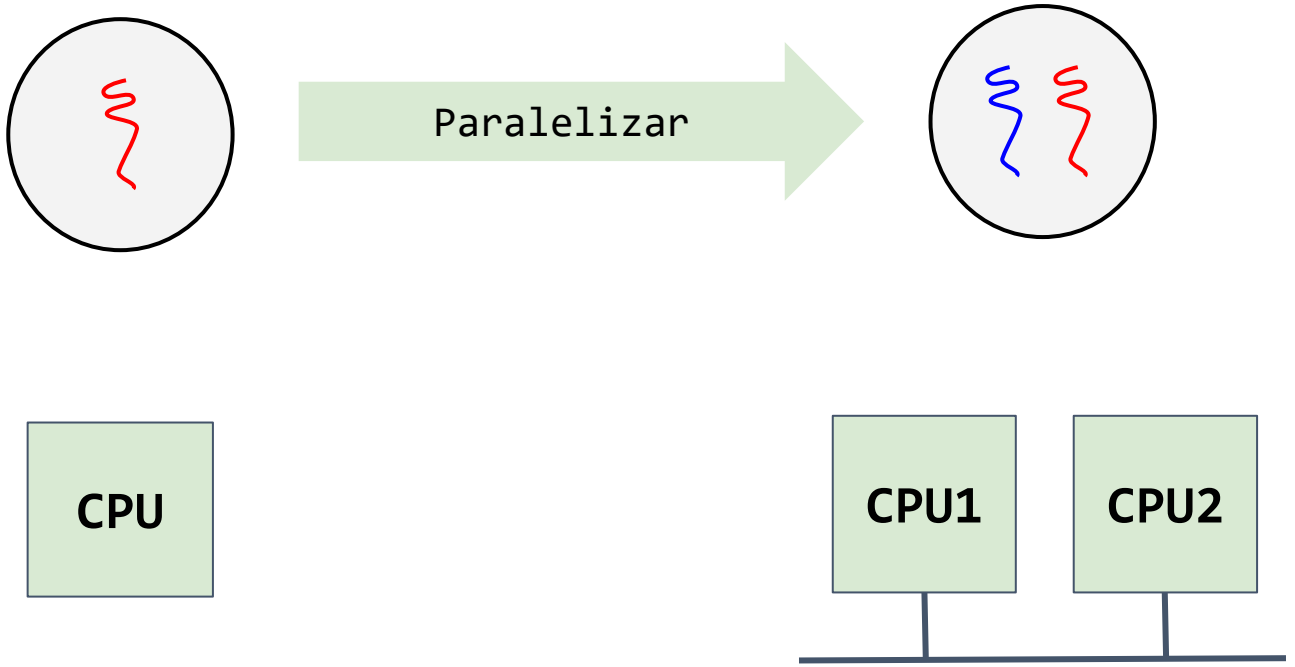


Se usa un hilo por CPU

Hilos

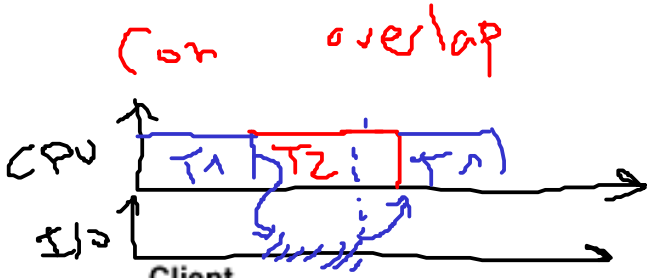
¿Por qué se usan los hilos? - Paralelizar

Paralelizar consiste en transformar un programa estándar de un solo hilo a **varios hilos** de ejecución.



Hilos

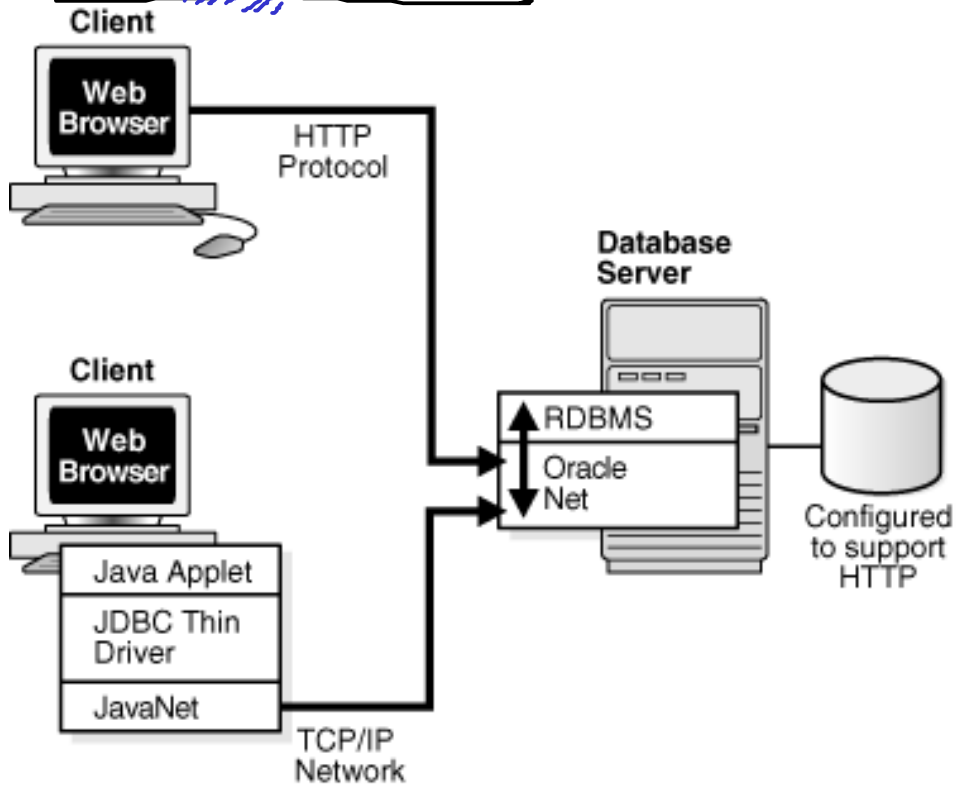




¿Por qué se usan los hilos? - Overlap

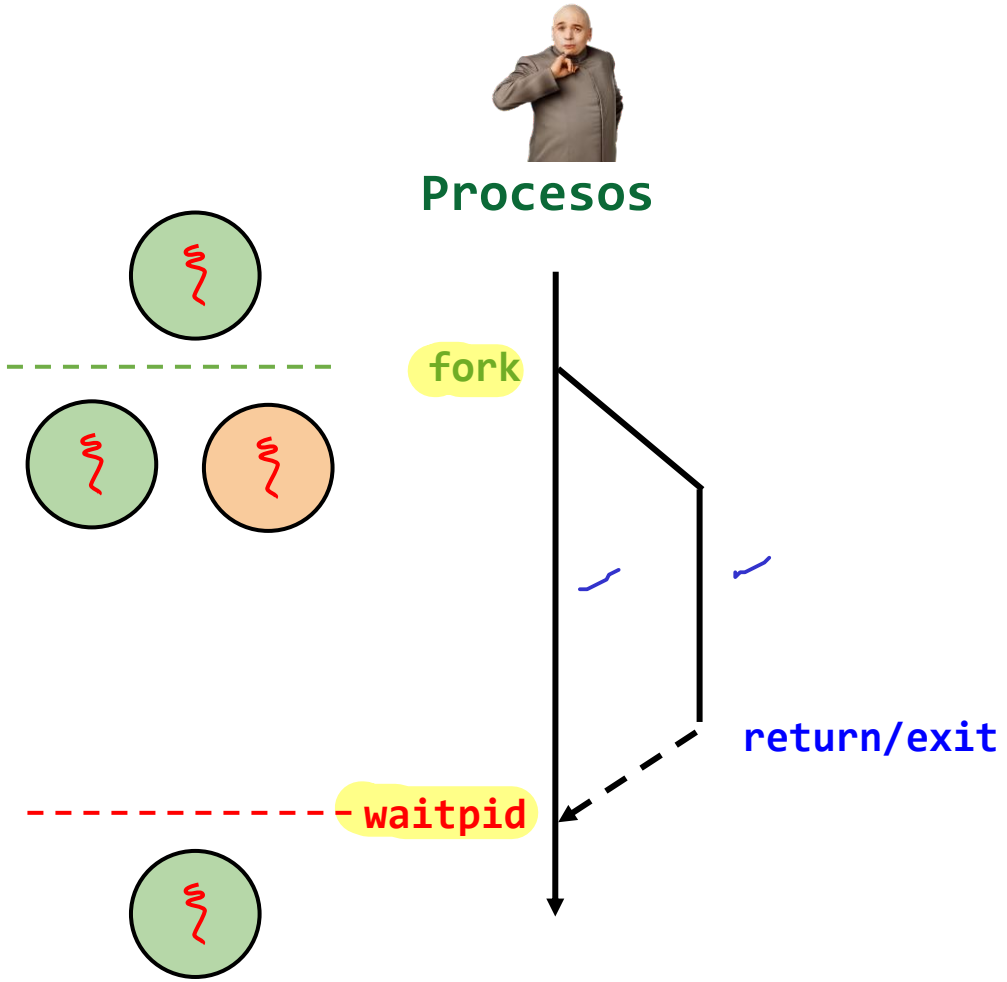
Evitar bloquear el progreso de un programa debido a una operación I/O más lenta pues al usar hilos:

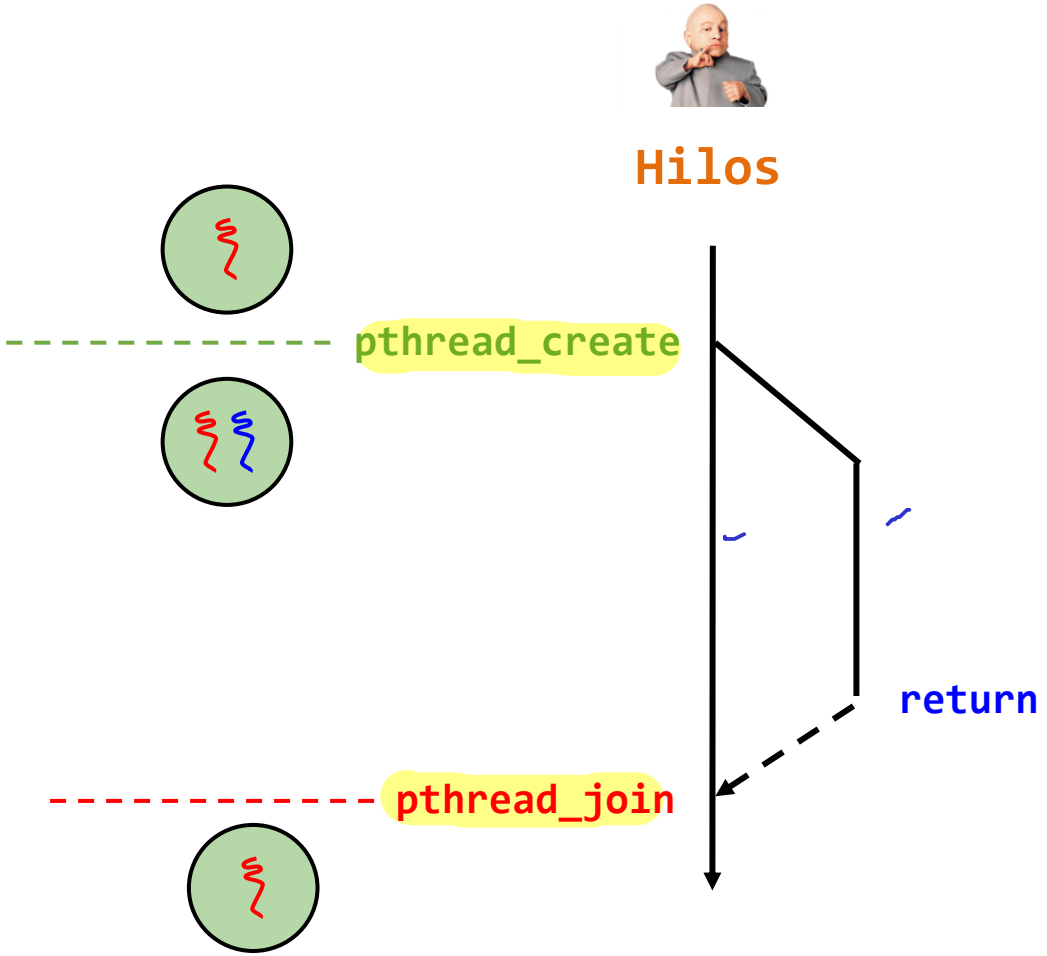
- No se hace necesario que un programa tenga que esperar que una operación I/O (la cual es muy lenta) se complete de modo que es posible que la CPU realice otras tareas.
- Casos típicos: Aplicaciones tipo cliente - servidor (web server, DBMS, etc).



Creación de hilos

- Existen similitudes en la gestión de hilos y procesos.



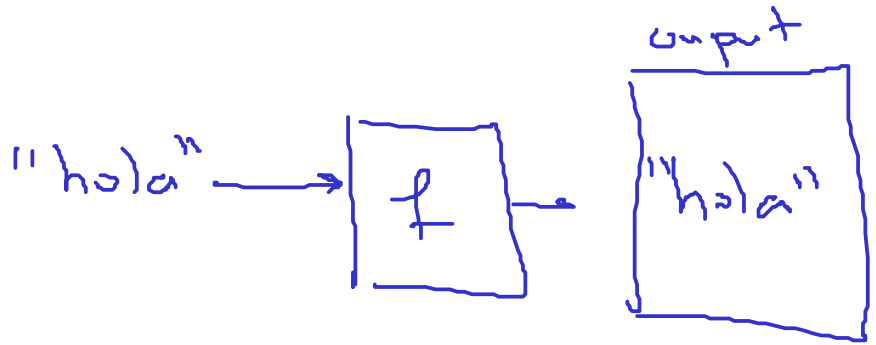


Creación de hilos

Ejemplo - Orden de ejecución

- El siguiente programa crea dos hilos independientes (**p1** y **p2**) que ejecutan una tarea (mythread)
- **Pregunta**: ¿Cuál es el orden de ejecución de los hilos?

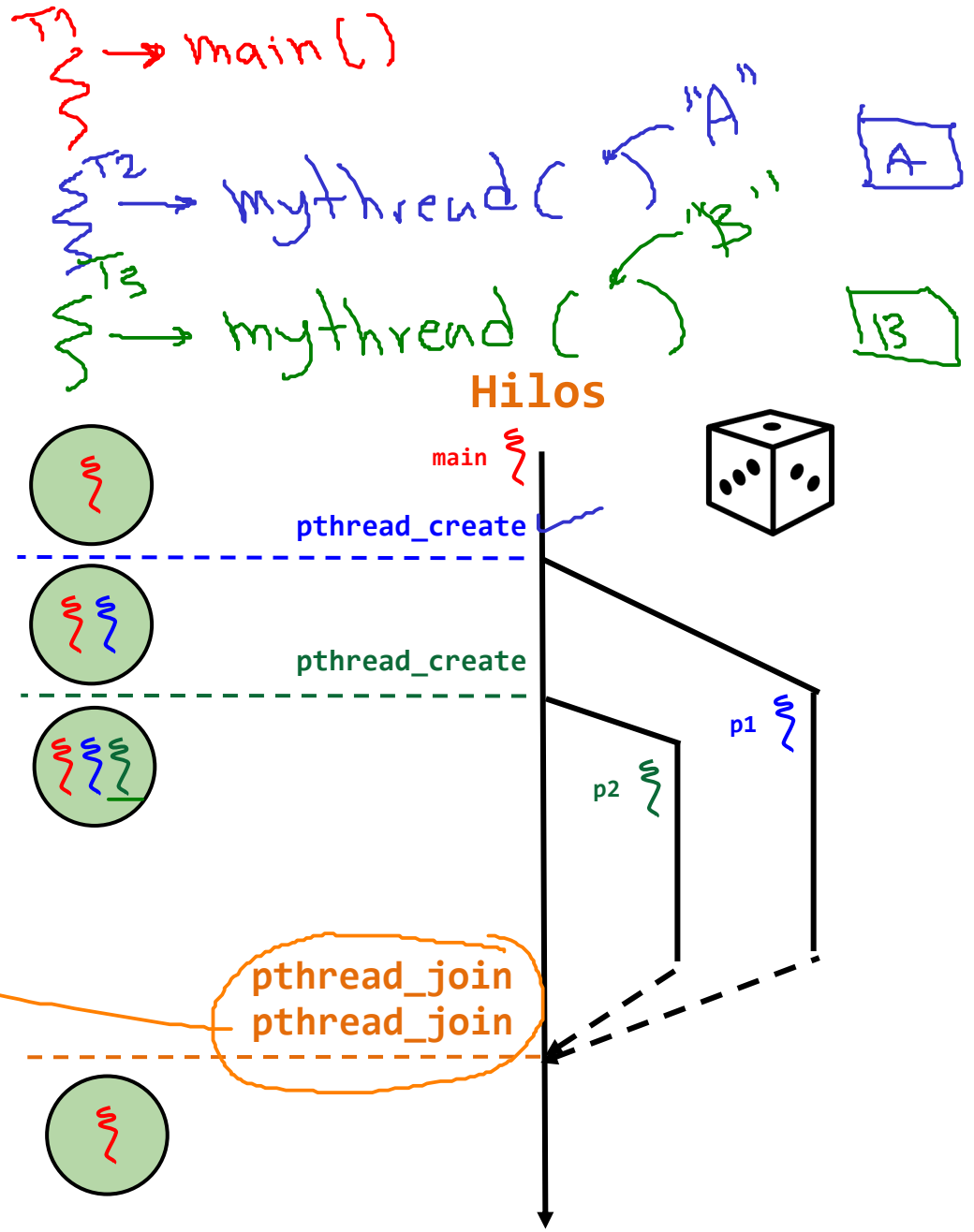
```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4
5  #include "common.h"
6  #include "common_threads.h"
7
8  void *mythread(void *arg) {
9      printf("%s\n", (char *) arg);
10     return NULL;
11 }
12
```



Creación de hilos

Ejemplo - Orden de ejecución

```
13  int main(int argc, char *argv[]) {
14      if (argc != 1) {
15          fprintf(stderr, "usage: main\n");
16          exit(1);
17      }
18
19      pthread_t p1, p2;
20      printf("main: begin\n");
21      Pthread_create(&p1, NULL, mythread, "A");
22      Pthread_create(&p2, NULL, mythread, "B");
23      // join waits for the threads to finish
24      Pthread_join(p1, NULL);
25      Pthread_join(p2, NULL);
26      printf("main: end\n");
27      return 0;
28  }
```



Creación de hilos

Ejemplo - Orden de ejecución

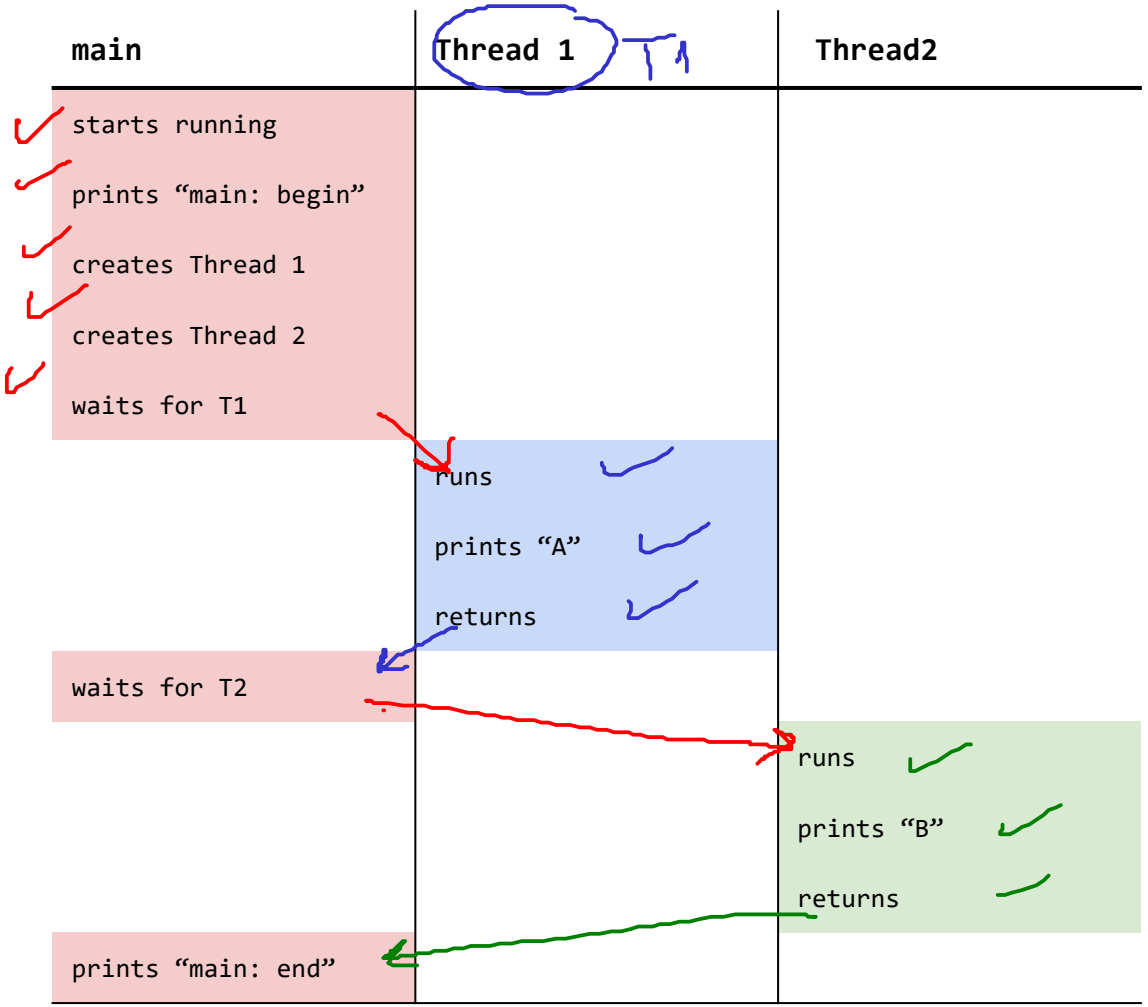
- **Pregunta**: ¿Cual hilo se ejecuta primero?
- **Respuesta**:
  - El orden es indeterminado (aleatorio).
  - Depende del scheduler.

La salida es **no determinista (Aleatoria)**



Creación de hilos

Ejemplo - Orden de ejecución - Traza 1

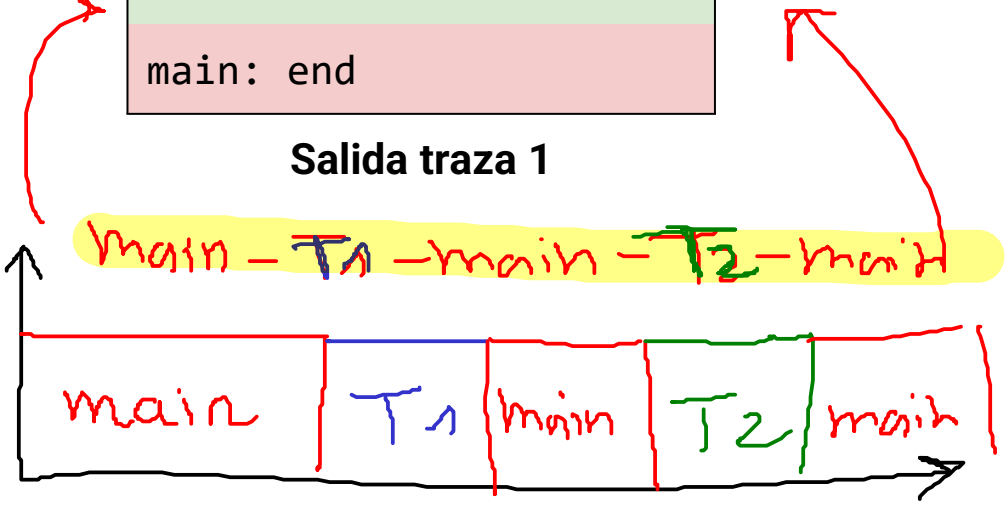


Thread Trace (1)

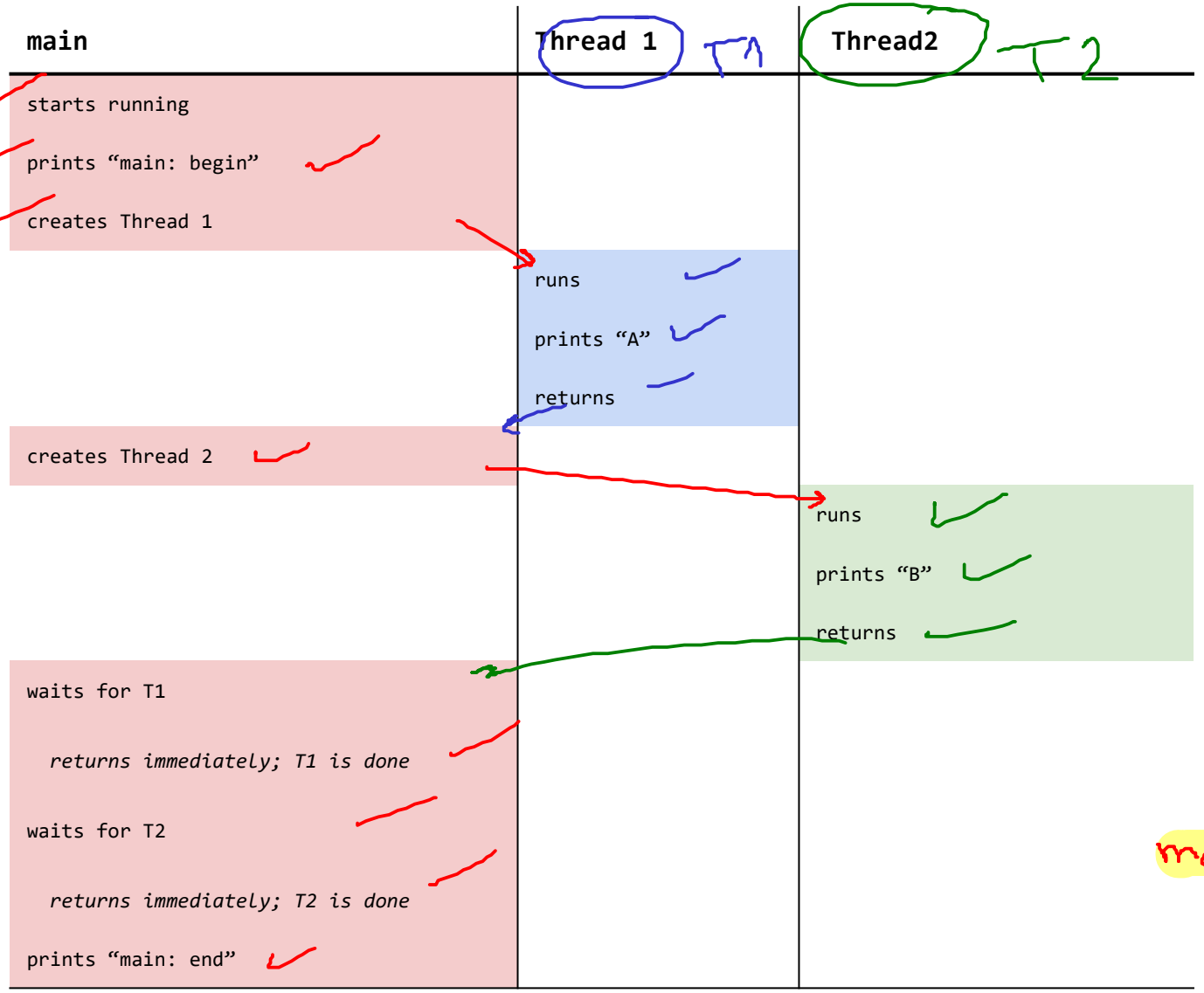


```
main: begin
A
B
main: end
```

Salida traza 1



Creación de hilos



Thread Trace (2)

Ejemplo - Orden de ejecución - Traza 2

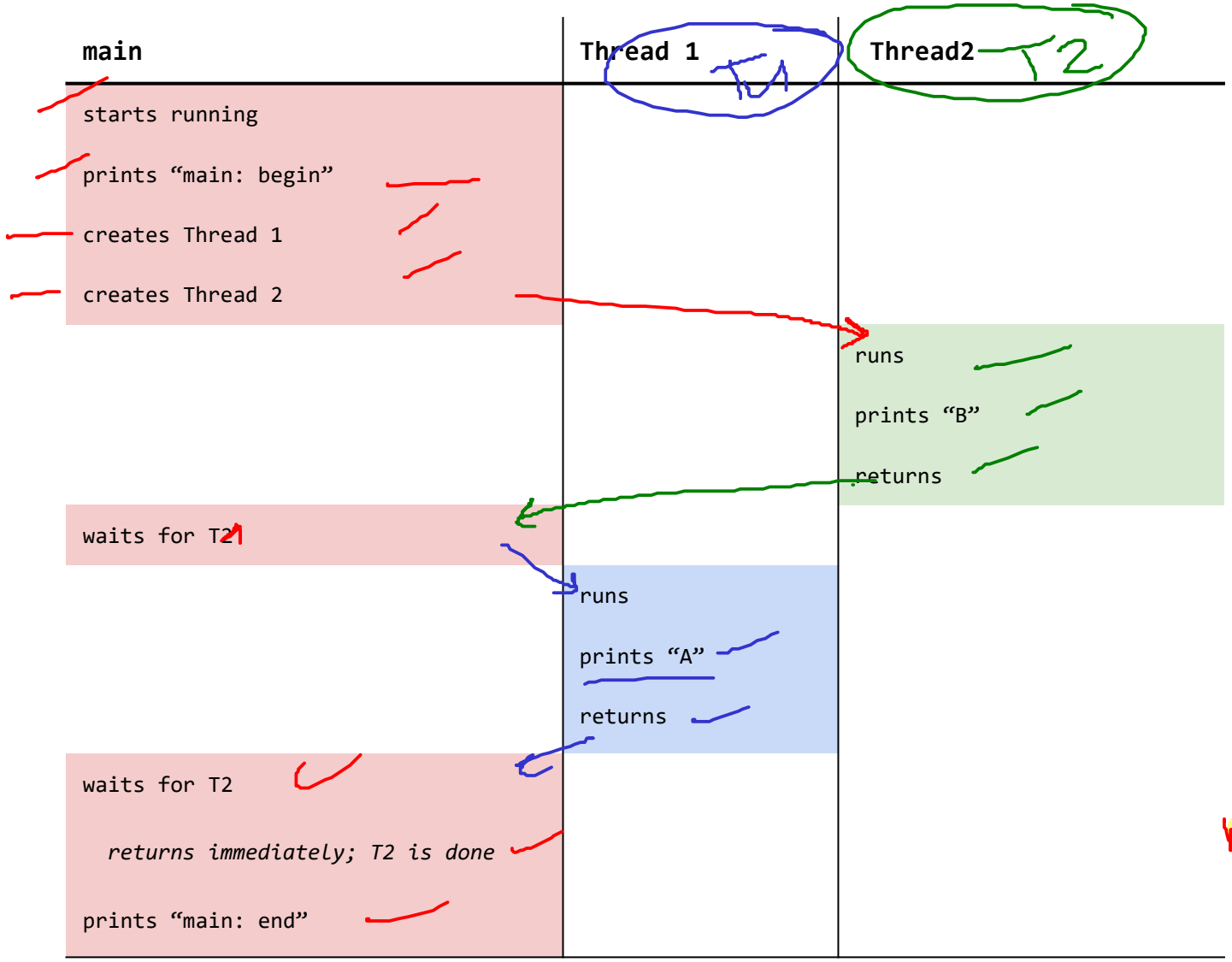


```
main: begin
A
B
main: end
```

Salida traza 2

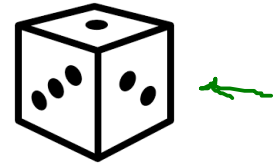
main – T1 – main – T2 – main

Creación de hilos



Thread Trace (3)

Ejemplo - Orden de ejecución - Traza 3



```
main: begin
B
A
main: end
```

Salida traza 3

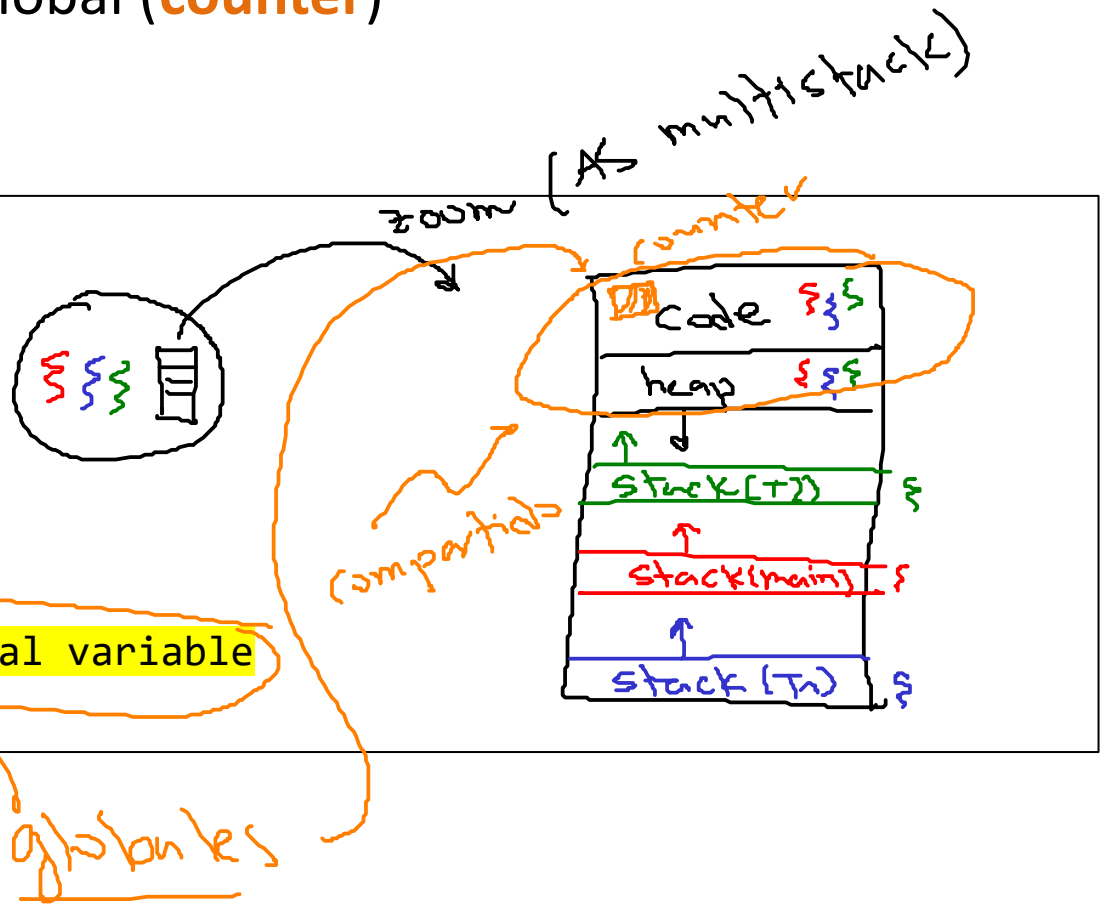
Datos compartidos

→ Fin: 15/10/2026
→ Inicio: 20/10/2026

Ejemplo - Datos compartidos

- El programa crea dos hilos independientes (**p1** y **p2**) que ejecutan una tarea (**mythread**)
- Ambos hilos intentan actualizar una variable global (**counter**)
- **Pregunta**: ¿Qué implicaciones trae esto?

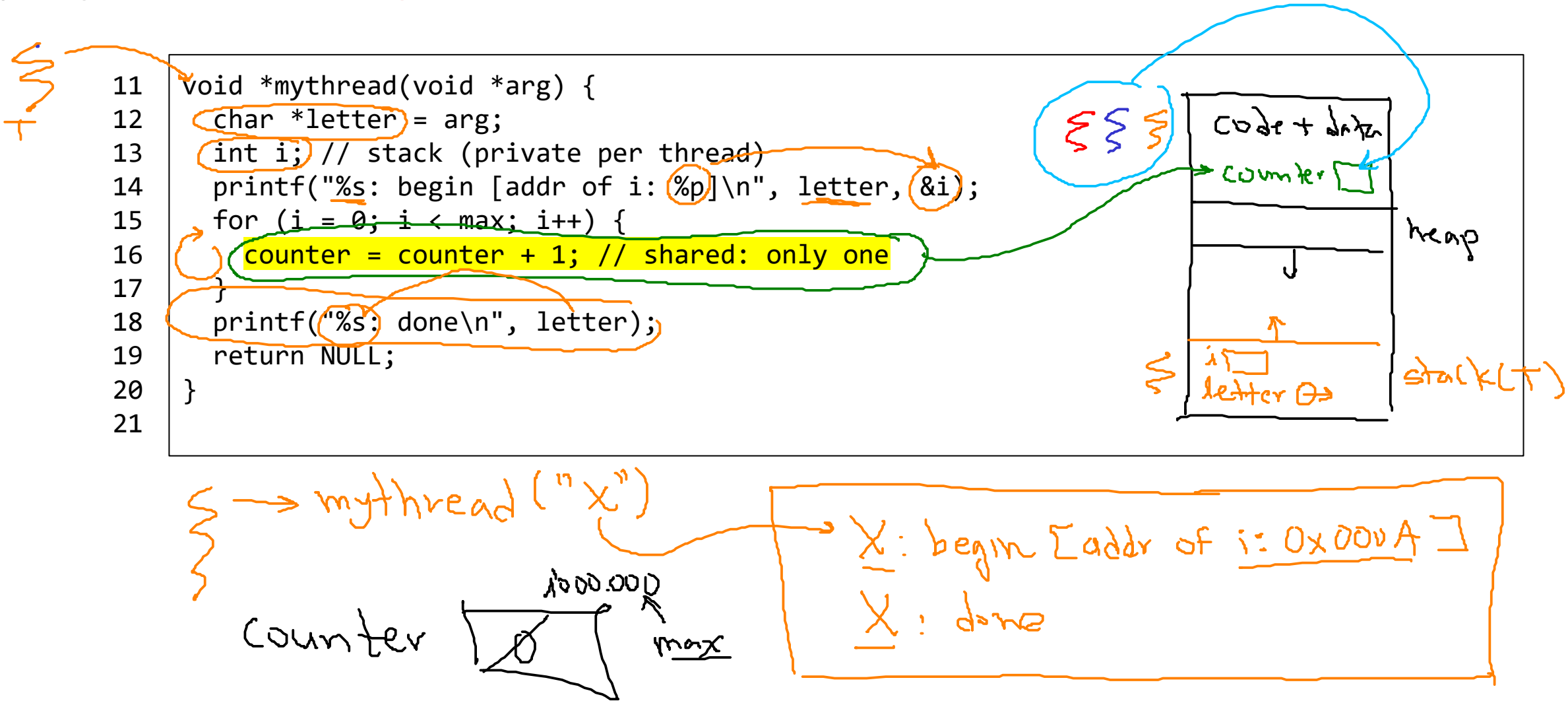
```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <pthread.h>
4
5   #include "common.h"
6   #include "common_threads.h"
7
8   int max;
9   volatile int counter = 0; // shared global variable
10
```



Datos compartidos

Ejemplo - Datos compartidos

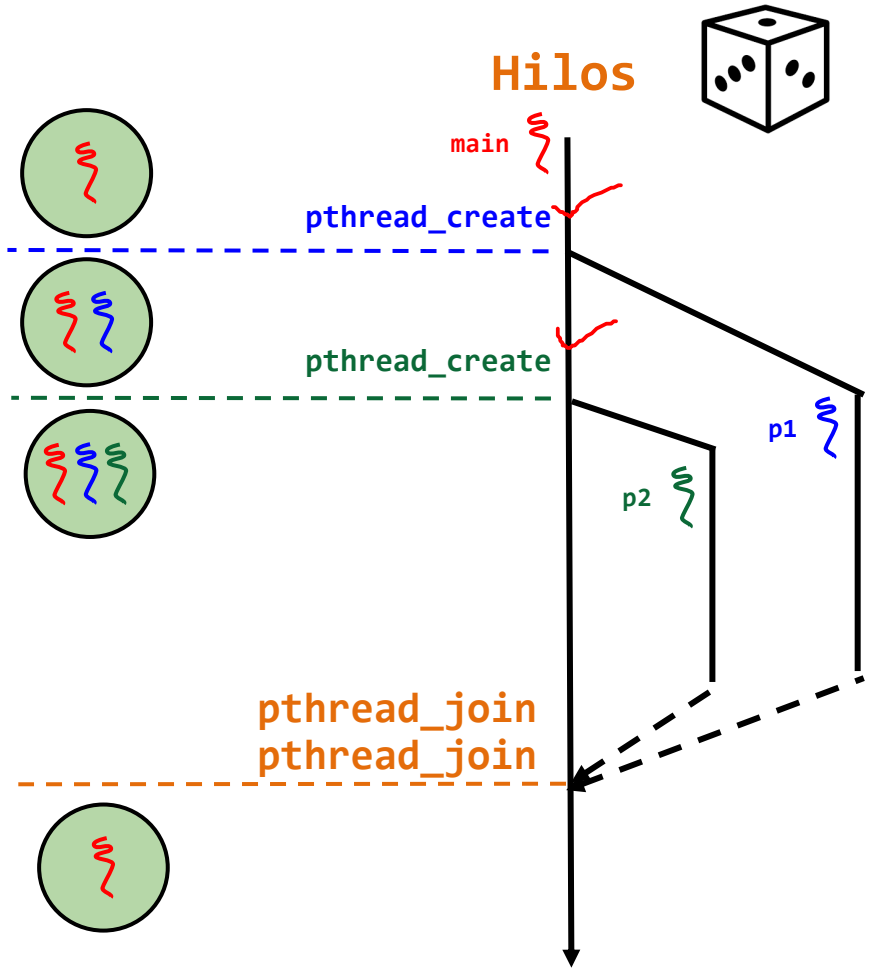
```
11  void *mythread(void *arg) {
12    char *letter = arg;
13    int i; // stack (private per thread)
14    printf("%s: begin [addr of i: %p]\n", letter, &i);
15    for (i = 0; i < max; i++) {
16      counter = counter + 1; // shared: only one
17    }
18    printf("%s: done\n", letter);
19    return NULL;
20  }
21
```



Datos compartidos

Ejemplo - Datos compartidos

```
22  int main(int argc, char *argv[]) {
23    if (argc != 2) {
24      fprintf(stderr, "usage: main-first <loopcount>\n");
25          exit(1);
26    }
27    max = atoi(argv[1]);
28
29    pthread_t p1, p2;
30    printf("main: begin [counter = %d] [%x]\n", counter,
31           (unsigned int) &counter);
32    Pthread_create(&p1, NULL, mythread, "A");
33    Pthread_create(&p2, NULL, mythread, "B");
34    // join waits for the threads to finish
35    Pthread_join(p1, NULL);
36    Pthread_join(p2, NULL);
37    printf("main: done\n [counter: %d]\n [should: %d]\n",
38           counter, max*2);
39    return 0;
40  }
```



Datos compartidos

Posibles resultados

Ejecución 1 - Salida

```
prompt> gcc -o main main.c -Wall -pthread; ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```



Ejecución 2 - Salida

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```



Ejecución 3 - Salida

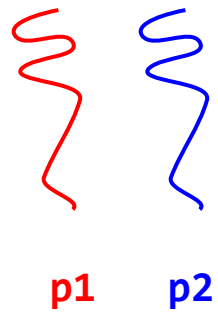
```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
```



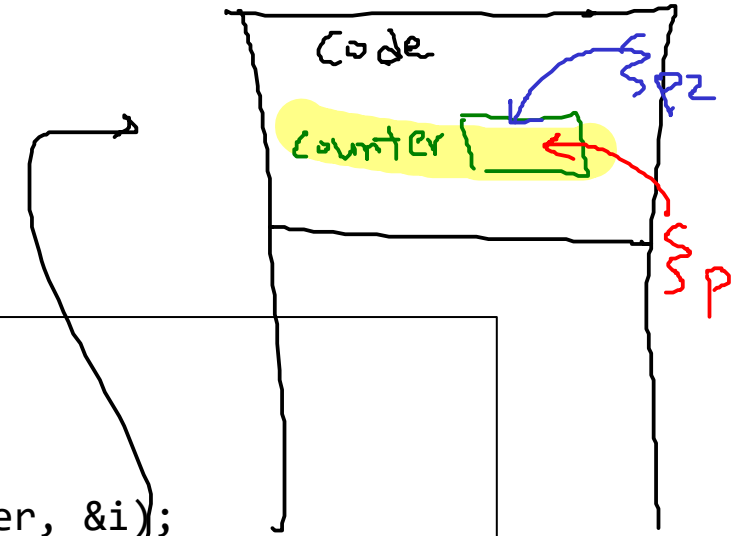
Datos compartidos

¿Por que estos resultados tan raros?

- Salida **no determinista**:
  - Diferente orden de ejecución para cada caso.
  - Diferentes valores desplegados en la variable **counter**.




```
void *mythread(void *arg) {
  char *letter = arg;
  int i; // stack (private per thread)
  printf("%s: begin [addr of i: %p]\n", letter, &i);
  for (i = 0; i < max; i++) {
    counter = counter + 1; // shared: only one
  }
  printf("%s: done\n", letter);
  return NULL;
}
```



SECCION CRITICA

```
counter = counter + 1;
```



```
100  mov 0x8049a1c, %eax
105  add $0x1, %eax
108  mov %eax, 0x8049a1c
```

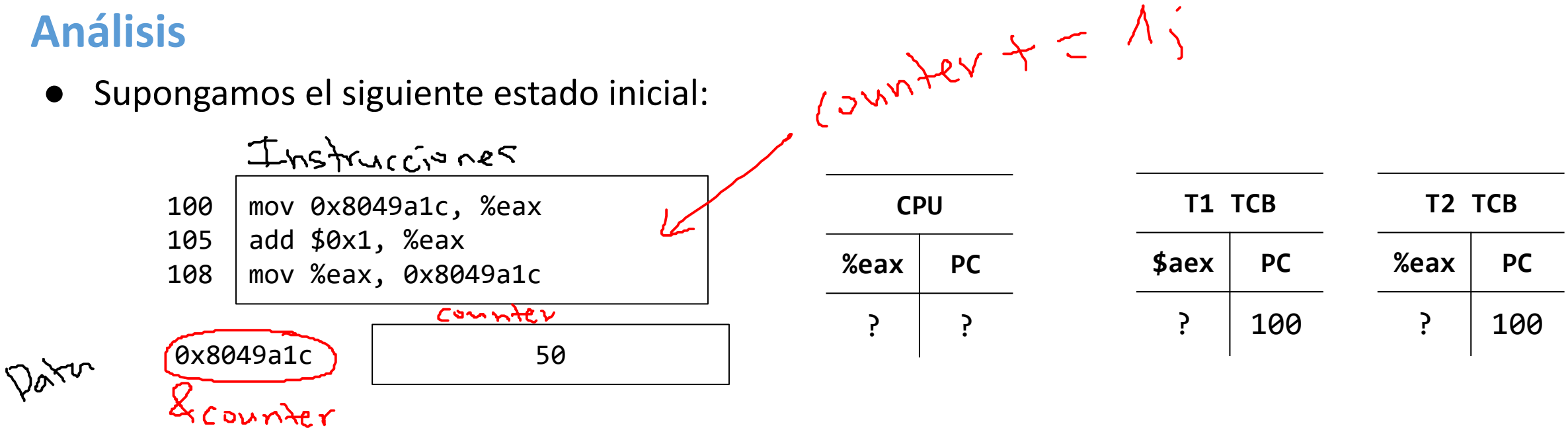
&counter

&counter

Datos compartidos

Análisis

- Supongamos el siguiente estado inicial:



CPU	
%eax	PC
?	?

T1 TCB	
$aex	PC
?	100

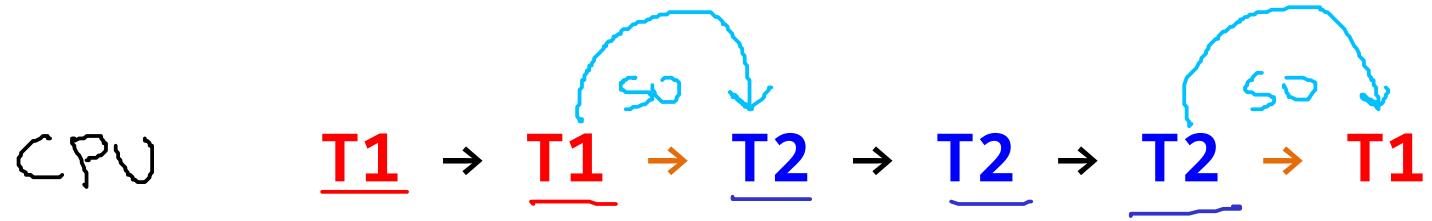
T2 TCB	
%eax	PC
?	100

- Supongamos que se lleva a cabo la siguiente secuencia de ejecución:

T1 → T1 → T2 → T2 → T2 → T1

Datos compartidos

Análisis




```
100   mov 0x8049a1c, %eax
105   add $0x1, %eax
108   mov %eax, 0x8049a1c
```

0x8049a1c | 50 → 51 → 51

CPU	
%eax	PC
? → 50 → 51 → ? → 50 → 51 → 51	? → 100 → 105 → 108 → 100 → 105 → 108 → 108

T1 TCB	
%eax	PC
? → 51	100 → 108

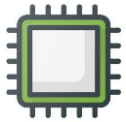
T2 TCB	
%eax	PC
? → 51	100 → 108

Deberia: counter = 52 ?

Real: counter = 51

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1

T2 T1

```
100  mov 0x8049a1c, %eax
105  add $0x1, %eax
108  mov %eax, 0x8049a1c
113
```

0x8049a1c	50

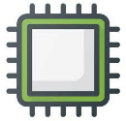
CPU	
%eax	PC
~~?~~	~~?~~ 100

T1 TCB	
%eax	PC
?	100

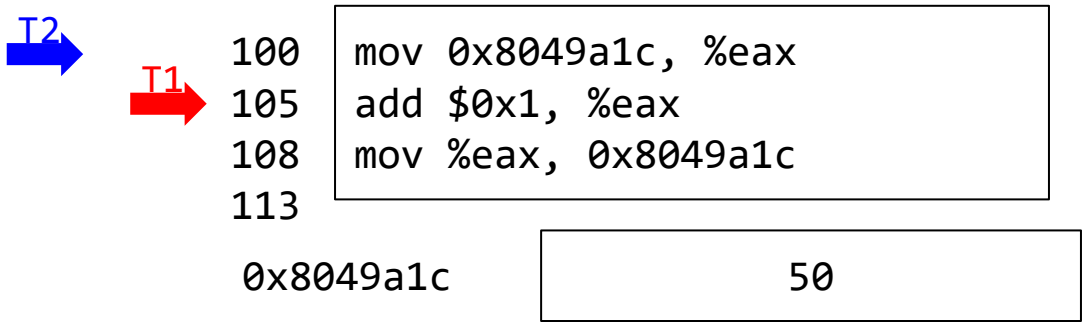
T2 TCB	
%eax	PC
?	100

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1



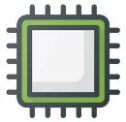
CPU	
%eax	PC
~~?~~ 50	~~?~~ ~~100~~ 105

T1 TCB	
%eax	PC
?	100

T2 TCB	
%eax	PC
?	100

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1

T2

T1

```
100   mov 0x8049a1c, %eax
105   add $0x1, %eax
108   mov %eax, 0x8049a1c
113
```

0x8049a1c	50

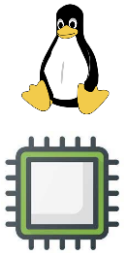
CPU	
%eax	PC
~~?~~	~~?~~
~~50~~	~~100~~
51	~~105~~
	108

T1 TCB	
%eax	PC
?	100

T2 TCB	
%eax	PC
?	100

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1

T2

T1

```
100   mov 0x8049a1c, %eax
105   add $0x1, %eax
108   mov %eax, 0x8049a1c
113
```

0x8049a1c	50

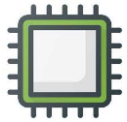
CPU	
%eax	PC
~~?~~	~~?~~
~~50~~	~~100~~
~~51~~	~~105~~
?	~~108~~
	100

T1 TCB	
%eax	PC
~~?~~	~~100~~
51	108

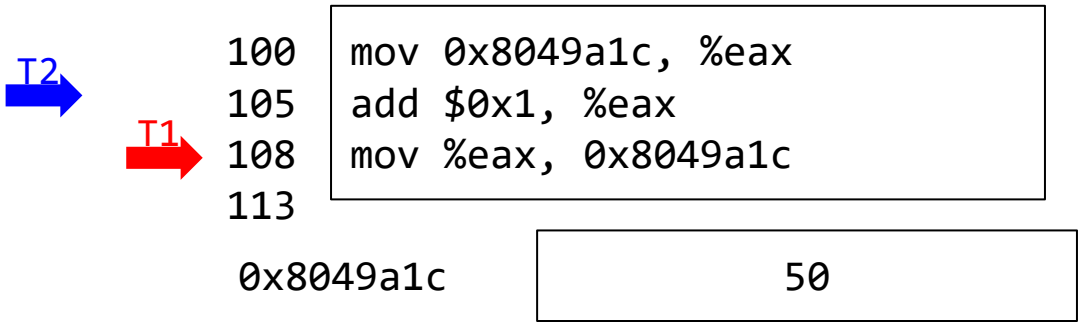
T2 TCB	
%eax	PC
?	100

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1



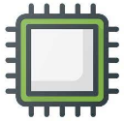
CPU	
%eax	Pc
~~?~~	~~?~~
~~50~~	~~100~~
~~51~~	~~105~~
~~?~~	~~108~~
50	~~100~~
	105

T1 TCB	
%eax	SP
~~?~~	~~100~~
51	108

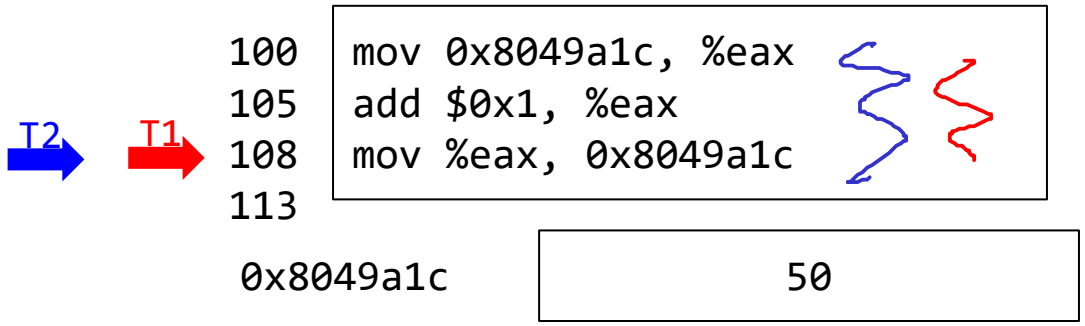
T2 TCB	
%eax	SP
?	100

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1



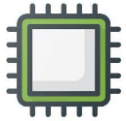
CPU	
%eax	PC
~~?~~	~~?~~
~~50~~	~~100~~
~~51~~	~~105~~
~~?~~	~~108~~
~~50~~	~~100~~
51	~~105~~
	108

T1 TCB	
%eax	SP
~~?~~	~~100~~
51	108

T2 TCB	
%eax	PC
?	100

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1

T2

T1

```
100   mov 0x8049a1c, %eax
105   add $0x1, %eax
108   mov %eax, 0x8049a1c
113
```

0x8049a1c
~~50~~
51

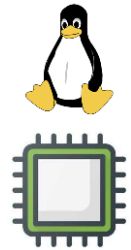
CPU	
%eax	PC
~~?~~	~~?~~
~~50~~	~~100~~
~~51~~	~~105~~
~~?~~	~~108~~
~~50~~	~~100~~
51	~~105~~
	~~108~~
	113

T1 TCB	
%eax	SP
~~?~~	~~100~~
51	108

T2 TCB	
%eax	PC
?	100

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1

T2

T1

```
100   mov 0x8049a1c, %eax
105   add $0x1, %eax
108   mov %eax, 0x8049a1c
113
```

0x8049a1c: ~~50~~ 51

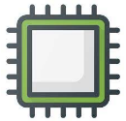
CPU	
%eax	PC
~~?~~	~~?~~
~~50~~	~~100~~
~~51~~	~~105~~
~~?~~	~~108~~
~~50~~	~~100~~
~~51~~	~~105~~
51	~~108~~
	~~113~~
	108

T1 TCB	
%eax	PC
~~?~~	~~100~~
51	108

T2 TCB	
%eax	PC
~~?~~	~~100~~
51	113

Datos compartidos

Análisis - Solución paso a paso



T1 → T1 → T2 → T2 → T2 → T1

T2 T1

```
100  mov 0x8049a1c, %eax
105  add $0x1, %eax
108  mov %eax, 0x8049a1c
113
```

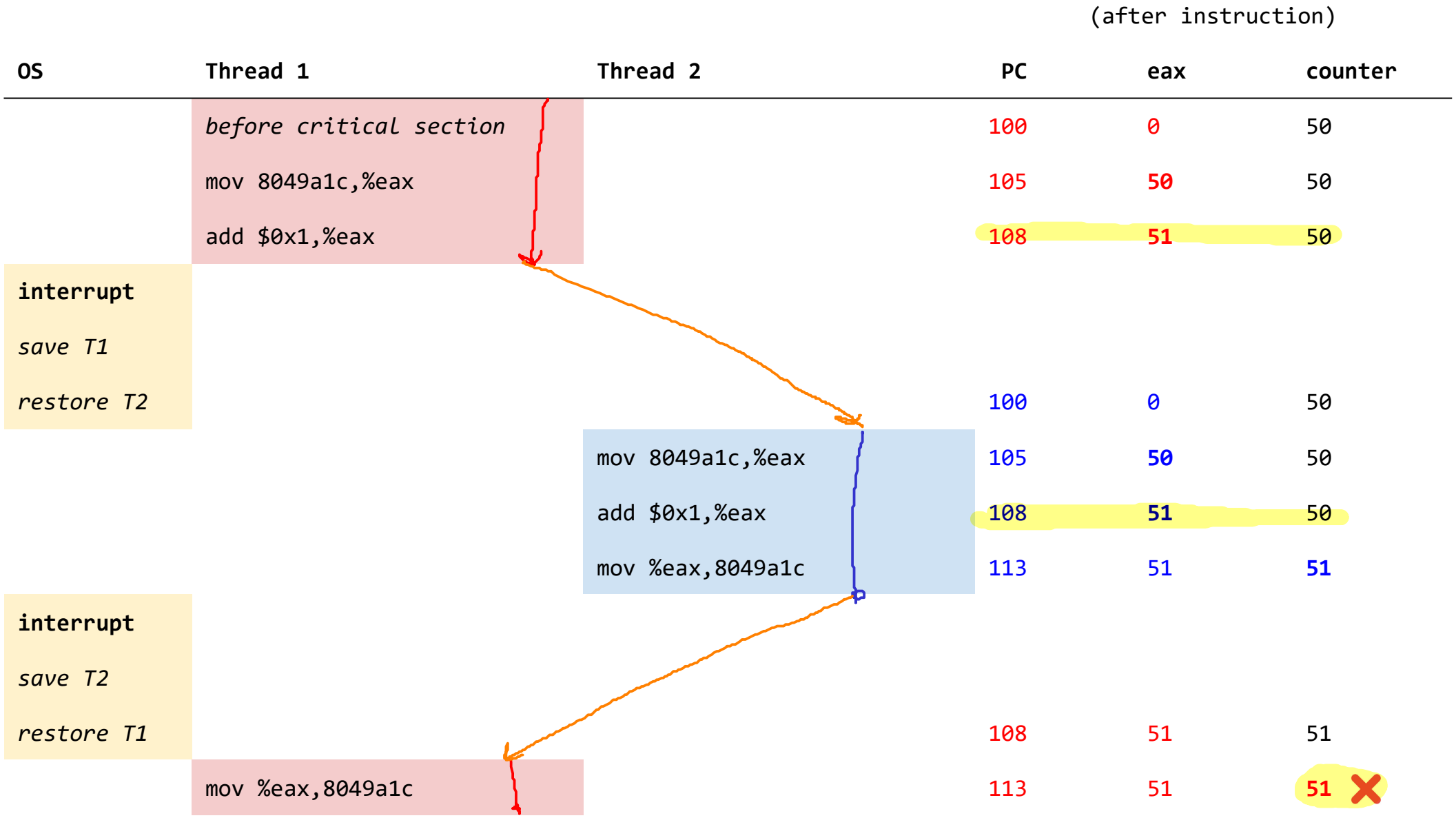
0x8049a1c
~~50~~
~~51~~
51

CPU	
%eax	PC
~~?~~	~~?~~
~~50~~	~~100~~
~~51~~	~~105~~
~~?~~	~~108~~
~~50~~	~~100~~
~~51~~	~~105~~
51	~~108~~
	~~113~~
	~~108~~
	113

T1 TCB	
%eax	PC
~~?~~	~~100~~
51	108

T2 TCB	
%eax	Pc
~~?~~	~~100~~
51	113

Datos compartidos



Datos compartidos

Conclusiones del análisis

- En el ejemplo con los dos hilos se analizó la instrucción:

```
counter = counter + 1
```

- Se asumió que la variable counter tenía un valor de `50` (por defecto).
- Tras la ejecución de esta instrucción por parte de ambos hilos se espera valor final de `52`, pero el resultado fue de **51**

¿Que paso entonces?

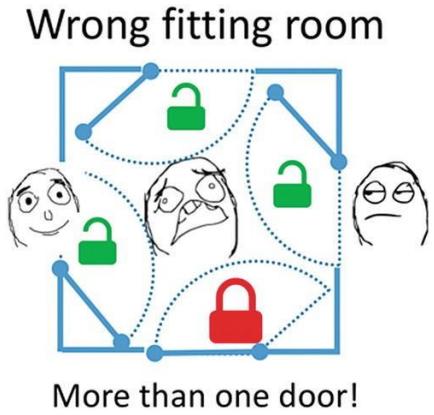
Datos compartidos

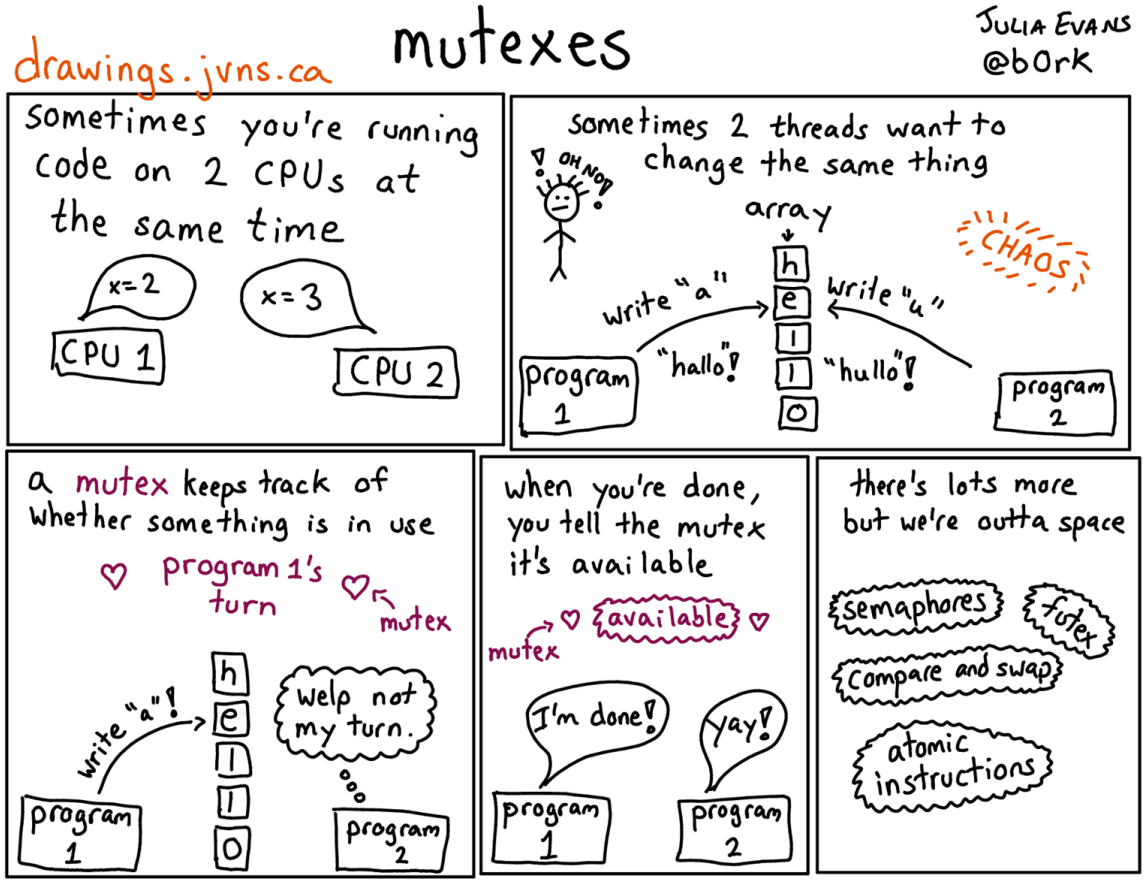
Condición de carrera (Race condition - Data race)

- Los dos hilos compiten modificando variables compartidas.
- Resultados no deterministas (por eso no dio lo que esperamos).

Sección crítica

- Los dos hilos compiten modificando variables compartidas.
- Resultados no deterministas (por eso no dio lo que esperamos).





link

Datos compartidos

Sección crítica

Porción de código donde se accede a una variable compartida.

```
void *mythread(void *arg) {
  char *letter = arg;
  int i; // stack (private per thread)
  printf("%s: begin [addr of i: %p]\n", letter, &i);
  for (i = 0; i < max; i++) {
    counter = counter + 1; // sección critica
  }
  printf("%s: done\n", letter);
  return NULL;
}
```

```
counter = counter + 1;
```

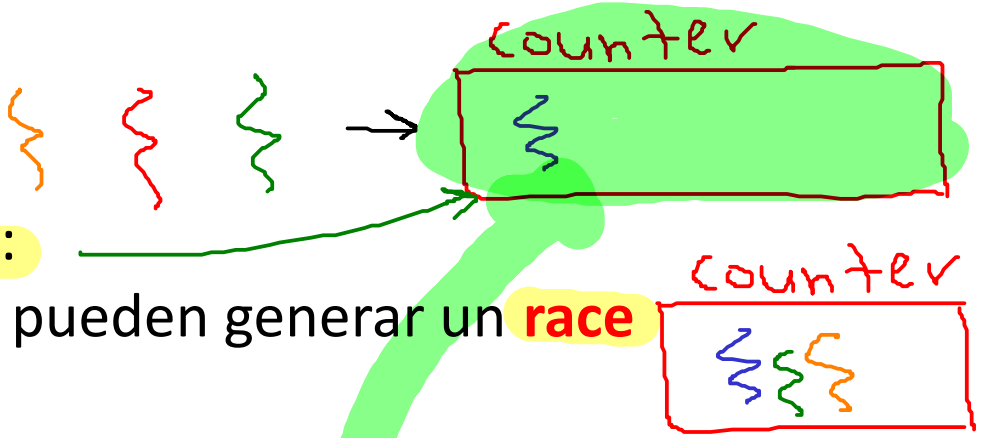


```
100  mov 0x8049a1c, %eax
105  add $0x1, %eax
108  mov %eax, 0x8049a1c
```

Datos compartidos

Sección crítica

- No debería ser accedida de manera concurrente:
  - Múltiples hilos ejecutando una región crítica pueden generar un **race condition**.
- Se requiere el **soporte de atomicidad** (instrucciones especiales que no pueden ser interrumpidas) para las regiones críticas
  - **Exclusión mútua**: Propiedad que garantiza que un solo hilo se encuentre ejecutando el código de la región crítica



No hay atomicidad

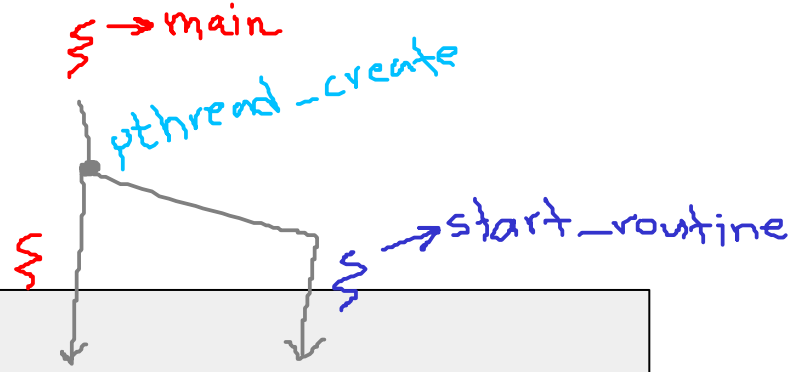
```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

Instrucción atómica

```
memory-add 0x8049a1c, $0x1
```

Creación de hilos





¿Como crear hilos?

```
#include <pthread.h>

int pthread_create(pthread_t* thread,
                   const pthread_attr_t* attr,
                   void* (*start_routine)(void*),
                   void* arg);
```


Donde:

- **thread**: permite manejar del hilo (***handler***).
- **attr**: permite crear atributos al hilo.
  - Tamaño del stack, prioridad en el planificador, ...
- **start_routine**: función que inicia ejecutando este hilo.
- **arg**: argumento pasado a la función (**start_routine**)
  - Un ***apuntador*** a **void** permite pasar argumentos de ***cualquier tipo***.

Creación de hilos

¿Como crear hilos?

- Si **start_routine** requiere argumentos de diferente tipo, la declaración sería como se muestra a continuación:
  - Un ***argumento*** tipo entero (**int**)

```
int pthread_create(..., // first two args are the same
                   void* (*start_routine)(int),
                   int arg);
```

  - Un ***retorno*** tipo entero (**int**)

```
int pthread_create(..., // first two args are the same
                   int (*start_routine)(void*),
                   void* arg);
```

Creación de hilos

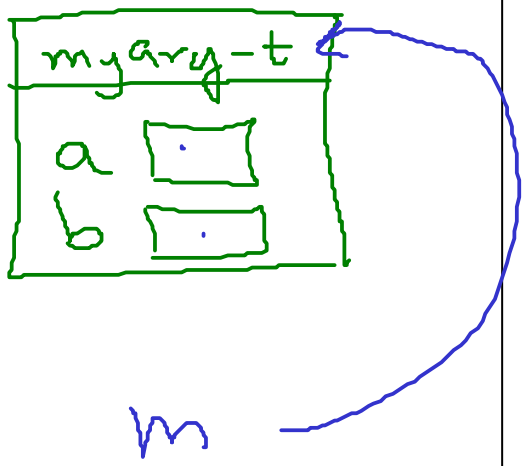
Ejemplo - Creando un hilo

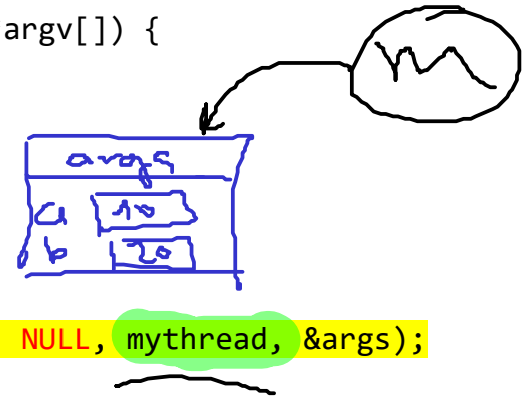
```c
#include <pthread.h>

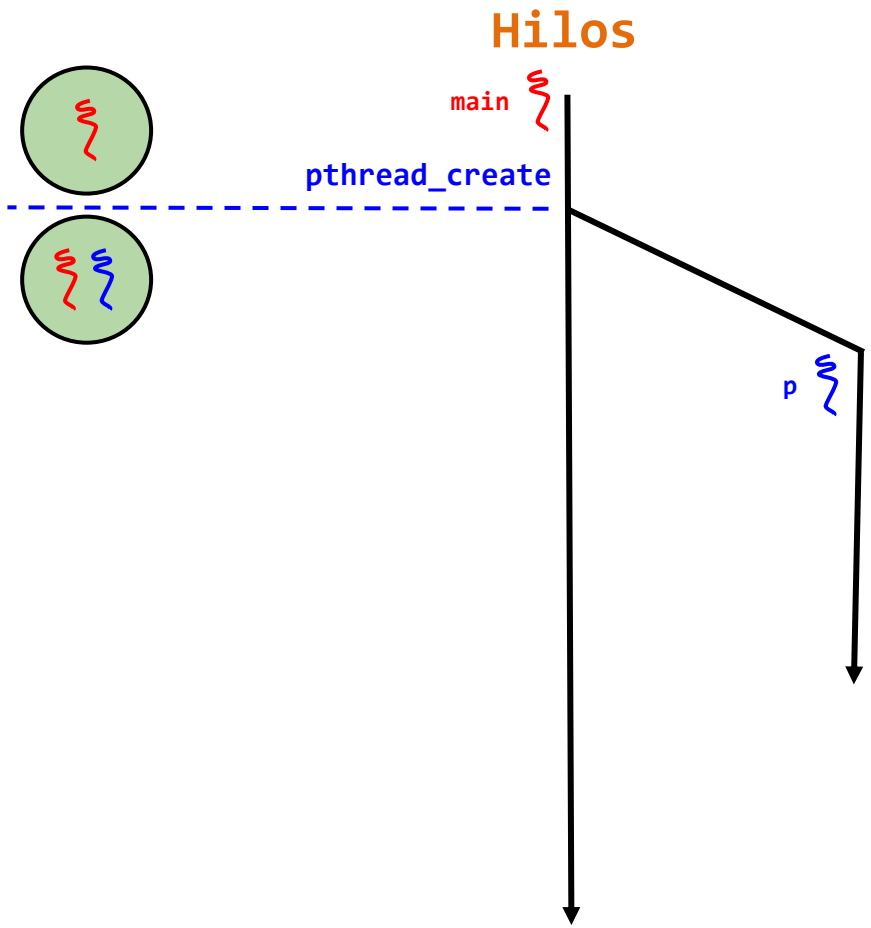
typedef struct __myarg_t {
  int a;
  int b;
} myarg_t;

void *mythread(void *arg) {
  myarg_t *m = (myarg_t *) arg;
  printf("%d %d\n", m->a, m->b);
  return NULL;
}

int main(int argc, char *argv[]) {
  pthread_t p;
  int rc;
  myarg_t args;
  args.a = 10;
  args.b = 20;
  rc = pthread_create(&p, NULL, mythread, &args);
  ...
}
```





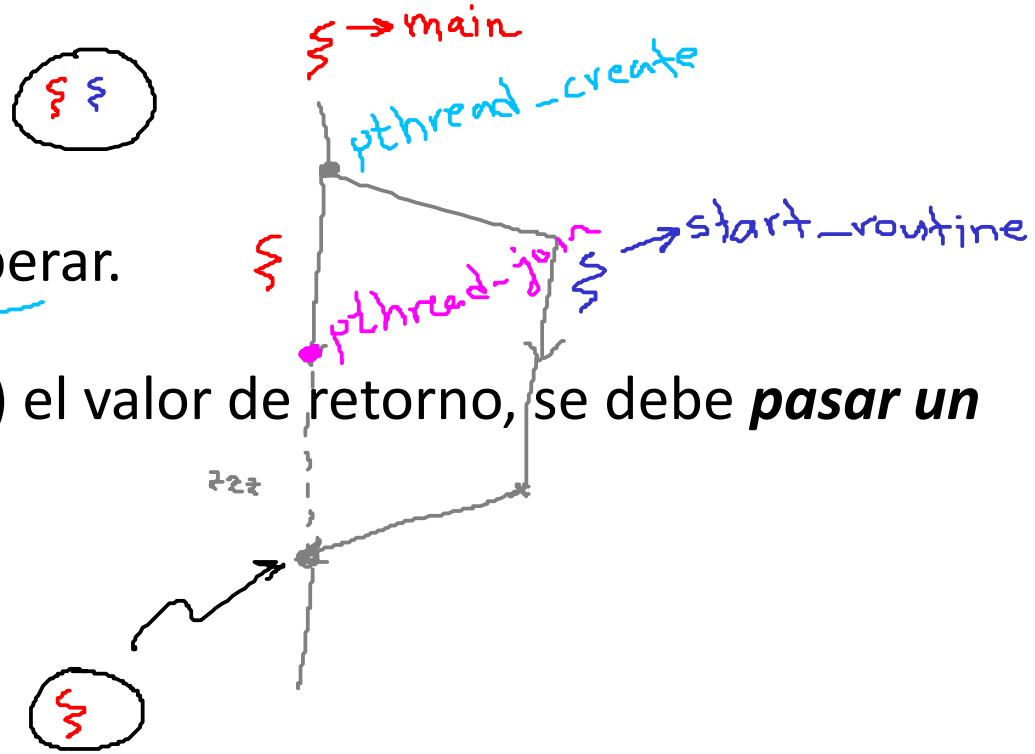


Esperar hasta que se complete el hilo

¿Como crear hilos?

Para esperar hacer que un hilo espere la terminación de otro para acabar se emplea **pthread_join**:

```
int pthread_join(pthread_t thread, void **value_ptr);
```



Donde:

- **thread**: Especifica el hilo por el cual se va a esperar.
- **value_ptr**: un apuntador al valor de retorno.
  - Como **pthread_join** cambia (internamente) el valor de retorno, se debe ***pasar un apuntador*** a ese valor.

Esperar hasta que se complete el hilo

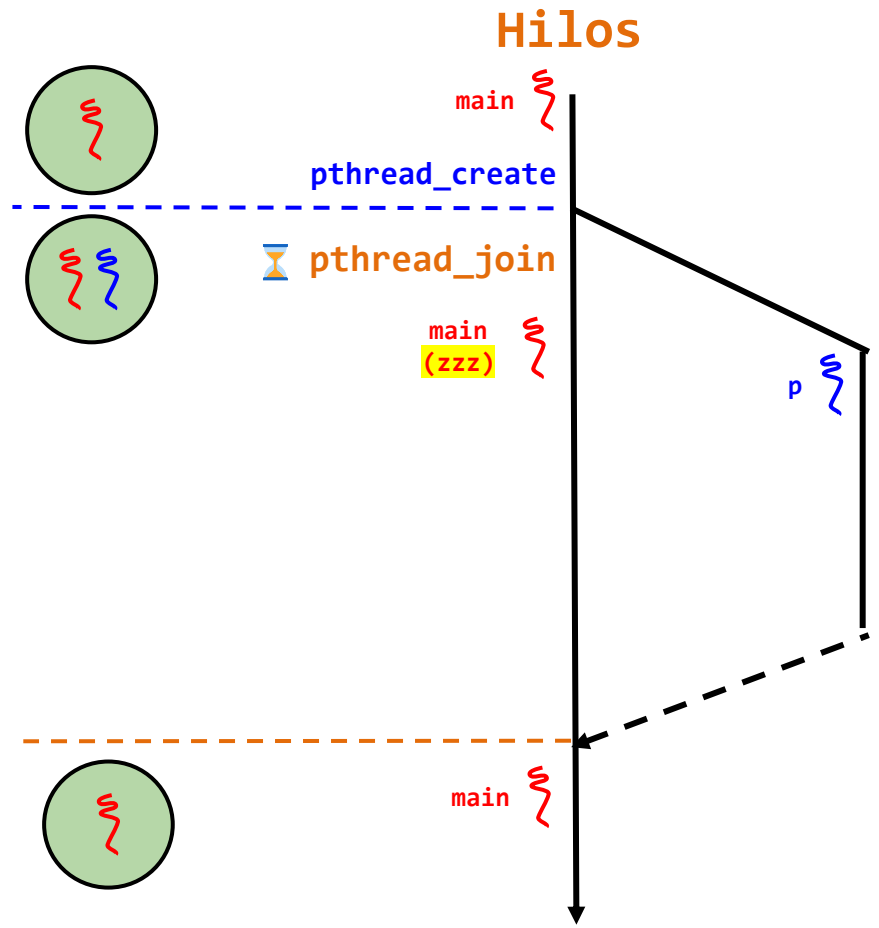
Ejemplo - Esperando a que un hilo se complete

```
#include <pthread.h>

typedef struct __myarg_t {
  int a;
  int b;
} myarg_t;

typedef struct __myret_t {
  int x;
  int y;
} myret_t;

void *mythread(void *arg) {
  myarg_t *m = (myarg_t *) arg;
  printf("%d %d\n", m->a, m->b);
  myret_t *r = malloc(sizeof(myret_t));
  r->x = 1;
  r->y = 2;
  return (void *) r;
}
...
```

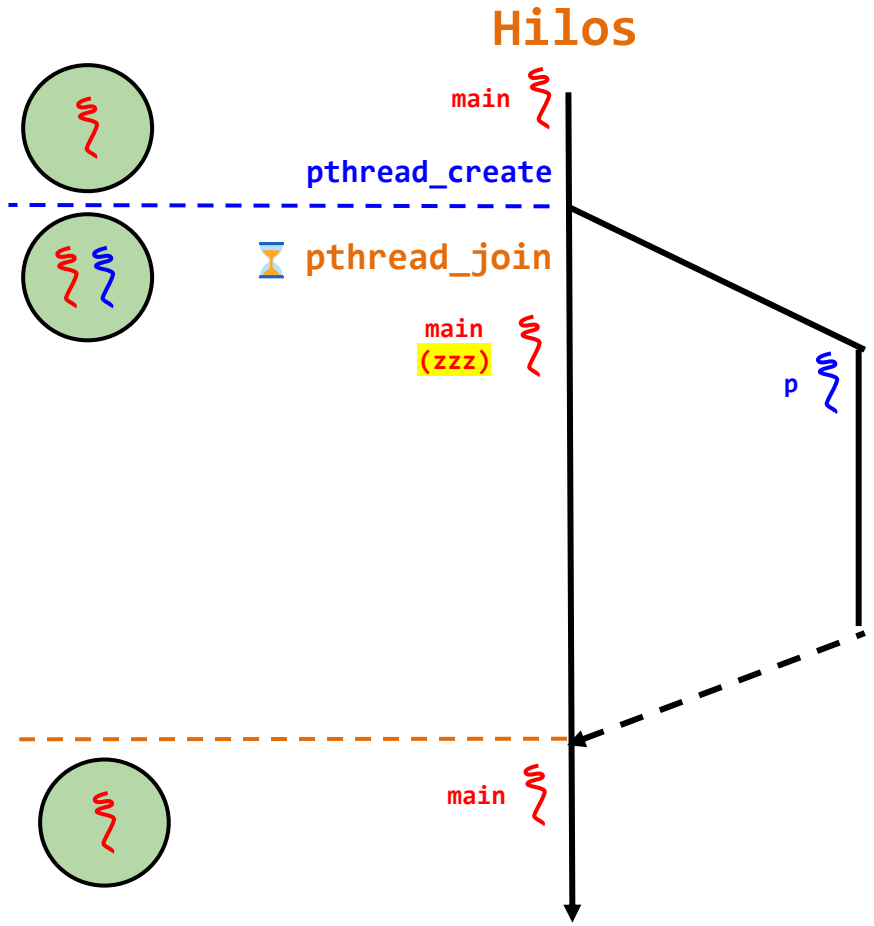


Esperar hasta que se complete el hilo

Ejemplo - Esperando a que un hilo se complete

```
...
int main(int argc, char *argv[]) {
  int rc;
  pthread_t p;
  myret_t *m;

  myarg_t args;
  args.a = 10;
  args.b = 20;
  pthread_create(&p, NULL, mythread, &args);
  pthread_join(p, (void **) &m); // this thread has been
                                 // waiting inside of the
                                 // pthread_join() routine.
  printf("returned %d %d\n", m->x, m->y);
  return 0;
}
```

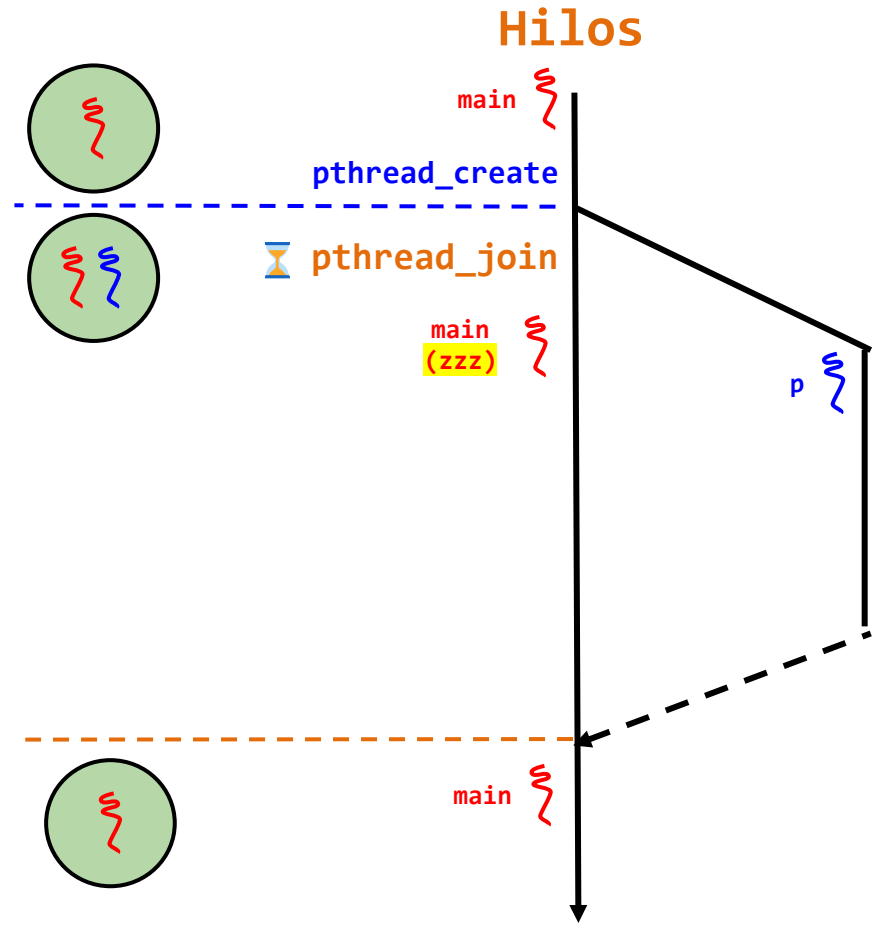


Esperar hasta que se complete el hilo

Ejemplo - Esperando a que un hilo se complete

Tenga cuidado con ***cómo se devuelven*** los valores desde un hilo.

```
void *mythread(void *arg) {
  myarg_t *m = (myarg_t *) arg;
  printf("%d %d\n", m->a, m->b);
  myret_t r; // ALLOCATED ON STACK: BAD!
  r.x = 1;
  r.y = 2;
  return (void *) &r;
}
```



Cuando la variable **r** retorna, la memoria es **liberada automáticamente**.

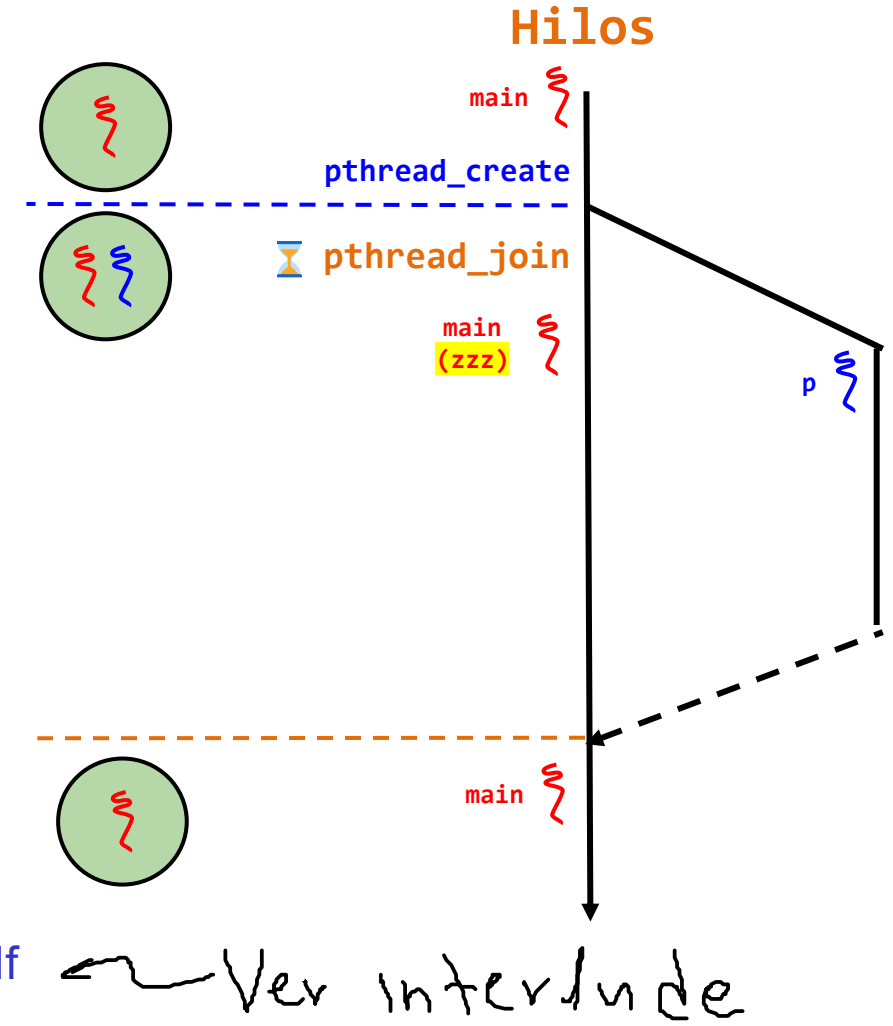
Trabajando con hilos

Ejemplo - Pasando un valor simple a un hilo

Simplemente pasando un único valor.

```c
void *mythread(void *arg) {
  int m = (int) arg;
  printf("%d\n", m);
  return (void *) (arg + 1);
}

int main(int argc, char *argv[]) {
  pthread_t p;
  int rc, m;
  pthread_create(&p, NULL, mythread, (void *) 100);
  pthread_join(p, (void **) &m);
  printf("returned %d\n", m);
  return 0;
}
```

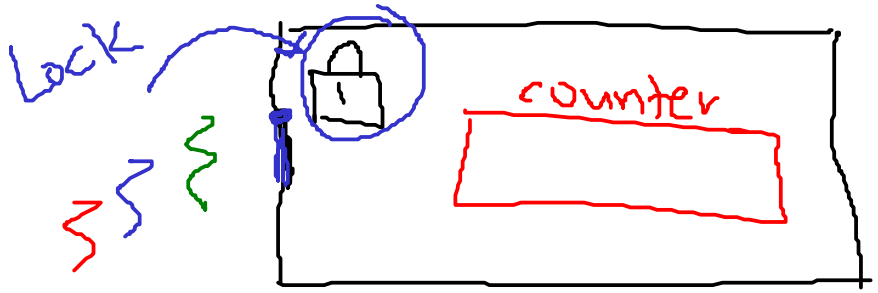


https://pages.cs.wisc.edu/~remzi/OSTEP/threads-api.pdf

Ver interlude

Locks

¿Para qué sirven?



Proporciona **exclusión mutua** a una **región crítica**

- **Interfaz**

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- **Uso** (sin inicialización , ni chequeo de errores)

```
pthread_mutex_t lock;
pthread_mutex_lock(&lock);
x = x + 1; // or whatever your critical section is
pthread_mutex_unlock(&lock);
```

- Si **no hay hilos bloqueando al lock** → **acceda a la región crítica**
- Si **hay un hilo bloqueando el lock** → el **hilo no retorna del llamado** hasta que pueda bloquear (adquirir) el lock

Locks

Inicialización

Todos los **locks** debe ser **adecuadamente inicializados**:

- **Una manera:** Usando **PTHREAD_MUTEX_INITIALIZER**

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

- **Forma dinámica:** usando la función **pthread_mutex_init()**

```
int rc = pthread_mutex_init(&lock, NULL);
assert(rc == 0); // always check success!
```

Locks

Verificación de errores

Chequeo de errores cuando se llame **lock** y **unlock**

- **Ejemplo usando una función *wrapper***

```
// Use this to keep your code clean but check for failures
// Only use if exiting program is OK upon failure
void Pthread_mutex_lock(pthread_mutex_t *mutex) {
  int rc = pthread_mutex_lock(mutex);
  assert(rc == 0);
}
```

Locks

Verificación de errores

Chequeo de errores cuando se llame **lock** y **unlock**

- **Otras funciones:** Las siguientes dos funciones se usan en la adquisición de locks

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);

int pthread_mutex_timelock(pthread_mutex_t *mutex,
                           struct timespec *abs_timeout);
```

- **trylock:** Error de retorno si el lock está adquirido (tomado).
- **timelock:** Retorna después de que se alcanza un tiempo.

Compilación y ejecución

Compilando aplicaciones con hilos

Para compilar se debe incluir explícitamente la librería **pthread.h** en el archivo

```
#include <pthread.h>
```

Se debe indicar al compilador que use la librería en tiempo de enlazado (***link***), a través del flag -pthread (-lpthread)

```
prompt> gcc –o main main.c –Wall -pthread
```

Conclusiones

Tópicos tratados

Al finalizar esta clase se espera que usted comprenda los siguientes términos:

- Hilos.
- Comportamiento determinístico y no determinístico.
- Variables compartidas.
- Critical section.
- Race condition.
- API Posix para hilos.

Referencias

- Página de Remzi H. Arpaci-Dusseau (link)
- Operating System Concepts - Tenth Edition (website)
- Operating Systems Notes (link)
- http://web.eecs.utk.edu/~mbeck/classes/cs560/560/oldtests/t2/2003/Answers.html
- https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md
- https://myaut.github.io/dtrace-stap-book/kernel/proc.html
- https://myaut.github.io/dtrace-stap-book/index.html
- https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml
- http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/
- https://courses.cs.duke.edu/spring19/compsci590.1/slides/
- http://csl.snu.ac.kr/courses/4190.307/2020-1/
- http://pages.cs.wisc.edu/~bart/537/lecturenotes/titlepage.html
- https://www.scs.stanford.edu/22wi-cs212/
- https://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html
- https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/5-Memory-Management.md

Referencias

- https://csapp.cs.cmu.edu/
- https://applied-programming.github.io/Operating-Systems-Notes/5-Memory-Management/
- http://www.cs.cmu.edu/~213/
- https://web.eecs.umich.edu/~akamil/teaching/sp04/
- https://courses.engr.illinois.edu/cs241/sp2014/
- https://emeryberger.com/teaching/grad-systems/
- https://oslab.kaist.ac.kr/courseware/
- https://oslab.kaist.ac.kr/ee488-fall-2020/
- https://github.com/JoeCameron1/CSC369-OperatingSystems-Assignments/tree/master/Assignment2
- http://www.cs.toronto.edu/~ylzhang/csc258/
- https://oslab.kaist.ac.kr/ee415-spring-2022/
- https://prepinsta.com/operating-systems/multiprocessor-operating-systems/
- https://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/definition.html
- https://www.quora.com/What-are-some-best-examples-of-multithreaded-applications
- https://cpentalk.com/986/examples-applications-benefit-directly-ability-throughput
- https://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/definition.html
- https://realpython.com/python-concurrency/
- https://applied-programming.github.io/Operating-Systems-Notes/5-Memory-Management/

Referencias

- https://developers.google.com/web/updates/2018/09/inside-browser-part1
- https://developers.google.com/web/updates/2018/09/inside-browser-part2
- https://www.informit.com/articles/article.aspx?p=25075&seqNum=3
- https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4_Threads.html
- https://realpython.com/python-sockets/
- https://realpython.com/intro-to-python-threading/
- https://realpython.com/python-concurrency/
- https://www.geeksforgeeks.org/socket-programming-multi-threading-python/
- https://eecs485staff.github.io/p4-mapreduce/threads-sockets.html
- https://prepinsta.com/operating-systems/inter-process-communication/
- https://applied-programming.github.io/Operating-Systems-Notes/6-Inter-Process-Communication/
- https://applied-programming.github.io/Operating-Systems-Notes/3-Threads-and-Concurrency/
- https://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/definition.html
- https://singa.apache.org/
- http://web.mit.edu/java_v1.0.2/www/tutorial/java/threads/deadlock.html
- http://www.cs.fsu.edu/~baker/realtime/restricted/notes/philos.html
- https://www2.seas.gwu.edu/~mfeldman/papers/portable-diners.html
- https://www.kalasim.org/examples/dining_philosophers/
- http://people.cs.aau.dk/~ask/Undervisning/MVP/html/deadlock.pdf

Referencias

- https://github.com/fab327/JavaFX_DiningPhilosophers
- https://github.com/djunderw/dining-philosophers
- https://www.geeksforgeeks.org/dining-philosopher-problem-using-semaphores/
- https://rosettacode.org/wiki/Dining_philosophers
- https://www.seas.upenn.edu/~cse39904/homework/hw06.pdf
- https://www.seas.upenn.edu/~cse39904/
- https://www.seas.upenn.edu/~cse39904/notes.html
- https://github.com/smonn/hungrythinkers
- https://github.com/jnafolayan/dining-philosopher
- https://jnafolayan.github.io/dining-philosopher/
- https://github.com/ZzCalvinzZ/dining_philosophers_web_workers
- https://dev.to/vunderkind/the-dining-philosophers-problem-or-your-introduction-to-thinking-in-computer-science-8ii
- https://github.com/lievi/dining_philosophers
- https://www.wikiwand.com/en/Dining_philosophers_problem
- https://adit.io/posts/2013-05-11-The-Dining-Philosophers-Problem-With-Ron-Swanson.html
- https://ruslanspivak.com/lsbaws-part1/
- https://ruslanspivak.com/lsbaws-part2/
- https://ruslanspivak.com/lsbaws-part3/